UniFLEX

COBOL

User's

Manual

TSC

MANUAL REVISION HISTORY

Revision	Date	Change
A	6/82	Original Release

Table of Contents

		Page
I.	Introduction	1-1
II.	COBOL Program Source Format	2-1
III.	COBOL Programming Elements	3-1
IV.	COBOL Programs and Files	4-1
V.	The IDENTIFICATION DIVISION	5-1
VI.	The ENVIRONMENT DIVISION	6-1
VII.	The DATA DIVISION	7-1
VIII.	The PROCEDURE DIVISION	8-1
IX.	Running the Compiler	9-1

Appendixes

A.	Glossary	A-1
B.	Compilation Error Messages	B-1
C.	Run-time Error Messages	C-1
D.	CRT Termcap Description	D-1
E.	Interactive COBOL Debug Tool	E-1
F.	COBOL Utilities	E-1
G.	Reserved Words	G-1
H.	Syntax Diagrams	H-1

I. INTRODUCTION

This manual describes Technical Systems Consultants' UniFLEX~ ANSI 1974 COBOL compiler and associated software. It is not intended to teach COBOL or programming, but rather to give the details of this particular COBOL implementation. (UniFLEX is a trademark of Technical Systems Consultants, Inc.)

The manual itself is laid out much as a COBOL program is. The first few chapters describe the COBOL programming environment. Then there are four chapters, each dealing with a specific DIVISION of COBOL programs. Then there is a chapter describing how to run the compiler. Finally, there are a set of appendices which describe the support software as well as giving concise lists of error messages and terms used by the compiler system.

Technical Systems Consultants' ANSI COBOL system is very complete, containing most level I features and many level II features of the COBOL language. In particular, our COBOL includes level II SORT/MERGE, level II REPORT WRITER and INDEXED SEQUENTIAL files which have most level II features. Also, we have incorporated the screen oriented input/output functions already used by two other COBOL systems manufacturers (NCR and Data General) giving users of our COBOL system screen oriented input/output compatible with either of those manufacturers programs.

Portions of this manual have been taken from the ANSI COBOL Standard document, X3.23-1974. We gratefully acknowledge permission for this privileged.

II. COBOL PROGRAM SOURCE FORMAT

The COBOL compiler accepts a slightly simplified source program format from that given in the ANSI specification. Our format is adapted to the interactive environment in which our compiler runs. There are no strict margin rules and basically, any source item which begins in column 2 may begin in any column past 2. Column 1 is reserved for the comment indicator.

Comment lines

Column 1 of every source line is a comment indicator. Any line which has any of the characters "*", "/", "D" or "d" in column 1 is considered to be a comment. Comments appear in the source listing, but are totally ignored by the compiler. If the "+d" option is being used during the compilation, lines which contain "d" or "D" are treated as though they had a space in the comment indicator. If the comment indicator contains the character "/", then the listing continues on a new printed page. This mechanism is equivalent to the "EJECT" command in other COBOL compilers.

All other fields

As mentioned above, there are no strict margin rules in this compiler. All source items which are not comments must begin in column 2 or greater, but beyond that, the program is free format. Spaces are totally ignored by the compiler except inside of literal strings. This means that existing programs which use the standard COBOL margins should compile with little or no trouble. One feature of standard COBOL margins which is not supported is that of continuations. This is particularly relevant with literal strings which are continued. Lines which need to be continued can most often simply be broken into separate lines.

III. COBOL PROGRAMMING ELEMENTS

This chapter describes the notation and terms used in the rest of the manual.

Meta-Language Elements

The rest of this manual is written using a simple meta-language.

Uppercase Words

Uppercase words which appear in this manual are reserved words. They may be spelled in either upper or lower case in actual COBOL programs. Reserved words must be spelled exactly as they are shown in the manual. Words which are underlined are required in the given context. Words which are not underlined may be left out at the programmer's option.

Lowercase words

Lowercase words are terms used to describe items present in a COBOL program. These items may be identifiers, PICTURE strings, literals, etc. The specific lowercase word will indicate what type of item is being described. The glossary (appendix A) contains a list of these words along with an explanation of their meaning.

Brackets []

Square brackets are used to indicate an optional item or phrase. Anything inside the brackets may be left out at the programmer's option, but must be written according to the rules if present.

Braces

Curly braces are used to indicate a choice between two or more items within the braces.

Elipsis ...

An elipsis is used to indicate that a given item may be present more than once in the given construct. If this is an item in brackets or braces, scan from the elipsis left to find the corresponding bracket or brace. Everything inside that pair may then be repeated one or more times.

Comma and Semicolon

Some statements specify either of these punctuation marks. They are equivalent (a comma may be used where a semicolon is written and vice-versa) and completely optional.

Level Numbers

Data items in the DATA DIVISION have level numbers associated with them. These numbers must be in the range 01 to 49, 77 or 88. The number 01 is equivalent to 1.

COBOL Elements

COBOL programs are composed of sentences made from characters, words and statements. The different classes of items making up a COBOL program are described below:

Character Set

COBOL programs may be written using the entire [printable] ASCII character set. This includes the upper and lower case letters 'A' through 'Z' and 'a' through 'z', the digits '0' through '9' and the special characters '"()=-+*;<>,. '. Characters not in this set may appear in comments and literal strings but must not appear in other COBOL elements.

Separators

COBOL program items may be separated by any of the following: a space, colon, semicolon or period. Whenever a space is used, more than one space may be present. The characters colon, semicolon and period are separators only if they are followed by a space. There is also an implicit space at the end of every source line.

COBOL Words

COBOL programs also include words, some of which are reserved or keywords. Words in COBOL, also referred to as identifiers, may contain any number of characters, digits and hyphen ('-') characters. Only the first 30 characters in a word are significant. Keywords may be spelled in any case, including mixed case. This means that the word "Identification" is the same word as "IDENTIFICATION" and also the same as "identification". For a complete list of the keywords in COBOL, see Appendix F. Identifiers must begin with a letter and may include any letter 'a'-'z', 'A'-'Z', or digit '0'-'9' or the hyphen character '-'. Paragraph and section names may begin with a digit. The hyphen character must not be the last character in an identifier. The case of letters in identifiers which are not keywords is significant, i.e. the identifier "Harry" is not the same identifier as "harry" or "HARRY".

Examples:
Legal identifiers:
```
   serial-data-base
   rewrite-error-100
```
Illegal identifiers:
```
  non-serial-data-base-
  periods-not-allowed.in-identifiers
```

Figurative Constants

Several keywords represent special data values known as figurative constants. These constants may be used anywhere a literal value may appear in a COBOL program. The figurative constants and their values are:

Keyword	Value
ZERO, ZEROS, ZEROES	The character "0" or the numeric value 0.0
SPACE, SPACES	The character " "
HIGH-VALUE, HIGH-VALUES	The largest possible value
LOW-VALUE, LOW-VALUES	The lowest possible value
QUOTE, QUOTES	The character '"'
ALL literal	The specified literal, repeated as many times as makes sense in the expression

Each of these values represents one or more occurrences of the specific value, repeated as many times as the expression in which it is used dictates.

Non-numeric literals (strings)

Non-numeric values are represented by strings of characters enclosed in quotation marks. Either the quotation mark '"' or the apostrophe "'" may enclose a string. Any printable character in the ASCII character set may be used in a string. Spaces, which are normally not significant, are significant within strings. If it is necessary to represent a quotation mark within the string either the other quotation symbol may be used (i.e. use '"' if "'" is required in the string), or the quotation mark may appear twice consecutively within the string to represent a single occurrence of that quotation mark. For example, the string '"' may be represented as """".

Examples:
Legal non-numeric literals:

```
   'This is a legal character string'
   'This string has a quote "'" in it'
```

Illegal non-numeric literals:

```
   'This string is not terminated properly
   "This string has the wrong number of quotation marks '"'"
```

Numeric literals

Numeric literals are representations of numeric values within a COBOL program. They may contain the digits '0'-'9', the minus sign '-', the plus sign '+', and the decimal point '.'. The sign character must be the leftmost character present and the decimal point may not be the rightmost character of a numeric literal.

Uniqueness of Reference

Qualification

Every user-specified name that defines an element of a COBOL source program must be unique, either because no other name has the identical spelling and hyphenation, or because that name exists within a hierarchy of names such that references to the name can be made unique by mentioning one or more higher levels of the hierarchy. The higher levels are called qualifiers and this process that specifies uniqueness is called qualification.

Qualification is performed by following a data name by one or more phrases composed of a qualifier and the keyword IN or OF. The general format for qualification is:

```
data-name   IN   data-name   ...
            OF
```

The rules for qualification are:

Each qualifier must be of a successively higher level and within the same hierarchy as the name it qualifies.

The same name must not appear at two levels in the hierarchy.

If a data-name is assigned to more than one data item in a program, then that data-name must be qualified every time it is referred to in the program, except in a REDEFINES clause where qualification is not allowed.

A data-name may not be subscripted when it is being used as a qualifier. This does not preclude subscripts from being qualified, however.

A name may be qualified even though no qualification is needed.

For an example of qualification, consider the hierarchies:

```
01 hierarchy-1  ...          01 hierarchy-2  ...
   02 xxx   ...                 02 xxx  ...
   02 yyy   ...                 02 yyy  ...
   02 zzz   ...                 02 aaa  ...
```

In this example, the "xxx" and "yyy" fields of the two hierarchies must be qualified in order to specify which field is actually being referred to. Thus "xxx of hierarchy-1" is unique from "xxx of hierarchy-2". Also notice that "zzz of hierarchy-1" does not need to be qualified since there is no data name "zzz" in "hierarchy-2".

More details on hierarchies and level numbers and their significance will be found in chapter 7.

_ul Subscripting / Indexing

The ANSI COBOL standard document defines subscripting and indexing of elementary data items. In this implementation, subscripting and indexing are treated equivalently.

Subscripts are used to reference an individual element within a table defined using the OCCURS clause. A subscript can be represented by an integer numeric data item or an integer numeric literal, or their sum or difference. When a data name is used, the data name may be qualified, but not subscripted.

The subscript value may be signed, but must always be positive. The lowest permissible subscript value is 1. This value accesses the first element in the table. The next sequential elements of the table are accessed by subscript values of 2, 3, ... The highest possible subscript value is specified in the OCCURS clause which defines the table.

If the table being referenced requires more than one subscript to access an elementary data item, then the subscripts must be written in order of successively less inclusive dimensions in the data organization.

The basic format for subscripted references is:

```
   data-name  (  data-name  +  literal  ,  data-name  +  literal  ... )
                            -                        -
                 literal                  literal
```

No more than 3 levels of a table may be defined, so even the format above implies a large number of subscripts possible, no more than three subscripts are allowed.

Sentences and Statements

COBOL programs are made up of three types of statements: imperative, conditional and compiler directives.

Imperative statements

An imperative statement is a statement which indicates a specific unconditional action to be taken by the program. The following verbs make up imperative statements:

ACCEPT	DISPLAY	MULTIPLY}	START}}
ADD}	DIVIDE}	OPEN	STOP
CALL}	EXIT	PERFORM	SUBTRACT}
CLOSE	GO	READ}}}	UNLOCK
COMPUTE}	INSPECT	REWRITE}}	WRITE}}
DELETE}	MOVE	SET	

(}) Without the optional SIZE ERROR phrase
(}}) Without the optional INVALID KEY phrase
(}}}) Without the optional INVALID KEY or AT END phrase

Conditional Statements:

Any statement which tests a condition and may take different paths depending on the result of the test is known as a conditional statement. The following verbs make up conditional statements:

ADD}	IF	START}}
COMPUTE}	MULTIPLY}	SUBTRACT}
DELETE}	READ}}}	WRITE}}
DIVIDE}	REWRITE}}	

(}) With the optional SIZE ERROR phrase
(}}) With the optional INVALID KEY phrase
(}}}) With the optional INVALID KEY or AT END phrase

Compiler Directives:

There are two statements which are not executed but rather direct the compilation process. These are the COPY and USE statements. The COPY statement is used to include the text of a file into the COBOL program currently being compiled. The USE statement associates a program section with specific asynchronous events in the program. For more details on these statements, refer to chapter 8.

Conditional Expressions

Conditional expressions represent conditions to be tested during the running of a COBOL program. The result of this test is either true (the condition exists as stated) or false. This truth value may then be used to control the execution of the program.

Simple Conditions:

Simple conditions are any of relational conditions, class conditions, sign conditions or switch values. Refer to chapter 6 for more detail on switches.

Complex Conditions

Complex conditions are made up of combinations of simple conditions.

Relation Conditions

Relation conditions test the logical relationship between two data values. The data values may be given as data names, literal values or arithmetic expressions. The general format for relation conditions is:

```
data-name              IS  NOT  GREATER THAN  data-name
literal                IS  NOT  LESS THAN     literal
arithmetic expression  IS  NOT  EQUAL  TO     arithmetic expression
                       IS  NOT  >
                       IS  NOT  <
                       IS  NOT  =
```

The relational operator specifies the type of comparison to be made. When used, "NOT" and the next keyword or relation character are one relational operator that defines the comparison to be executed; e.g. "NOT EQUAL" is a test which is true if the operands are not equal.

When comparing numeric operands, the comparison is made with respect to their algebraic value.

When comparing non-numeric operands, or one numeric operand with a non-numeric operand, the comparison is made with respect to the machine's collating sequence. If one of the operands is numeric, it must be an integer data item or an integer literal. If the two operands are not the same length, the comparison is made as if the shorter operand is padded on the right with the appropriate number of spaces.

Class Condition

The class condition tests whether a given operand has a numeric or alphabetic value. For the NUMERIC test, any character other than a digit or sign character will cause the result to be false. For the ALPHABETIC, all characters must be letters, either upper or lower case, for the result to be true. The basic format for class conditions is:

```
data-name  IS  NOT  NUMERIC
                    ALPHABETIC
```

The data item to be tested must have USAGE DISPLAY. The NUMERIC test can only be used with numeric or alphanumeric data items.

Sign Condition

The sign condition determines the algebraic sign of a value. The data item to be tested must be a numeric data item. The basic form for the sign condition is:

```
                                  POSITIVE
   arithmetic expression  IS  NOT  NEGATIVE
                                  ZERO
```

The data item must have a value greater than zero to be POSITIVE, or less than zero to be NEGATIVE.

Combined Conditions

Conditions may be combined by using the boolean operations AND and OR. The AND of two conditions is a new condition which is only true if both the constituent conditions are true, false otherwise. The OR of two conditions is a condition which is true if either of the constituent conditions is true and false only if both the conditions are false. Conditions may have their truth value negated by using the boolean operator NOT. The general format for a negated condition is:

```
   NOT  simple condition
```

The general format for a combined condition is:

```
   condition  AND  condition  ...
              OR
```

File Status Codes

Whenever an I/O operation takes place in a COBOL program, the I/O control system returns a code which represents the success or failure of the operation. This is a two byte alphanumeric code which is placed in the FILE STATUS data item for the file which the I/O operation was for. The possible values for the file status codes are listed below, along with a short explanation of each.

CODE	MEANING
00	Successful I/O operation.
02	Duplicate key created/read.
10	AT END condition.
21	(INVALID KEY) Sequence error. A file with SEQUENTIAL access mode was not accessed sequentially.
22	(INVALID KEY) Duplicate key error.
23	(INVALID KEY) Record not found.
24	(INVALID KEY) Boundary error. This error occurs if the COBOL runtime system cannot handle the size of the file, i.e. the file is too large. Currently, the error is possible for an INDEXED file with more than 4**10 (1,048,576) records, or for a RELATIVE file with more than 32768 records. Contact Technical Systems Consultants for help with this error.
91	OPEN error. The file is not properly opened, or an OPEN statement is executed on an already open file.
92	Access mode error. The file open mode (INPUT, OUTPUT, etc) does not allow the specified operation. For example, a WRITE operation on a file open in INPUT mode.
93	Record is locked. The record being accessed is currently locked by another program. Depending on the application, the program may wish to wait and retry the operation later.
94	Inconsistent descriptor. The data file for an INDEXED file does not correspond with the definition of the file in the FILE-CONTROL paragraph. This may be that the number of keys is different, the keys are not defined in the same fashion, etc.
95	File organization error. The actual data file specified does not match the file organization as defined for the file in the FILE-CONTROL paragraph.
96	File corrupted. The contents of an INDEXED file have become damaged. This file should not be used, but may be recreated using the "vsamcopy" utility.

CODE	MEANING
97	Inconsistent parameters. The file specification in the COBOL program does not make sense to the COBOL runtime package. This error should be brought to the attention of Technical Systems Consultants.
98	Current record is undefined. No current record is defined for a READ ... NEXT operation.
99	Record length error. A REWRITE operation on a SEQUENTIAL file specified a record longer than the record previously read. The record to be written on a SEQUENTIAL or RELATIVE file exceeds the maximum record length.
9A - 9Z	UniFLEX I/O error. Code "9A" is UniFLEX error code 1, "9B" is error code 2, etc.

IV. COBOL PROGRAMS AND FILES

The COBOL programming environment includes COBOL programs and the data they work on. These data may reside in the computer's main memory in which case they are referred to as data items, or they may reside on some external medium, most often referred to as a data file.

Chapter 7 gives a complete description of the various data items which can be declared and used in a COBOL program.

The COBOL programming environment supports three organizations of data files: SEQUENTIAL, RELATIVE and INDEXED. Each file organization has certain characteristics specific to that organization and thus may be better suited for a given application than the other file organizations. The remainder of this chapter will describe each of these file organizations, how they are used and the implications of their use.

SEQUENTIAL files are the simplest file organization. Records in SEQUENTIAL files have an implicit ordering. This ordering is given by the order in which the records are written to the file. When writing a SEQUENTIAL file, records are appended, one at a time, starting with the first data record, to the file. When reading a SEQUENTIAL file, records are retrieved in exactly the same order they were written. SEQUENTIAL files must always be read from the beginning. A SEQUENTIAL file may have new records appended to the end of the file by using the EXTEND file access mode rather than rewriting the entire file.

The other two file types, RELATIVE and INDEXED files, organize the data records in a [possibly] random fashion. The ordering of data records in these files is given by an access key rather than the order in which the records are written. For RELATIVE files, the access key is an integer value between 0 and 32767. This value is known as the record number for the given data record and is established when the record is written. The data record may later be read or rewritten using the record number to access the record. Records in a RELATIVE files do not need to have consecutive record numbers, nor do the records necessarily need to be written in ascending record number value. This allows records to be written in a totally random sequence and then retrieved either randomly or sequentially. Records which have never been written simply do not exist in the data file and do not take up any space on the external medium.

INDEXED files also allow data records to be organized based on an access key. The access key may be any string of characters which is a part of the data record itself. This key is known as a record key and may contain any number of characters between 1 and 127. All the record key values in a given file must be unique since this is the mechanism for accessing the data records. Records may be accessed directly using the primary key value or they may be accessed sequentially (in ascending primary key order) by using the READ ... NEXT statement.

INDEXED files may also be accessed by secondary record keys. This allows a single data record in the file to be accessed by two different keys, either the primary record key or by a secondary record key. In fact, INDEXED files may have up to 4 secondary or alternate record keys besides the primary record key. Unlike the primary record key, alternate key values do not have to be unique with the file. If this is the case, then all the data records in the file which have a particular value for a specific alternate record key are grouped together. Each of these data records may be accessed in turn by using the READ ... NEXT statement which will access each of the data records which has the given value for the alternate key in the sequence in which they were written into the file. In this sense, the alternate key values make up miniature sub-files within the total INDEXED file which can be accessed sequentially.

Data files in a COBOL program are defined in two different portions of the program. The first place a file must be mentioned is in the FILE-CONTROL paragraph of the INPUT-OUTPUT SECTION of the ENVIRONMENT DIVISION. In this paragraph, the file is described in terms of its organization, how the program intends to access it, and the external file that corresponds to the COBOL file. The description of the access key(s) for the file are also given in this paragraph. Refer to chapter 6 for more detail on the FILE-CONTROL paragraph.

The second location in the COBOL program that files are defined is in the FILE SECTION of the DATA DIVISION. This section defines, for each file in turn, the structure of the data records which appear in the file. More detail on the FILE SECTION can be found in chapter 7.

V. THE IDENTIFICATION DIVISION

The IDENTIFICATION DIVISION must be included in every COBOL source program. This division identifies the source program and the resultant program listing. In addition, the user may include the date the program was written and such other information as comments about how the program is to be run, etc. The basic format of the IDENTIFICATION DIVISION is:

```
    IDENTIFICATION DIVISION.

    PROGRAM-ID.  program-name.

   [AUTHOR.         [comment-entry] ...]

   [INSTALLATION.   [comment-entry] ...]

   [DATE-WRITTEN.   [comment-entry] ...]

   [DATE-COMPILED.  [comment-entry] ...]

   [SECURITY.       [comment-entry] ...]
```

Example of IDENTIFICATION DIVISION Paragraph:

```
         IDENTIFICATION  DIVISION.
         PROGRAM-ID.   Data-base-maintenance.
         AUTHOR.  Sam The Programmer.
         INSTALLATION.  Applications by Sam, Inc.
         DATE-WRITTEN.  May 1, 1982.
         DATE-COMPILED. May 1, 1982.
         SECURITY.  This program allows an operator to add/change/review
                    entries in the master data-base.
```

A "comment-entry" is any string of symbols, terminated by a period. The "program-name" is only used for documentation purposes and may be any legal COBOL identifier. All entries in the IDENTIFICATION DIVISION are for documentation purposes only and are simply listed in the output listing as entered. The DATE-COMPILED entry will be followed by a line in the listing [redundant] which contains the date and time the program was compiled.

VI. THE ENVIRONMENT DIVISION

The ENVIRONMENT DIVISION must follow the IDENTIFICATION DIVISION in every COBOL program. This division defines the environment external to the COBOL program. Such things as the files the COBOL program uses, the machine it is designed for, etc., are present in the ENVIRONMENT DIVISION. The basic format of the ENVIRONMENT DIVISION is:

```
ENVIRONMENT DIVISION.

CONFIGURATION SECTION.

SOURCE-COMPUTER.  computer-name [WITH DEBUGGING MODE] .

OBJECT-COMPUTER.  computer-name

                                WORDS
    ,  MEMORY  SIZE  integer    CHARACTERS   .
                                MODULES

[SPECIAL-NAMES.

   SWITCH  literal  ,  ON   STATUS  IS  condition-name
                    ,  OFF  STATUS  IS  condition-name

  [, CURRENCY  SIGN  IS  literal]

  [, DECIMAL-POINT IS  COMMA] .]


INPUT-OUTPUT SECTION.

FILE-CONTROL.
   File Control Entry  ...


I-O-CONTROL.

          RECORD
  SAME    SORT          AREA  FOR  file-name   , file-name
          SORT-MERGE
```

Any valid COBOL identifier may be used for "computer-name" in the SOURCE-COMPUTER and OBJECT-COMPUTER phrases. These phrases are for documentation purposes only.

If the CURRENCY SIGN phrase is present, then the specified literal value replaces the meaning on the normal currency sign ($). This is true during the compilation of the program and at runtime as well. The literal value must be a single character and not any of the following:

' 0123456789ABCDLPRSVXZ*+-,.()"/='

If the DECIMAL-POINT IS COMMA phrase is present, then both during compilation and at runtime the decimal point "." and the comma character "," exchange their normal meaning in PICTURE strings and data values. This clause does not alter the interpretation of the period or comma characters to the compiler.

The SWITCH phrase provides a simple mechanism to pass runtime parameters to COBOL programs. When the COBOL program is invoked, single character options may be present or absent on the command line. The state of these options is reflected in the variables mentioned in the SWITCH phrase. If the option is present then the variable mentioned in the ON STATUS clause will be true. If the option is not present then the variable mentioned in the OFF STATUS clause will be true.

The FILE-CONTROL paragraph contains one or more SELECT statements. These SELECT statements define the files the COBOL program uses and creates an association between COBOL files and the UniFLEX files they represent. The formats for SELECT statements follow:

Sequential SELECT:

```
                                              DISC
                                              STD-IN
   SELECT  [OPTIONAL]  file-name   ASSIGN  TO STD-OUT
                                              literal
                                              file-name-variable

      ;  RESERVE  integer  AREA
                            AREAS

     [;  ORGANIZATION  IS  SEQUENTIAL]

     [;  ACCESS  MODE  IS  SEQUENTIAL]

     [;  FILE  STATUS  IS  data-name]   .
```

Relative SELECT:

```
                                DISC
                                STD-IN
   SELECT  file-name ASSIGN  TO STD-OUT
                                literal
                                file-name-variable

      ;  RESERVE  integer  AREA
                           AREAS

      ;  ORGANIZATION  IS  RELATIVE

      ;  ACCESS  MODE  IS  SEQUENTIAL  ,  RELATIVE  KEY  IS  data-name

                           RANDOM
                           DYNAMIC     ,  RELATIVE  KEY  IS  data-name

      ;  FILE  STATUS  IS  data-name    .
```

Indexed-Sequential SELECT:

```
                                DISC
                                STD-IN
   SELECT  file-name ASSIGN  TO STD-OUT
                                literal
                                file-name-variable

      ;  RESERVE  integer  AREA
                           AREAS

      ;  ORGANIZATION  IS  INDEXED

                           RANDOM
      ;  ACCESS  MODE  IS  SEQUENTIAL
                           DYNAMIC

      ;  RECORD  KEY  IS  data-name

      ;  ALTERNATE  RECORD  KEY  IS  data-name  WITH  DUPLICATES

      ;  FILE  STATUS  IS  data-name    .
```

The SELECT statement allows the COBOL program to define the characteristics of the files to be used in the program. First, this statement allows the COBOL programmer to specify the file organization mode desired. Three file organizations are provided: SEQUENTIAL, RELATIVE, and INDEXED.

SEQUENTIAL files provide a method for reading or writing records from/to a file in a specific order. SEQUENTIAL files are written

one record at a time and are read one record at a time in the order

they were written.

RELATIVE files allow a file to be created where each record in the file has a unique record number. This record number, given by the RELATIVE KEY phrase in the SELECT statement, is specified when the record is written. The data item specified in the RELATIVE KEY phrase must be a numeric data item defined in either the WORKING-STORAGE or LINKAGE section of the COBOL program. The record may then be read by specifying its record number. The file access modes RANDOM and DYNAMIC allow the COBOL program to read/write records in any order desired. Records in RELATIVE files may also be rewritten using the REWRITE verb.

INDEXED files are similar to RELATIVE files in that each record has a unique name. In the case of INDEXED files, this name is called a key and may contain an arbitrary number (<127) of alphanumeric characters. INDEXED files may also specify secondary keys using the ALTERNATE RECORD KEY phrase. A given record in the file may then be accessed using either the primary key or any one of the alternate keys. Up to four (4) alternate keys may be specified for a given INDEXED file. The data items specified as keys for INDEXED files must be alphanumeric data items defined within one of the data records associated with the file.

The SELECT statement also may specify the way in which the file is to be accessed. Three access modes are provided: SEQUENTIAL, RANDOM, and DYNAMIC.

A file in SEQUENTIAL access mode may only be read/written in the primary record sequence for that file. This is the only possible access mode for SEQUENTIAL files, but may also be specified for RELATIVE and INDEXED files also. For a RELATIVE file, records are accessed by increasing record number. For INDEXED files, records are accessed by increasing value of the primary key, as given by the collating sequence on the machine.

A file in RANDOM access mode may be read/written in any order specified by the COBOL program. In this mode, each record must be accessed by some key value, either a record number for RELATIVE files or a key for INDEXED files.

Files may also be accessed dynamically using the DYNAMIC access mode. In this mode, records may be accessed directly by using a key value as in RANDOM access mode, or they may be accessed sequentially using the START and READ ... NEXT statements as if they were in SEQUENTIAL access mode.

The ASSIGN phrase in the SELECT statement specifies the association between the COBOL file and the name of the UniFLEX file it represents. This association may be given directly in the ASSIGN phrase using a literal value, or it may be deferred to runtime by specifying that the actual file name to use is given by a variable. In this latter case, when the file is opened, the name of the file to be associated with the COBOL file is determined by the value of the variable used in the ASSIGN phrase. If a variable file name is used, it must represent an alphanumeric data item declared in the WORKING-STORAGE or LINKAGE section of the program. If the keyword DISC is specified in the SELECT statement, then the COBOL file will be associated with a UniFLEX file with the same name as the COBOL file name. The keywords STD-IN and STD-OUT allow the COBOL file to be associated with the standard input and standard output files, respectively.

The file name specified by a SELECT statement may represent an actual UniFLEX file or, if the file is to be opened for writing, may represent a program to which the data is "piped". This mechanism is specified by having the first character of the file name be the pipe symbol ' ' or ' '. If this is the case then the remainder of the file name is expected

to be a command string used to execute the program which is used to receive the data. The parameters of the command string are passed to the program. However, no command line substitutions will be performed (i.e. the '*' and '?' characters will not be expanded). This mechanism is useful for sending output files to spoolers, etc.

The FILE STATUS phrase in the SELECT statement allows the programmer to specify a variable whose value will reflect the state of the file. This variable must be an alphanumeric data item with exactly two characters (PICTURE XX) defined in either the WORKING-STORAGE or LINKAGE section of the program. For more details on the possible values of the FILE STATUS item, refer to chapter 3.

Example of ENVIRONMENT DIVISION paragraph:

```
      ENVIRONMENT DIVISION.
      CONFIGURATION SECTION.
      SOURCE-COMPUTER.  Motorola-6809.
      OBJECT-COMPUTER.  Motorola-6809.
      SPECIAL-NAMES.
        SWITCH "1", ON STATUS IS switch-1.
      INPUT-OUTPUT SECTION.
      FILE-CONTROL.
        SELECT seq-file ASSIGN TO "/data-base",
          ORGANIZATION IS INDEXED,
          RECORD KEY IS primary-serial-number of seq-file-record,
          FILE STATUS IS seq-file-status.
        SELECT sort-file ASSIGN TO DISC.
```

VII. THE DATA DIVISION

The DATA DIVISION describes the data a COBOL program uses, either as input, internally or as output. The DATA DIVISION is composed of five sections: the FILE SECTION, the WORKING-STORAGE SECTION, the LINKAGE SECTION, the SCREEN SECTION, and the REPORT SECTION. The DATA DIVISION must follow the ENVIRONMENT DIVISION in a COBOL program and the different sections of the DATA DIVISION must appear in the order mentioned above.

The general format for the DATA DIVISION is:

```
   DATA DIVISION.

   FILE  SECTION.
   FD Entry  ...
   SD Entry  ...

   WORKING-STORAGE  SECTION.
      77 - Level Description Entry    ...
      Record  Description  Entry

   LINKAGE  SECTION.
      77 - Level Description Entry    ...
      Record  Description  Entry

   SCREEN  SECTION.
     Screen  Group  Description  Entry   ...

   REPORT  SECTION.
     RD Entry  ...
```

FILE SECTION

The FILE SECTION of a COBOL program describes the files which are used by the program. This description is in two parts. The first part is a file description heading paragraph which describes some basic features of the file. There are two forms of this paragraph; the FD paragraph is used to describe normal files; the SD paragraph is used to describe files used internally in the SORT/MERGE process. The FD or SD paragraphs must be followed by one or more data structure definitions which describe the actual data records within a file. More than one structure definition may be given for those files which have more than one structuring of the data within the file. FD paragraphs which contain the REPORT IS phrase do not require such a record definition although the RECORD CONTAINS phrase must be present if no record definition is given.

The basic format for the FD and SD paragraphs is:

```
   FD  file-name

      ;  BLOCK  CONTAINS  integer   TO   integer   RECORDS
                                                   CHARACTERS

      ;  RECORD  CONTAINS  integer  TO  integer  CHARACTERS

      ;  LABEL  RECORD   IS     STANDARD
                RECORDS  ARE    OMITTED

      ;  DATA   RECORD   IS     data-name

      ;  LINAGE  IS  integer  LINES
         ,  WITH  FOOTING  AT  integer
         ,  LINES  AT  TOP  integer
         ,  LINES  AT  BOTTOM  integer

      ;  REPORT  IS  report-name

      Record  Description  Entry


   SD  file-name

      ;  RECORD  CONTAINS  integer  TO  integer  CHARACTERS

      ;  DATA   RECORD   IS     data-name

      Record  Description  Entry
```

There must be one FD or SD paragraph for every file which is mentioned in a SELECT statement in the ENVIRONMENT DIVISION. No file may be named in more than one FD or SD paragraph, nor may any file be named in both an FD or SD paragraph.

The BLOCK CONTAINS, LABEL RECORDS and DATA RECORD clauses are not used by the compiler and may be included for documentation purposes only.

The RECORD CONTAINS clause defines the length of the maximum record for the file. If omitted, the data record structure definitions will define the record size implicitly. This clause must be included for any file which does not have a data record structure definition associated with the FD paragraph.

The REPORT IS clause associates a file with a report defined in the REPORT SECTION. All operations on that report will implicitly reference the given file.

The LINAGE clause is used to define the physical page structure of a file which is possibly to be printed on a hard copy device. This clause defines the page in terms of a number of lines on a page. The logical page is the sum of all the phrases except the FOOTING phrase. The FOOTING phrase must reference a line between 1 and the number of lines on the page. If the FOOTING phrase is not specified, the value is assumed to be the value given in the LINAGE IS phrase. If the LINES AT TOP or LINES AT BOTTOM phrases are not given, these values are assumed to be zero.

The LINAGE and REPORT IS clauses must not both be specified for a given file.

_ul Data Structure Descriptions

Data items in the FILE, WORKING-STORAGE and LINKAGE sections are described using a data structure definition. The format is the same for each section except that the VALUE clause may not appear in the FILE or LINKAGE sections. These structure definitions describe data items in terms of a hierarchy. The hierarchy is described through the use of level numbers.

The basic format for a data structure definition is:

```
   level-number  FILLER
                 data-name

      ;  REDEFINES  data-name

      ;  PICTURE  IS  picture-specification
         PIC
                    COMPUTATIONAL
                    COMP
                    COMPUTATIONAL-1
      ;  USAGE  IS  COMP-1
                    COMPUTATIONAL-3
                    COMP-3
                    DISPLAY
                    INDEX

      ;  SIGN  IS   LEADING    SEPARATE   CHARACTER
                    TRAILING

      ;  OCCURS  integer  TIMES   INDEXED  BY  index-name

      ;  SYNCHRONIZED   LEFT
         SYNC           RIGHT

      ;  JUSTIFIED      RIGHT
         JUST

      ;  BLANK  WHEN  ZERO

      ;  VALUE  IS  literal  .
```

Level Numbers

Each data item, either elementary data items or group data items, has an associated level. The lower the level number, the more encompassing the level. Level numbers may be any value from 01 through 49 or the special values 77 or 88. Level 77 entries are those data items which have no direct relationship with any other data items (structurally). Level 77 entries are only allowed in the LINKAGE and WORKING-STORAGE sections. Level 88 entries are used to define condition names. More about level 88 entries later.

Data name / FILLER

The data name or FILLER clause must follow the level number immediately. This clause gives a name to the data entry being described. For those entries that will never need to be referenced by name, the keyword FILLER may be used. All other data names must be unique or qualifiable to a unique name.

Redefines

The REDEFINES clause allows the same memory area to be described by different data formats.

The REDEFINES clause, if present, must appear immediately after the data item name.

The level numbers of the data item being defined and the data item being refined must be the same. Furthermore, no data items of a higher level (lower level number) may have been defined between the REDEFINES clause and the data item it references.

The data description of the data item being redefined must not include a REDEFINES clause, i.e. the REDEFINES clause may not redefine a data item which itself is defined by a REDEFINES. Also, the data item being redefined may not contain an OCCURS clause.

Redefinition begins with the specified data item and ends when a level-number less than or equal to that of the data item is encountered.

Multiple redefinitions of the same data description entry are allowed, but each new redefinition must reference the original entry.

The entries which are subordinate to the REDEFINES clause must not contain any VALUE clauses.

The size of the REDEFINES data structure and the data structure being redefined must be the same.

Picture

The PICTURE clause describes the general characteristics and editing requirements of an elementary item. The PICTURE clause may only appear at the elementary level and may be abbreviated using the keyword PIC. The PICTURE string is a string of 30 or fewer characters selected from the set "ABPSVXZ90/,.+-CRDB*$", as outlined below. Lower case letters may be used in picture strings in any position where the corresponding upper case letter is allowed. Also, repeated occurrences of the same character may be written as a single occurrence of the character, followed by the number of desired occurrences as an integer literal enclosed in parentheses. For example, the picture string "XXXXX" may be written as "X(5)".

A picture string describes the category of a data item. There are five possible data categories: alphabetic, alphanumeric, numeric, numeric edited and alphanumeric edited. Each of these categories is uniquely defined by a specific combination of picture string characters.

Alphabetic: Only the symbols "A" and "B" are allowed. This category represents data items which may contain only letters, Either upper or lower case, and the space character.

Numeric: Only the symbols "9", "P", "S" and "V" are allowed. This category represents data items which contain numeric values. The contents of such a data item must be a combination of the Arabic numerals "0", "1", "2", "3", "4", "5", "6", "7", "8", and "9". If signed, the item may also contain a "+" or "-" sign character, or one of the combined sign characters "a" through "i" (refer to the SIGN clause).

Alphanumeric: Only the symbols "A", "X" and "9" are allowed. The character string is treated as if it contains only the symbol "X". This category of data item may contain any printable character from the ASCII character set.

Alphanumeric edited: Only the symbols "A", "X", "9", "B", "0", "/" are allowed. The picture string must contain at least one "B" and at least one "X" or at least one "0" and at least one "X" or at least one "/" and at least one "X", or the picture string must contain at least one "0" and at least one "A" or at least one "/" and at least one "A". This category of data item may contain any combination of printable ASCII characters.

Numeric edited: Only certain combinations of the symbols "B", "/", "P", "V", "Z", "0", "9", ",", ".", "*", "+", "-", "CR", "DB", "$" are allowed. The picture string must contain at least one "9" or "Z" and at least one of the edit symbols "B", "/", "0", ",", ".", "+", "*", "-", "CR" and "DB". Which combinations are allowed are defined by the various editing rules. This category of data item contains representations of numeric values, with formatting as explained by the various editing rules.

PICTURE Symbols

Each symbol used in a picture string has a specific function describing the data represented by that picture string. The following section describes each of the picture symbols and their functions.

A - Each "A" in the picture string represents a character position which can contain only a letter of the alphabet, either upper or lower case, or the space character.

B - Each "B" in the picture string represents a character position into which the space character will be inserted.

P - Each "P" in the picture string indicates an assumed decimal scaling position and is used to specify the location of an assumed decimal point when the point is not within the number that appears in the data item. The scaling position character "P" can appear only to the left or right as a continuous string of "P"s within a picture string. Since the character "P" indicates an assumed decimal point (to the left of the "P"s if the the "P"s are leftmost and to the right of the last "P" if the "P"s are rightmost), the assumed decimal point character "V" is redundant. The decimal scaling character "P" and the decimal point insertion character "." cannot both occur within the same picture string.

S - The letter "S" in a picture string is used to indicate the presence, but neither the representation nor, necessarily, the position of an operational sign for the data item. The "S" picture symbol must appear as the leftmost symbol. The actual character position representing the sign is determined by the SIGN clause.

V - The "V" is used in a picture string to indicate the location of the assumed decimal point and may appear only once within a given picture string specification. The "V" does not represent an actual character position, but simply the position of the decimal point within the value. If the "V" appears as the rightmost picture symbol, the "V" is redundant and may be left out.

X - Each "X" in a picture string represents a character position which may contain any printable ASCII character.

Z - Each "Z" in a picture string is used to represent the leftmost leading numeric character positions which will be replaced by space characters when the contents of that character position would be zero.

9 - Each "9" in a picture string represents a character position which contains a numeral.

0 - Each "0" (zero) in a picture string represents a character position into which the numeral zero will be inserted.

/ - Each "/" (stroke) in a picture string represents a character position into which the stroke character will be inserted.

, - Each "," (comma) in a picture string represents a character position into which the comma character "," will be inserted. The symbol "," may not appear as the last element in a picture string.

. - When the character "." (period) appears in a picture string, it is an editing symbol which represents the decimal point for aligning purposes, and in addition, represents a character position into which the character "." will be inserted. The "." picture symbol must not be the last element in a picture string.

+, -, CR, DB - These symbols are used as editing sign control symbols. When used, they represent the character position into which the editing sign control symbol will be placed. The symbols are mutually exclusive in any single picture string.

* - Each "*" (asterisk) in a picture string represents a leading numeric character position into which the character "*" will be placed if that position would contain the digit zero. The use of the "*" picture symbol and the BLANK WHEN ZERO clauses are mutually exclusive.

$ - The currency symbol (whose representation may be altered by the CURRENCY SIGN phrase in the ENVIRONMENT DIVISION) represents a character position into which the currency symbol will be placed. This symbol may represent either a fixed insertion character or leading numeric positions in which case it is a floating insertion character.

Editing Rules

There are two general methods of performing editing in the PICTURE clause, either by insertion or by suppression and replacement. There are four types of insertion editing available:

Simple insertion
Special insertion
Fixed insertion
Floating insertion

There are two types of suppression and replacement editing:

Zero suppression and replacement with spaces
Zero suppression and replacement with asterisks

The type of editing which may be performed depends on the category of the data item. The following table summarizes the possible editing combinations:

Category	Type of Editing
Alphabetic	Simple insertion "B" only
Numeric	None
Alphanumeric	None
Alphanumeric Edited	Simple insertion "0", "B" and "/"
Numeric Edited	All

Floating insertion editing and editing by zero suppression and replacement are mutually exclusive in a PICTURE clause. Only one type of replacement may be used with zero suppression in a PICTURE clause.

Simple Insertion Editing: The "," (comma), "B" (space), "0" (zero) and "/" (stroke) are used as the insertion characters. Each insertion character represents a character position into which the specified character will be inserted.

Special Insertion Editing: The "." (period) is used as the insertion character. In addition to being the insertion character,

it also represents the decimal point for alignment purposes.

Fixed Insertion Editing: The currency symbol and the editing sign control symbols "+", "-", "CR" and "DB" are the insertion characters. Only one currency symbol and only one of the editing sign control symbols may appear in a given picture string. When the "+" or "-" characters appear in a picture string, they must be either the leftmost or rightmost symbol in the string. If the currency symbol appears in the picture string, it must be the leftmost symbol unless preceded by either "+" or "-". If the "CR" or "DB" editing sign control symbol appears in a picture string, it must appear as the rightmost element of the string. Fixed insertion editing results in the insertion character occupying the same character position in the edited item as it occupied in the picture string. Editing sign control symbols produce the following results depending upon the value of the data item:

Editing Symbol in PICTURE string	Data item Positive or Zero	Data item Negative
+	+	-
-	space	-
CR	2 spaces	CR
DB	2 spaces	DB

Floating Insertion Editing: The currency symbol and editing sign control symbols "+" and "-" are the floating insertion characters and as such are mutually exclusive in a given picture string.

Floating insertion is indicated in a picture string by using a string of at least two of the floating insertion characters. This string of floating insertion characters may contain any of the fixed insertion symbols or have fixed insertion characters immediately to the right of this string. These simple insertion characters are considered part of the floating string.

The leftmost character of the floating insertion string represents the leftmost limits of the floating symbol in the data item. The rightmost character of the floating string represents the rightmost limit of the floating symbols in the data item.

The second floating character from the left represents the leftmost limit of the numeric data that can be stored in the data item.

In a given picture string, either all the numeric character positions must be represented by the floating symbol string or any or all of the leading numeric character positions to the left of the decimal point must be represented by the floating symbol string.

If the insertion characters are only to the left of the decimal point in the string, the result is that a single floating insertion character will be placed into the character position immediately preceding either the decimal point or the first non-zero digit in the data, whichever is farther left. The character positions preceding the insertion character are replaced with spaces.

If all numeric character positions in the picture string are represented by the insertion character, the result depends on the value of the data. If the value is zero, the entire data item will contain spaces. If the value is non-zero, the result is the same as if the insertion character appeared only to the left of the decimal point.

To avoid truncation, the minimum size of the picture string for the receiving data item must be the number of characters in the sending data item, plus the number of non-floating insertion characters being edited into the receiving data item, plus one for the floating insertion character.

Zero Suppression Editing: The suppression of leading zeroes in numeric character positions is indicated by the use of the picture symbols "Z" or "*" in a picture string. These symbols are mutually exclusive in a given picture string. If "Z" is used, the replacement character will be the space character and if "*" is used, the replacement character will be the asterisk character "*".

Zero suppression and replacement is indicated in a picture string by using a string of one or more of the allowable symbols to represent leading numeric character positions which are to be replaced when the associated character position in the data contains a zero. Any of the simple insertion characters embedded in the string or to the immediate right of this string are part of the string.

In a given picture string, either all numeric character positions in the string must be represented by suppression characters or any or all of the leading numeric character positions to the left of the decimal point must be represented by the suppression string.

If the suppression symbols appear only to the left of the decimal point, any leading zero in the data which corresponds to a symbol in the string is replaced by the replacement character. Suppression terminates with the first non-zero digit in the data represented by the suppression symbol string, or at the decimal point, whichever is encountered first.

If all numeric character positions in the picture string are represented by suppression symbols and the value of the data is not zero, the result is the same as if the suppression symbols appeared only to the left of the decimal point. If the value is zero and the suppression symbol is "Z", the entire data item will be spaces. If the value is zero and the suppression symbol is "*", the data item will be all "*", except for the actual decimal point.

Usage

The USAGE clause defines the internal representation of the given data item. There are two distinct representations. USAGE DISPLAY indicates that the given data item is represented internally in ASCII character form. For alphabetic or alphanumeric data, this is the only representation available. For numeric data, USAGE DISPLAY indicates that the numeric value will be represented as a string of digit characters, possibly with a sign. For more compact representation of numeric values, USAGE COMPUTATIONAL may be used. Values which are COMPUTATIONAL are stored in packed BCD internally. This makes their storage requirements roughly one half that of the same number with USAGE DISPLAY. The usages COMPUTATIONAL-1 and COMPUTATIONAL-3 are accepted by the compiler for compatibility, but are treated exactly the same as COMPUTATIONAL. The keywords COMP, COMP-1 and COMP-3 are abbreviations for COMPUTATIONAL, COMPUTATIONAL-1 and COMPUTATIONAL-3, respectively. If the USAGE clause is omitted, USAGE DISPLAY is assumed.

Sign

The SIGN clause is used to describe the representation of the sign for a numeric data item. The sign may be specified as appearing at the beginning of the value (LEADING) or at the end of the value (TRAILING). Also, the sign may be specified as occupying a separate character or as being combined with either the first or last data character. If the sign is combined with one of the data characters, the following table indicates the character value present in that data position.

Value	Representation	Value	Representation
0	0	0	0
1	1	-1	a
2	2	-2	b
3	3	-3	c
4	4	-4	d
5	5	-5	e
6	6	-6	f
7	7	-7	g
8	8	-8	h
9	9	-9	i

Thus the value "-125" would be represented as "12e" in combined trailing sign form.

Occurs

The OCCURS clause is used to define repeated data items within a data structure. Up to three levels or dimensions may be defined for a given table. Each of the repeated data items may then be accessed by a subscript expression. If the OCCURS clause appears on a group data item, then every data item within that group must be subscripted in order to be referenced.

An example of a one dimension table definition would be:

```
01 one-level-table.
   02 table-entry     PIC  9  OCCURS 10 TIMES.
```

The above example defines a table, "one-level-table" which contains 10 entries. To reference one of the entries, for example the fifth element, the program would use a subscripted expression such as:

```
MOVE 0 TO table-entry(5).
```

A two dimensional table would be a definition of a set of items which was itself a set of items. An example of this type of table would be a height/weight table with one column for males and a different column for females. A specific weight value would then be determined by the person's sex and by his/her height. A data definition for this table might be:

```
01 height-weight-table.
   02 sex-dimension            OCCURS 2 TIMES.
      03 weight       PIC 999  OCCURS 40 TIMES.
```

This table definition would allow up to 40 different height/weight values to be stored for each sex. A given value would then be accessed by giving first the sex and then the height. Assume that the sex is represented as 1 for male and 2 for female and that the person's height is offset by 5 feet. Thus the entry for a 6 foot, 11 inch man would be found in the entry:

```
MOVE weight(1, 24) TO weight-value.
```

The OCCURS clause may be used to define even more complex structures, those which include three dimensions. Referencing an element in a three dimensional table is similar to that for one and two dimensional tables.

Synchronized

The SYNCHRONIZED clause is accepted by the compiler but is ignored.

Justified

The keyword JUST is an abbreviation for JUSTIFIED. This clause may only be specified for elementary alphabetic or alphanumeric data items. The JUSTIFIED clause alters the way in which data is stored in the data

item. Normally, when data is moved from one location to another, the sending and receiving fields are aligned so that the leftmost character from the sending field is moved to the leftmost character of the receiving field. Specifying the JUSTIFIED clause indicates that the rightmost character of the sending field should align with the rightmost character of the receiving field. If the sending field is shorter than the receiving field, then the receiving field is padded on the left with spaces.

Blank when zero

When present, the BLANK WHEN ZERO clause indicates that the data item being defined, which must be an elementary numeric or numeric edited data item, will have a value equal to all spaces when the data item represents the value zero. Numeric data items will be treated like numeric edited data items if this clause is present.

Value

The VALUE clause is used to assign an initial value to a data item. The specified data item will have the given value when the COBOL program begins execution, but of course may be modified by the program. The VALUE clause may only be specified for elementary, non-edited data items. Also, the VALUE clause may not be used in conjunction with REDEFINES or OCCURS. The VALUE clause may only appear in the LINKAGE or WORKING-STORAGE sections of the program.

The specified literal value must not conflict with the other data description clauses for this data item. For example, a numeric literal cannot be used in the VALUE clause for an alphabetic data item.

Condition Names

In the WORKING-STORAGE SECTION, a data name may be defined as a condition name. A condition name is a shorthand form of a conditional expression whose value can be described in a static form. In this case, the condition name represents the condition where a certain data item contains a specific value or range of values. The condition name may then be used anywhere in the PROCEDURE DIVISION that a conditional expression can be used in which case its use represents the calculation of the implied conditional expression.

The basic format for declaration of a condition name is:

```
88  condition-name;  VALUE   IS   literal   THROUGH   literal  .
                     VALUES  ARE            THRU
```

The level 88 entry must occur immediately following the data item which is to be involved in the conditional expression. If the VALUE IS form is given, use of the condition name is the same as a conditional expression which tests if the data name has exactly the specified value. If the VALUES ARE form is given, then the data name is checked for

values between the first value given and the second value given, inclusively.

For example the declaration:

```
   01 more-data-remains-flag   PIC  x  VALUE 'y'.
      88 no-more-data                 VALUE 'n'.
      88 more-data                    VALUE 'y'.
```

This example declares two different condition names, both relative to the data item "more-data-remains-flag". The condition name "no-more-data" will be true only if "more-data-remains-flag" has exactly the value 'n'. Similarly, the condition name "more-data" will be true only if "more-data-remains-flag" has the value 'y'.

SCREEN SECTION

The SCREEN SECTION describes the format of data to be displayed and accepted from a CRT terminal. The SCREEN SECTION is composed of one or more screen group descriptions. Each screen group description describes a particular set of functions, either input or output, to be executed with respect to the CRT terminal.

The baic formats for a screen group description entry is:

```
   level-number  screen-name

      ;  BLANK  SCREEN

      ;  BLANK  LINE

      ;  BELL

      ;  LINE  NUMBER  IS  PLUS  integer

      ;  COLUMN  NUMBER  IS  PLUS  integer
         COL

                                  ;  FROM   data-name
                                            literal
      ;  PICTURE  IS  picture-string  ;  TO     data-name
         PIC                       ;  USING  data-name

      ;  VALUE  IS  literal

      ;  BLANK  WHEN  ZERO

      ;  JUSTIFIED  RIGHT
         JUST

      ;  SECURE

      ;  AUTO

      ;  REQUIRED

      ;  FULL    .
```

The optional clauses in a screen group description are used to describe the characteristics of that screen group. At least one optional clause must be present in any screen group description. The following section describes each of these entries and their function within a screen description.

Level Number

The level number entries in screen descriptions are used to describe hierarchies within screen descriptions, much the same as for data items. Screen descriptions with level 01 must have a data name specified.

Data name

The data name specified is used to give a name to the screen description. Screen descriptions may represent a single entry, or they may represent a hierarchical description, much as data structures are represented, using the level number to indicate the hierarchy. If the data name is left out, this level of the hierarchy cannot be accessed directly, but must be accessed at a higher level. The data name specified is used by the DISPLAY and ACCEPT verbs to access specific screen descriptions.

Line / Column

These two clauses specify the location on the screen for the associated operations, either output or input. Positions on the screen are numbered left to right, top to bottom, with the upper left corner of the screen being line 1, column 1. When the PLUS keyword is not given, this indicates an absolute value for the cursor position in either the vertical (LINE) or horizontal (COL) direction. If the PLUS keyword is present for either clause, the specified value will be relative to the last cursor position. If the COL clause is not specified and the LINE clause is, this implies that the cursor be placed in column 1 of the given line. If the LINE clause is not specified, the vertical position of the cursor will not change.

Blank Screen / Blank Line

These clauses indicate that either the entire screen or a specified line are to be made blank, i.e. all spaces.

Bell

If this clause is present, the audible bell character on the terminal will sound when this field is displayed.

Auto

This clause may only be specified in conjunction with a TO/USING field. The presence of the AUTO clause indicates that the input operation will automatically be complete when the last data character has been entered into the field without the user entering either a carriage return or the tab character.

Secure

This clause may only be specified in conjunction with a TO/USING field and its presence indicates that any characters entered will be echoed back to the terminal as the asterisk character "*".

Full

This clause may only be present with a TO/USING field and it indicates that the user must completely fill the field. The field termination characters tab and carriage return will only be allowed after the entire field has been entered.

Required

This clause may only be present with a TO/USING field and it indicates that at least one character must be entered in the field.

Value

This clause is used to indicate that a literal string is to be displayed at the given cursor position. The VALUE clause and the PICTURE clause may not both appear in the same screen group description entry.

Picture

The PICTURE clause is used to indicate the format of data to be displayed/accepted on the screen. If used with a FROM field, data will be moved from a data item to the screen using the given picture. If used with a TO field, data will be accepted from the screen using the given picture and then moved to the given data item.

From

This phrase indicates that the specified data item is to be displayed on the screen at the given cursor position. The data is first moved into a temporary space, using the specified picture. This data is then displayed on the screen. Using this mechanism, simple data may be displayed in more a usable format on the screen, e.g. a numeric data item may be specified in a FROM field with a numeric edited picture, thus allowing the data item to be displayed in a more readable form.

To

The TO phrase describes an input field. When a screen description with a TO field is used with the ACCEPT verb, the cursor will be positioned to the beginning of this field on the screen and the user will be allowed to enter data. On completion of the operation, the data entered by the user will be moved into the specified data item. The entry may be completed by either entering a carriage return or the tab character. If the tab character is entered, then any character positions to the right of the cursor position when the tab was entered will not be altered by the operation. If the carriage return is entered, then any character positions to the right of the cursor position will be filled, either with spaces for alphabetic and alphanumeric data items, or with zeroes for numeric data items. The data item specified in a TO phrase

must not be numeric edited or alphanumeric edited, nor can they be USAGE COMPUTATIONAL or USAGE INDEX.

Using

The USING phrase is equivalent to specifying both a TO and a FROM phrase for the same data item.

Blank When Zero / Justified Right

These clauses have the same meaning as for any other data item and may only appear in conjunction with a FROM/USING clause. If the BLANK WHEN ZERO clause is present, it indicates that if the data item to be displayed has the value zero, the value displayed will be all spaces. If the JUSTIFIED RIGHT clause is present, it indicates that the data will be right justified when it is moved to the screen, using the picture as specified.

REPORT SECTION

The REPORT SECTION of a COBOL program describes one or more reports to be produced by the program. The reports are described in terms of the physical page structure they will be printed in, controls which are used to produce summaries and report group descriptions. These report group descriptions cover the basic report functions such as report heading, page heading, control heading and detail lines. Each of these report group descriptions will be described in detail below.

Each report in the REPORT SECTION consists of an RD paragraph which describes the basic characteristics of the report, followed by one or more report group descriptions which describe the details of the report.

The basic format of an RD paragraph is:

```
   RD  report-name

      ;  CONTROL   IS    data-name  ,  data-name
         CONTROLS  ARE   FINAL  ,  data-name

      ;  PAGE  LIMIT   IS   integer   LINE
               LIMITS  ARE            LINES

           ,  HEADING  integer

           ,  FIRST  DETAIL  integer

           ,  LAST  DETAIL   integer

           ,  FOOTING   integer  .

      Report  Group  Description  Entry  ...
```

The following section describes the optional phrases in the report description (RD) paragraph.

Report Name

The report name must be the same as a report named in an FD paragraph in the FILE SECTION. Every report named in an FD paragraph must have a separate report description.

Control

The CONTROL clause establishes the levels of the control hierarchy for the report.

The data names and the keyword FINAL specify the levels of the control hierarchy. FINAL, if specified, is the highest control. The first data name given is the next highest control, etc, with the last data name given being the lowest level control. The data name entries must have been defined elsewhere in the DATA DIVISION, in any section other than the SCREEN or REPORT sections. Each data name specified must reference a different data item.

The execution of the first GENERATE statement for a given report causes the Report Writer Control Sytem (RWCS) to save the values of the control data items associated with that report. On subsequent executions of a GENERATE statement for that report, control data items are tested by the RWCS for a change of value. Any change of value causes a control break to occur.

A control break for the control FINAL only occurs with the initialization and termination of the report.

Page Limits

The PAGE LIMIT clause defines the length of a page and the vertical subdivisions within which report groups are presented.

The HEADING, FIRST DETAIL, LAST DETAIL and FOOTING phrases must be written in the order given and any values given must be in the relationship:

1 <= HEADING <= FIRST DETAIL <= LAST DETAIL <= FOOTING <= PAGE LIMIT

Certain report groups may appear only within certain regions of the page. These restrictions are given in the following table:

Report Groups That May Be Presented In The Region	First Line Number Of The Region	Last Line Number Of The Region
REPORT HEADING with NEXT GROUP NEXT PAGE	HEADING	PAGE LIMIT
REPORT FOOTING with LINE <HEADING> NEXT PAGE		
REPORT HEADING without NEXT GROUP NEXT PAGE	HEADING	FIRST DETAIL-1
PAGE HEADING		
CONTROL HEADING		
DETAIL	FIRST DETAIL	LAST DETAIL
CONTROL FOOTING	FIRST DETAIL	FOOTING
PAGE FOOTING		
REPORT FOOTING without LINE <HEADING> NEXT PAGE	FOOTING + 1	PAGE LIMIT

The report group description entry specifies the characteristics of a report group and of the individual items within a report group. The basic formats for a report group description entry are:

Format 1:

```
   01   data-name

      ;  LINE  NUMBER  IS  integer  ON  NEXT  PAGE
                           PLUS  integer

      ;  NEXT  GROUP  IS  integer
                          PLUS  integer
                          NEXT  PAGE

                    REPORT HEADING
                    RH
                    PAGE HEADING
                    PH
                    CONTROL  HEADING   data-name
                    CH                 FINAL
      ;  TYPE  IS   DETAIL
                    DE
                    CONTROL FOOTING    data-name
                    CF                 FINAL
                    PAGE FOOTING
                    PF
                    REPORT FOOTING
                    RF

      ;  USAGE  IS  DISPLAY  .
```

Format 2:

```
   level-number  data-name

      ;  LINE  NUMBER  IS  integer  ON  NEXT  PAGE
                           PLUS  integer

      ;  USAGE  IS  DISPLAY   .
```

Format 3:

```
   level-number  data-name

      ;  BLANK  WHEN  ZERO

      ;  GROUP  INDICATE

      ;  JUSTIFIED  RIGHT
         JUST

      ;  LINE  NUMBER  IS  integer  ON  NEXT  PAGE
                           PLUS  integer

      ;  COLUMN  NUMBER  IS  integer

      ;  PICTURE  IS  picture-string
         PIC

      ;  SOURCE  IS  data-name

      ;  VALUE  IS  literal

      ;  SUM  data-name  ,  data-name   ...
           UPON  data-name  ,  data-name   ...    ...
           RESET  ON  data-name
                      FINAL

      ;  USAGE  IS  DISPLAY   .
```

Level Number

In format 1, the level number value must be 01. For formats 2 and 3, the level number may be any value between 02 and 49. The level number is used to represent a hierarchy within the various report group description entries.

Data Name

The data name field is used to name a report group. This name is used in the GENERATE and USE statements to reference specific report groups. If this is a CONTROL FOOTING report group with SUM clauses present, the data name is used to name the associated sum counters. Only level 01 report groups with TYPE DETAIL need have a name, all others may be named only if they are referenced explicitly in the program.

Line Number

The LINE NUMBER clause specifies the vertical position for a report group. A LINE NUMBER clause is required to establish each print line of a report group.

The value specified may be an absolute integer value, a value relative to the last line printed, or an absolute line number on the following page.

Next Group

The NEXT GROUP clause specifies the vertical positioning of a report group following the presentation of the last line of the present report group.

The occurrence of a NEXT GROUP clause controls the vertical positioning of the next report group presented. This clause has no effect on the presentation of the current report group. The value specified may be either an absolute line number, a relative line number or the first relevant line on the following page.

Type

The TYPE clause specifies the particular type of report group that is described by a report group entry and indicates the time at which the report group is to be processed by the Report Writer Control System (RWCS).

DETAIL report groups, abbreviated DE, are processed by the RWCS as a direct result of a GENERATE statement. All other report group types are processed automatically by the RWCS.

The REPORT HEADING report group, abbreviated RH, is processed only once as the first report group presented for a given report. The REPORT HEADING group is processed during the execution of the first GENERATE statement for a report.

The PAGE HEADING report group, abbreviated PH, is processed by the RWCS as the first report group on every page.

The CONTROL HEADING report group, abbreviated CH, is processed by the RWCS at the beginning of a control group for a designated control data item, or in the case of FINAL, is processed during the execution of the first GENERATE statement for the report. During the execution of any GENERATE statement for which a control break is detected, any CONTROL HEADING report groups associated with the highest level of control break through the lowest level of control are processed.

The CONTROL FOOTING report group, abbreviated CF, specifies a report group that is processed at the end of a control group for a designated control name. The CONTROL FOOTING report group for the control FINAL is only processed during the execution of the TERMINATE statement for the report. During the execution of a GENERATE statement for which a control break is detected, any CONTROL FOOTING report groups associated with the highest level of control break through the lowest level of control are presented, from lowest level of control through the highest level of the control break.

The PAGE FOOTING report group, abbreviated PF, specifies a report group that is processed by the RWCS as the last report group on a page.

The REPORT FOOTING report group, abbreviated RF, specifies the last report group in a given report. The REPORT FOOTING report group is processed during the TERMINATE statement for the given report.

Group Indicate

The GROUP INDICATE clause specifies that the associated printable item is printed only on the first occurrence of its report group after a control break or page advance. The GROUP INDICATE clause may only be specified within a TYPE DETAIL report group.

The presence of the GROUP INDICATE clause causes the SOURCE or VALUE clause to be ignored, and the appropriate number of spaces provided, except:

- On the first presentation of the DETAIL report group in the report, or
- On the first presentation of the DETAIL report group after a page advance, or
- On the first presentation of the DETAIL report group after a control break.

Column Number

The COLUMN NUMBER clause specifies the absolute column number that a data item is to be printed at. The COLUMN NUMBER clause must be specified for every item to be printed on a line in a report.

Picture

The PICTURE clause specifies the picture to be used to print the data item. When presented, the value of a SUM, SOURCE or VALUE clause is moved to an intermediate area within the RWCS using the specified picture. This [possibly edited] value is then printed on the specified line at the given column position.

Source

The SOURCE clause specifies a data item to be printed in a report group. When processing this report group entry, the RWCS will move the specified source data item to an intermediate area using the associated picture string (see PICTURE, above). This formatted data is then printed at the specified location on the line.

Value

The VALUE clause specifies a literal string to be printed. The VALUE clause is treated identically to the SOURCE clause except that the value is a literal string.

Sum

The SUM clause establishes a sum counter and specifies the data items to be summed. The SUM clause may only appear in a CONTROL FOOTING report group. The SUM data names must reference numeric data items, either elementary data items in the DATA DIVISION, or other sum counters defined in previous report groups.

The data names appearing in the UPON and RESET phrases must be the name of report groups defined within this report. If the keyword FINAL appears in the RESET phrase, it must also appear in the CONTROLS ARE phrase in the RD paragraph for this report.

The SUM clause establishes a sum counter which is a numeric data item with an operational sign. A runtime, the RWCS adds directly into the sum counter then values of each of the data items specified.

The size of the sum counter is the number of receiving character positions in the associated picture string.

When the CONTROL FOOTING report group is presented, the sum counter is printed as if it contained a SOURCE clause with the sum counter specified as the source data item.

Addition of the data names into sum counters is performed by the RWCS during the execution of GENERATE and TERMINATE statements. There are three categories of sum counter incrementing called subtotalling, crossfooting, and rolling forward. Subtotalling is accomplished during the execution of GENERATE statements only.

The UPON phrase provides the capability to accomplish selective subtotalling for the DETAIL report groups named in the phrase.

The RWCS updates sum counters at a time that depends on the characteristics of the addend.

Crossfooting: When the addend is a sum counter defined in the same CONTROL FOOTING report group, then the accumulation of that addend into the sum counter is known as crossfooting.

Crossfooting occurs when a control break takes place and at the time the CONTROL FOOTING report group is processed.

Crossfooting is performed according to the sequence in which sum counters are defined within the CONTROL FOOTING report group. That is, all crossfooting into the first sum counter defined in the report group is performed first, followed by the crossfooting into the second sum counter in the report group, etc.

Rolling Forward: When the addend is a sum counter defined in a lower level CONTROL FOOTING report group, the accumulation of that addend into the sum counter is known as rolling forward. A sum

counter in a lower level CONTROL FOOTING report group is rolled forward when a control break occurs and at the time the lower level CONTROL FOOTING report group is processed.

Subtotalling: When the addend is not a sum counter, the accumulation process is known as subtotalling. If the SUM clause contains the UPON phrase, the addends are subtotalled when a GENERATE statement for the specified report group is executed. If the SUM clause does not contain the UPON phrase, the addends which are not sum counters are subtotalled when a GENERATE statement for the report group which contains the SUM clause is executed.

The RESET phrase must reference a data item named in the CONTROLS ARE clause of the RD paragraph for the report. If the RESET phrase is not present, the RWCS will set a sum counter to zero at the time that the CONTROL FOOTING report group which contains the sum counter is processed, after the report group has been totally processed. If the RESET phrase is present, the sum counter is set to zero at the time the RWCS is processing the specified level of the control hierarchy.

VIII. THE PROCEDURE DIVISION

The PROCEDURE DIVISION of a COBOL program contains the statements which describe the functions and operations to be performed by the program. This includes statements to perform input/output, numeric calculations, etc. The PROCEDURE DIVISION is constructed of named paragraphs, each paragraph being composed of a sequence of statements. Paragraphs may also be grouped into named sections.

Format 1:

```
PROCEDURE DIVISION USING data-name , data-name ... .

DECLARATIVES.

   section-name SECTION. Use Statement
   paragraph-name. sentence ... ... ...

END DECLARATIVES.

section-name SECTION.
paragraph-name. sentence ... ... ...
```

Format 2:

```
PROCEDURE DIVISION USING data-name , data-name ... .

paragraph-name. sentence ... ... ...
```

ACCEPT

The ACCEPT verb is used to obtain input from the terminal on which the COBOL program is running. The input format may be described using the SCREEN SECTION or input may be free format.

Format:

```
ACCEPT screen-name ; ON ESCAPE imperative statement
```

Format 2:

```
ACCEPT data-name

    , LINE      identifier
                literal

    , POSITION  identifier
                literal

    , SIZE      identifier
                literal

    , PROMPT

    , ECHO

    , CONVERT

    , SECURE
```

Format 3:

```
                          DAY
ACCEPT data-name  FROM    DATE
                          TIME
```

Examples:

```
ACCEPT menu-0; ON ESCAPE GO TO next-program-function.
ACCEPT cobol-date FROM DATE.
```

There are two unique functions performed by the ACCEPT statement. These functions are represented by the three different formats.

The first format allows an operator to enter data on a terminal device for use by a COBOL program. In most cases, this terminal device is a CRT terminal, and for this purpose, the ACCEPT verb has built-in features which make use of the facilities of CRT display terminals. If the data-name given in the ACCEPT statement is the name of a screen description, then the system will position the cursor on the terminal to the first data field in the screen description which contains either a TO or a USING field. The operator may then enter data for this field, or leave the data unaltered by hitting the TAB key. The cursor will then be positioned to the next input field. The cursor may be repositioned to a previous field in the screen by entering the "back-tab" character. Since not all terminals have a back-tab character defined, COBOL uses the Line Feed character for this purpose. The cursor may be moved backwards as many times as desired, or until the cursor reaches the first field in the screen. This process continues until data has been entered for the last input field in the screen description or the operator enters the control-C key. If the operator enters a control-C, all screen input functions will be aborted and the ON ESCAPE sentence executed. If no ON ESCAPE sentence is present, the program will terminate. See chapter 7 for a complete discussion of screen formatted data using the SCREEN SECTION.

The second format is very similar to the first format in that the user is allowed to enter data from the terminal. In this format, however, each input field is described as it is being input using the optional fields for the statement. The LINE clause specifies the absolute line number on the screen which the cursor is to be positioned before accepting data. The POSITION clause specifies the cursor's horizontal position. The SIZE field specifies how wide the field to be accepted is. If this clause is not present, then sufficient characters to fill the data item will be accepted. The PROMPT clause, if present, instructs the program to display a prompt for the field when it is being accepted. This is done by writing the appropriate number of underscore ('_') characters on the screen at the position where data is to be entered. If the PROMPT clause is not present, the data on the screen is not altered until characters are entered. If the SECURE clause is present, the asterisk character '*' will be displayed on the screen for all characters entered into the field. The ECHO and CONVERT clauses are currently accepted by the compiler but have no runtime function.

The third format of the ACCEPT statement is used to obtain the current day, date or time in a format usable by the COBOL program. These values are considered to be numeric source fields and are moved to the given data-item using the standard move mechanism. The format of the DAY field is: YYDDD, where YY represents the current year (modulo 1900) and DDD represents the Julian day for the current day. The format of the DATE field is: YYMMDD, where YY is the current year, MM is the current month (January being 01 and December being 12) and DD is the day of the month. The format of the TIME field is: HHMMSS0000, where HH is the current hour (24 hour clock), MM is the current minute and SS is the number of seconds in the current minute. The last four (4) digits of this field are always zero.

ADD / ADD CORRESPONDING

The ADD verb is used to add two or more numeric items to obtain their sum.

Format:

```
ADD  data-name   ,  data-name   ...  TO  data-name  ROUNDED
     literal        literal

   ;  ON  SIZE  ERROR  imperative statement


ADD  data-name  ,  data-name   ...  GIVING  data-name  ROUNDED
     literal       literal

   ;  ON  SIZE  ERROR  imperative statement


ADD  CORRESPONDING  data-name  TO  data-name  ROUNDED
     CORR

   ;  ON  SIZE  ERROR  imperative statement
```

Examples:

```
ADD amount TO balance; ON SIZE ERROR GO TO balance-overflow.
ADD val-1, val-2 GIVING val-3 ROUNDED.
```

The ADD verb computes the sum of two or more operands, either GIVING a result or adding TO an existing value. All source operands must be numeric data items or numeric literals. If the GIVING form of the verb is used, the receiving identifier may be either numeric or numeric-edited. In all other forms, the receiving identifier must be numeric.

If the result exceeds 18 total digits or the result cannot be moved successfully into the receiving data field, then a SIZE ERROR will occur and the result stored is undefined.

In the ADD CORRESPONDING form, all corresponding numeric fields in the data items are added, with the resultant sum replacing the value in the second data item. The SIZE ERROR will occur if any of the additions exceed 18 total digits. In this case, the values of all the corresponding fields will be undefined. See MOVE CORRESPONDING for a complete description of how corresponding operations are defined.

CALL

The CALL PROGRAM statement allows a COBOL program to invoke the execution of another program.

Format:

```
CALL  PROGRAM  data-name  USING  data-name  ,  data-name   ...
               literal

    ;  ON  EXCEPTION  imperative statement
           OVERFLOW
```

Examples:

```
CALL PROGRAM "prog-1" USING title, page-no.
CALL PROGRAM "+copy file-1 file-2".
```

The CALL PROGRAM statement allows a COBOL program to invoke the execution of another program. There are two ways that a program may be called. First, a COBOL program may call another COBOL program, possibly passing some information to that program via the USING clause. In this case, the called program replaces the calling program, using a chain-like mechanism. The identifier or literal in the statement is the name of the program to be invoked and should be a well-formed UniFLEX file pathname. The called program must be a COBOL program and if the called program contains a USING clause in its PROCEDURE DIVISION header, then the calling program should contain a similar USING clause in the CALL PROGRAM statement. In this form of the CALL PROGRAM statement, the called program never returns to the calling program. If the program to be executed cannot be located or executed, the COBOL program will continue execution with the statement following the CALL PROGRAM. For this form of the CALL PROGRAM statement, control returns to the calling program only in the case of an error condition. The calling COBOL program should be written to take care of this eventuality.

There is a second form of the CALL PROGRAM which does allow the called program to return control to the calling program. Using this form, any variety of program may be called, not just COBOL programs. This form of the CALL PROGRAM statement is distinguished from the first by the presence of the character "+" as the first character of the name of the called program. In this case, the identifier or literal in the CALL PROGRAM statement is actually a command string passed to a shell. When the shell program terminates, the COBOL program continues with the statement following the CALL PROGRAM statement. There is no provision for passing information to the called program when using this form of the CALL PROGRAM statement (except the standard UniFLEX parameter mechanism) as the USING phrase is not allowed.

CLOSE

The CLOSE verb is used to terminate all input/output operations on a file.

Format:

```
CLOSE file-name , file-name ...
```

Examples:

```
CLOSE serial-file.
CLOSE data-base, report-file.
```

The purpose of this statement is to terminate all activity on a file. This function is known as "closing" the file. After the file is closed, no other operations except an OPEN statement or DELETE FILE statement may be executed on the file.

The state of the file after the CLOSE file is reported in the file status bytes associated with the file.

It is good programming practice to close all open files before the program terminates. However, the runtime system will close any open files when the program terminates, either normally or abnormally.

COMPUTE

The COMPUTE verb is used to assign the result of a numeric computation to a variable. This statement is a shorthand form of the arithmetic statements ADD, SUBTRACT, MULTIPLY and DIVIDE.

Format:

```
COMPUTE  data-name  ROUNDED  =  arithmetic expression

   ;  ON  SIZE  ERROR  imperative statement
```

Examples:

```
COMPUTE circumference = diameter * 3.14159.
COMPUTE net-pay ROUNDED = (hours * pay-rate) - deductions.
```

This statement assigns the value of an arithmetic expression to a data-item. The receiving data item may be either numeric or numeric edited data item. The expression is made up of terms using either numeric data items or numeric literals and the operators +, -, *, and /.

If the keyword ROUNDED is specified, then the result will be rounded before the value is assigned.

If the result value cannot be successfully moved into the receiving data item, or if any intermediate result exceeds 18 significant digits, then the SIZE ERROR condition occurs and the value of the receiving field is undefined.

COPY

The COPY statement causes lines from a file to be inserted into the text of a COBOL program.

Format:

```
COPY literal-file-name
```

Examples:

```
COPY "seq-file-fd".
COPY "/usr/include/std-defs".
```

The COPY statement allows a COBOL program to include the text of a file into the source of the COBOL program. The COPY statement is replaced by the contents of the specified file. The literal must be a well-formed UniFLEX file pathname. The copied file must not contain any COPY statements, i.e. COPY statements are not allowed to be nested. The COPY statement must be the only statement on the source line. Anything which appears to the right of the period on the same line as the COPY statement will be ignored.

DELETE

The DELETE verb is used to delete specific records from a file or to delete an entire file from the file system.

Format:

```
DELETE file-name RECORD ; INVALID KEY imperative statement
```

Format 2:

```
DELETE FILE file-name
             literal
```

Examples:

```
DELETE random-file RECORD;  INVALID KEY go to delete-error.
DELETE FILE random-file.
DELETE FILE "/library/books-file".
```

There are two forms of this verb. The first form allows the COBOL program to delete a specific record from a RELATIVE or INDEXED file. This verb works differently depending on the ACCESS MODE of the file.

If the file is open in SEQUENTIAL access mode, then the COBOL program must have successfully executed a READ operation on the file immediately prior to the DELETE verb. In this case, the record just read will be the record deleted.

If the file is open in RANDOM or DYNAMIC access mode, then the record to be deleted is given by the record access key, either a relative key (record number) for RELATIVE files, or the primary and alternate record keys for INDEXED files.

After the execution of the DELETE verb, the file status item associated with the file will contain information about the operation. Refer to chapter 3 for complete details on possible file status codes. If the operation fails for any reason, then the INVALID KEY condition exists and the statement in the INVALID KEY clause will be executed. A USE procedure must be defined for the file if the INVALID KEY clause is not present. If both the INVALID KEY clause and a USE procedure are present, then only the INVALID KEY clause will be executed.

The second form of the DELETE verb allows the COBOL program to delete an entire file. This is not a standard ANSI COBOL feature, but has been added to make COBOL more useful. The file to be deleted may be specified either by specifying a COBOL file name in which case the file associated with that COBOL file will be deleted, or by specifying the actual name of the file to be deleted. The user should be cautious when using this verb because if the file to be deleted cannot actually be deleted, the program will abort in the attempt. The most common use for

this mechanism is to clean up scratch files used by a COBOL program when the program has no more need for them.

DISPLAY

The DISPLAY verb is used to display data values on the user's terminal.

Format:

```
DISPLAY  screen-name  ,  screen-name  ...
```

Format 2:

```
DISPLAY  data-name  ,  LINE  data-name  ,  POSITION  data-name
         literal             literal                 literal

   ,  SIZE  data-name
            literal
```

Examples:

```
DISPLAY menu-0.
DISPLAY "*** I/O Error ***", LINE 10, POSITION 1.
```

This verb displays data on the user's terminal. For the most part, it is assumed that this terminal is a CRT device which may be used to display formatted data. To this end, there are two forms of the verb, both designed to give the COBOL programmer wide control over what is displayed on the terminal, where it is displayed and the format it is displayed in.

The first format is used in conjunction with the SCREEN SECTION of the program. Data to be displayed are described in this section using screen record descriptions. See chapter 7 for more detail on the SCREEN SECTION.

When using the first format of the DISPLAY verb, all fields in the given screen descriptions which contain VALUE, FROM or USING clauses will cause data to be displayed on the terminal screen. The terminal is assumed to have two modes of display output. Foreground mode is brighter and represents data which may possibly be changed by the terminal operator. Background mode is lighter and represents data which cannot be modified. (If the terminal being used does not support these different modes, then all data will be displayed in the terminal's normal mode). Values which are specified in a USING clause will be displayed in foreground mode. All other values will be displayed in background mode.

The second format of the DISPLAY verb allows the COBOL program to display data values, one data value at a time, specifying information about where and how to display the value directly in the DISPLAY statement. The vertical position where the data is to be displayed is given by the LINE clause. This clause specifies the absolute vertical position for the data either in the form of a numeric literal value, or given by the value of a numeric data item. If the LINE clause is not specified, then the data is displayed at the current vertical position of the cursor. The horizontal position of the data is given by the POSITION clause. If the POSITION clause is not present and the LINE clause is, then data will be displayed starting in column 1 of the specified line. The SIZE clause allows the COBOL program to specify how many characters to display for a particular data item.

If no LINE or POSITION clause is present, the data will be displayed at the current cursor position and the cursor will be placed at the beginning of the next line following the completion of the DISPLAY statement.

A given COBOL program may use both forms of the DISPLAY verb, but the two formats must be specified separately. That is, a single DISPLAY statement may not contain both screen descriptions and non-screen data items to be displayed.

DIVIDE

The DIVIDE statement divides one numeric data item into another and sets the value of data items to the quotient and remainder.

Format:

```
DIVIDE  data-name  INTO  data-name  ROUNDED
        literal

   ;  ON  SIZE  ERROR  imperative statement


DIVIDE  data-name  INTO  data-name  GIVING  data-name  ROUNDED
        literal          literal

   REMAINDER  data-name  ;  ON  SIZE  ERROR  imperative statement


DIVIDE  data-name  BY  data-name  GIVING  data-name  ROUNDED
        literal        literal

   REMAINDER  data-name  ;  ON  SIZE  ERROR  imperative statement
```

Examples:

```
DIVIDE 3.14159 INTO circumference.
DIVIDE number-of-items INTO total-cost GIVING cost-per-item;
  ON SIZE ERROR go to bad-divide-handler.
DIVIDE circumference BY 3.14159 GIVING diameter.
```

In the INTO format, the DIVIDE statement divides the first data item into the second data item, producing a quotient and remainder. If the GIVING phrase is present, then the quotient is moved into the data item specified by the GIVING phrase, otherwise the quotient is moved into the data item specified by the INTO phrase. In the BY format, the first data item is divided by the second data item and the resulting quotient is moved into the data item specified by the GIVING phrase.

If the REMAINDER phrase is present, then the integer remainder of the division is moved into the data item specified. If this phrase is not specified, then the remainder of the division is discarded.

The resulting value, as well as any intermediate values, must contain no more than 18 significant digits, otherwise the SIZE ERROR condition will occur. Also, if either the quotient or remainder value cannot be moved successfully into the appropriate result field or if the divisor is zero, then the SIZE ERROR condition will occur. If the SIZE ERROR condition occurs, then the result of the operation is undefined and the ON SIZE ERROR statement will be executed.

All source values for this operation must be either numeric literals or numeric data items. The receiving data fields may be numeric or numeric edited data items.

EXIT

The EXIT statement provides a common end point for a series of procedures.

Format:

```
EXIT.
```

Examples:

```
EXIT.
```

The EXIT statement is used to exit a paragraph or section which has been "PERFORMED". The EXIT statement must be the only statement in a paragraph and is primarily used to provide an exit point which can be branched to using the GO TO statement. The EXIT statement has no effect if the paragraph in which it occurs is not the termination of a current PERFORM statement. That is, the statement must occur either as the last statement in a performed section or as the only statement in a paragraph which is mentioned in a THRU clause in a PERFORM statement.

GENERATE

The GENERATE statement directs the Report Writer Control System (RWCS) to produce a report in accordance with a report description given in the REPORT SECTION.

Format:

```
GENERATE  report-detail-group-name
```

Examples:

```
GENERATE summary-detail-line.
```

The GENERATE statement is the only statement used in conjunction with the Report Writer feature which actually generates report data.

The report detail group name specified in the GENERATE statement must refer to a TYPE DETAIL report group and may be qualified by the report name for that group. The report must have been previously initiated, using the INITIATE statement.

The first GENERATE statement executed after the report is initiated will cause any REPORT HEADING report group to be executed. This is followed by the presentation of all CONTROL HEADING report groups, beginning with the lowest control up through and including the highest control variable. Control variables are ordered from lowest to highest based on the order they appear in the CONTROLS ARE clause in the RD statement. The first control variable is known as the lowest control, up to the last control variable which is referred to as the highest.

Before the DETAIL report group is presented, the current value of all control variables will be checked and if any of these values has changed since the last GENERATE statement for this report was executed, a control break will occur. When a control break occurs, the CONTROL FOOTING report groups for each control higher than or equal to the breaking control will be presented, in reverse order beginning with the highest control variable. Then all CONTROL HEADING report groups for the control variables higher than or equal to the breaking control will be presented, in order beginning with the breaking control and ending with the highest control.

After presentation of the DETAIL report group, all sum counters will be updated.

If any report group which is presented causes the logical page to become full, then the PAGE FOOTING report group, if any, will be presented. This is followed by presentation of the PAGE HEADING report group. This comment applies to all report group presentations: those caused either directly or indirectly by a GENERATE statement.

GO TO

The GO TO statement causes control to be transferred from one part of the PROCEDURE DIVISION to another.

Format:

```
GO TO  procedure-name

GO TO  procedure-name , procedure-name ... DEPENDING ON data-name
```

Examples:

```
GO TO rewrite-error.
GO TO function-1, function-2, function-3 DEPENDING ON function-choice.
```

The first form of the GO TO statement is an unconditional transfer of control. The second form allows a choice of destinations, based on the value of the "DEPENDING ON" identifier. This identifier must describe a numeric data item with an integer value. If the data item has the value 1, then control is transferred to the first label in the list. If it has the value 2, control passes to the second label, etc. If the data item has a value less than 1 or greater than the number of labels in the list, execution continues with the statement following the GO TO statement.

IF

The IF statement evaluates a conditional expression and based on the truth value of that expression, executes one of two possible statements.

Format:

```
IF  condition  ;  statement        ;  ELSE  statement
                  NEXT  SENTENCE      ELSE  NEXT  SENTENCE
```

Examples:

```
IF program-choice IS NUMERIC; PERFORM proc-1; ELSE PERFORM proc-2.
IF serial-status NOT = "00" GO TO open-error.
```

Statement-1 and statement-2 may be any statement, including IF statements. The ELSE NEXT SENTENCE phrase may be omitted.

If the conditional expression is true, the statement sequence following the conditional expression, up to the keyword ELSE, is executed and the statement sequence which follows the ELSE is not executed. If the conditional expression is false, the statement sequence following the keyword ELSE is executed.

INITIATE

The INITIATE statement causes the Report Writer Control System (RWCS) to begin the processing of a report.

Format:

```
INITIATE  report-name
```

Examples:

```
INITIATE audit-report.
```

The report name must have been defined in an RD statement in the REPORT SECTION and must not have had an INITIATE statement executed on it without a TERMINATE statement intervening. The file associated with the report must be open in either OUTPUT or EXTEND mode.

Upon execution of the INITIATE statement, the following functions are performed:

All sum counters are set to zero.

The LINE-COUNTER value for this report is set to zero.

The PAGE-COUNTER value for this report is set to one.

INSPECT

The INSPECT statement provides the ability to tally (count), replace, or tally and replace occurrences of single characters within a data item.

Format:

```
INSPECT  data-name

                                  ALL         data-name
   TALLYING  ,  data-name  FOR  ,  LEADING     literal
                                  CHARACTERS

      BEFORE            data-name
      AFTER    INITIAL  literal    ...  ...

   REPLACING

                    data-name   BEFORE             data-name
      CHARACTERS  BY  literal     AFTER    INITIAL  literal

      ALL
      LEADING  ,  data-name  BY  data-name
      FIRST       literal        literal

         BEFORE            data-name
         AFTER    INITIAL  literal    ...  ...
```

Examples:

```
INSPECT middle-name TALLYING space-count ALL SPACES AFTER INITIAL ".".
INSPECT num-item REPLACING LEADING SPACES BY ZEROES.
INSPECT num-item TALLYING space-count FOR LEADING SPACES
                 REPLACING LEADING SPACES BY ZEROES.
```

The INSPECT data item must reference a data item described as USAGE DISPLAY. The TALLYING data item must be a numeric data item. All other data items must be USAGE DISPLAY.

The INSPECT statement searches a given data item for specific characters and either counts (tallies) occurrences or replaces occurences of characters with other characters. The search may be qualified by using the BEFORE/AFTER clause. If the BEFORE clause is specified, then the search is restricted to all characters in the data item which occur before (and not including) the character given by the BEFORE clause. If the character in the BEFORE clause does not occur in the INSPECT data item, then the all characters in that data item will be searched. Similarly, the AFTER clause restricts the search to all characters following a given character. If the character in the AFTER clause is not found, then no characters in the INSPECT data item will be searched.

The characters to be searched for are specified by the ALL/LEADING/CHARACTERS clause or the ALL/LEADING/FIRST clause. The ALL clause specifies that all occurrences of the given character are to be used in the search. The LEADING clause specifies that only those characters which occur consecutively from the starting position are considered. The CHARACTERS selection clause signifies any possible character. The FIRST selection clause signifies only the first occurrence of the given character and not any subsequent occurrences.

The TALLYING option of the INSPECT verb searches the given data item and counts occurrences of a given character. For every occurrence of the search character (given by the ALL/LEADING/CHARACTERS) clause, the tally data item will be incremented.

The REPLACING option of the INSPECT statement allows characters to be replaced by other characters. The characters which are to be replaced are specified by the ALL/LEADING/FIRST clause. For every character which satisfies the selection, that character is replaced by the character given by the BY clause.

More than one TALLYING or REPLACING option may be present in a given INSPECT statement. When more than one of these optional phrases is present, they will be executed from "top" to "bottom", starting with the phrase closest to the INSPECT verb itself and continuing to the end of the statement. The phrases are not executed in parallel, so some care must be taken to avoid side-effects produced by the REPLACING clause.

MERGE

The MERGE statement combines two or more identically sequenced files on a set of specified keys, and during the process makes records available, in merge order, to an output procedure or an output file.

Format:

```
                        ASCENDING
MERGE  file-name  ON    DESCENDING  KEY  data-name  ,  data-name  ...

                        ASCENDING
                  ON    DESCENDING  KEY  data-name  ,  data-name  ...

    USING  file-name  ,  file-name  ,  file-name  ...

    OUTPUT  PROCEDURE  IS  section-name  THROUGH  section-name
                                         THRU

    GIVING  file-name
```

Examples:

```
MERGE merge-file ON merge-key of merge-record
  USING input-file-1, input-file-2
  GIVING output-file.
```

The MERGE statement will merge all records in the files given in the USING phrase, as specified by the key ordering given by the ON ASCENDING/DESCENDING KEY phrases. This merged file may then be assigned directly to a separate file by using the GIVING phrase, or may be processed one at a time, in merge order, by specifying the OUTPUT PROCEDURE phrase.

Before execution of the MERGE statement, all files named in the statement must be closed. The file named in the GIVING clause will be implicitly opened for OUTPUT, written, and then closed.

The sorting sequence keys must reference alphanumeric data items which are defined as part of the record definition associated with the merge file. There may be no more than 20 key specifications given, but at least one must be given. When the ASCENDING phrase is specified, the merge sequence will be from the lowest value present for that given key to the highest value for the key. When the DESCENDING key phrase is specified, the merge sequence is from the highest key value to the lowest.

If more than one key specification is given, the first key specified is the most significant, the second key is the next most significant, etc.

MOVE / MOVE CORRESPONDING

The MOVE statement transfers data from one data area to another in accordance to the rules of editing.

Format:

```
MOVE  data-name  TO  data-name
      literal
```

Format 2:

```
MOVE CORRESPONDING  data-name  TO  data-name
     CORR
```

Examples:

```
MOVE ALL SPACES TO name.
MOVE serial-no of serial-record TO input-serial-no.
MOVE CORR current-date TO work-date.
```

Any MOVE in which the sending and receiving items are both elementary items is an elementary move. Every elementary item belongs to one of the following categories: numeric, alphabetic, alphanumeric, numeric edited, alphanumeric edited. These categories are described by the PICTURE clause when a data item is defined. The figurative constant SPACE belongs to the category alphabetic. Numeric literals and the figurative constant ZERO belong to the category numeric. Nonnumeric literals and all other figurative constants belong to the category alphanumeric.

The following rules apply to an elementary move between these categories:

The figurative constant SPACE, a numeric edited, alphanumeric edited, or alphabetic data item must not be moved to a numeric or numeric edited data item.

A numeric literal, the figuative constant ZERO, a numeric data item or a numeric edited data item must not be moved to an alphabetic data item.

A non-integer numeric literal or non-integer numeric data item must not be moved to an alphanumeric or alphanumeric edited data item.

The second format of the MOVE statement specifies that corresponding fields between two different structures be moved from the first structure to the second structure. The source and destination fields must represent group data items. This statement then moves each field from the source data structure to the corresponding field in the receiving data structure.

The rules for deciding what fields correspond are:

The data items have the same name, including qualifications, up to, but not including, the top-level data names.

The data item is not named FILLER.

At least one of the data items is elementary.

An example of fields which correspond is:

```
01 normal-date.                01 cobol-date.
   02 n-month  pic 99.             02 n-year      pic 99.
   02 n-day    pic 99.             02 month       pic 99.
   02 n-year   pic 99.             02 n-day       pic 99.
```

In this example, the statement:

```
MOVE CORR normal-date TO cobol-date.
```

would be equivalent to the statements:

```
MOVE n-day of normal-date TO n-day of cobol-date.
MOVE n-year of normal-date TO n-year of cobol-date.
```

In the example above, the "n-month of normal-date" data item does not correspond to any field in the "cobol-date" structure and thus would not be moved.

MULTIPLY

The MULTIPLY statement causes numeric data items to be multiplied and saves the result in the specified data item.

Format:

```
MULTIPLY  data-name  BY  data-name  ROUNDED
          literal

   ;  ON  SIZE  ERROR  imperative statement

MULTIPLY  data-name  BY  data-name  GIVING  data-name  ROUNDED
          literal        literal

   ;  ON  SIZE  ERROR  imperative statement
```

Examples:

```
MULTIPLY diameter BY 3.14159 GIVING circumference.
MULTIPLY 10 BY result.
```

In the first format, the second data item is multiplied by the first data item and the result placed in the second data item. In the second format, the first two data items are multiplied and the result placed in the GIVING data item. In both formats, the source operands must be purely numeric data items or literals and the result field must be either numeric or numeric edited. If the result cannot be stored in the destination data item, the SIZE ERROR condition occurs. When this happens, the value of the result data item is undefined, regardless of whether a SIZE ERROR phrase is present in the statement.

OPEN

The OPEN statement initiates the processing of files.

Format:

```
        INPUT
   OPEN OUTPUT  file-name  ,  file-name  ...
        I-O
        EXTEND
```

Examples:

```
   OPEN INPUT library-file, transaction-file
       OUTPUT report-file.
```

An OPEN statement must be executed before any other operation is permitted on a COBOL file. The OPEN statement specifies the name of the file to be opened and the mode in which the file is to be opened.

If the file is open in the INPUT mode, the file must exist and be readable by the user running the COBOL program. In this mode, only input statements (READ, START) may be executed. If the file does not exist and the COBOL file was specified as being OPTIONAL (in the SELECT statement for the file), then the OPEN will be successful, but the end of file condition will exist for that file.

A file which is opened in OUTPUT mode replaces any existing version of the file being opened. In this mode, only output statements (WRITE) may be executed on the file.

The EXTEND mode is very much like the OUTPUT mode except that data records are placed at the end of the file, thus extending and not replacing the data file. Only SEQUENTIAL files may be opened in EXTEND mode.

The I-O mode allows the program to perform both input and output statements on the given file. This mode specifies that existing data files will simply be changed, not completely replaced. If the corresponding UniFLEX file does not exist, the OPEN statement will create it as in OUTPUT mode.

After the execution of the OPEN statement, the file status data item will reflect the state of the file and the success/failure of the OPEN statement. If the file status code is "00" was the OPEN statement successful.

The following table describes permissible statements after the successful opening of a file, depending on the open mode and the file access mode.

File Access Mode	Statement	Open Mode: INPUT	OUTPUT	I-O	EXTEND
SEQUENTIAL	READ	SRI		SRI	
	WRITE		SRI		S
	REWRITE			SRI	
	START	RI		RI	
	DELETE			RI	
RANDOM	READ	RI		RI	
	WRITE		RI	RI	
	REWRITE			RI	
	START				
	DELETE			RI	
DYNAMIC	READ	RI		RI	
	WRITE		RI	RI	
	REWRITE			RI	
	START	RI		RI	
	DELETE			RI	

S - Statement allowed with SEQUENTIAL files
R - Statement allowed with RELATIVE files
I - Statement allowed with INDEXED files

PERFORM

The PERFORM statement transfers program control to one or more procedures and returns control to the statement following the PERFORM statement whenever the specified procedure is complete.

Format:

```
PERFORM  procedure-name  THROUGH  procedure-name
                         THRU

PERFORM  procedure-name  THROUGH  procedure-name  data-name  TIMES
                         THRU                    literal

PERFORM  procedure-name  THROUGH  procedure-name  UNTIL  condition
                         THRU

PERFORM  procedure-name  THROUGH  procedure-name
                         THRU

   VARYING  data-name  FROM  data-name
                             literal

      BY  data-name  UNTIL  condition
          literal

   AFTER    data-name  FROM  data-name
                             literal

      BY  data-name  UNTIL  condition

   AFTER    data-name  FROM  data-name
                             literal

      BY  data-name  UNTIL  condition
```

Examples:

```
PERFORM select-users THRU throw-out-users.
PERFORM count-data 10 TIMES.
PERFORM set-indicators-2 THRU set-indicators-3
  VARYING support-product-index FROM 1 BY 1
    UNTIL support-product-index > max-support-product-index.
```

The function of this statement is to execute the statements which occur within the scope of the mentioned procedure name or procedure names and return control to the statement following the PERFORM statement when those statements have been executed. The statements may be executed unconditionally one or more times, or they may be executed only if certain conditions hold.

The keywords THROUGH and THRU are equivalent. The procedure names must be the names of a paragraph or section appearing in the PROCEDURE DIVISION of the COBOL program.

If the first procedure name is a section name, the second procedure name must also be a section name. In this case, all statements which occur between the beginning of the first section and the end of the second section are within the scope of the PERFORM statement.

If the first procedure name is a paragraph name then the scope of the PERFORM statement extends from the beginning of that paragraph through the end of that paragraph unless the THROUGH clause is present. In this case, the scope extends to the end of the paragraph or section named in the THROUGH clause.

If the scope of one PERFORM statement includes another PERFORM statement, then the scope of the second PERFORM statement must be totally included or totally excluded from the scope of the original PERFORM statement. This implies that the scope of PERFORM statements must not overlap.

Examples of Valid PERFORM Constructs:

```
x  PERFORM  a  THRU  m
a
d  PERFORM  f  THRU  j
f
j
m
```

```
x  PERFORM  a  THRU  m
a
d  PERFORM  f  THRU  j
h
m
f
j
```

Example of Invalid PERFORM Construct:

```
x  PERFORM  a  THRU  m
a
d  PERFORM  f  THRU  j
f
m
j
```

The data item in the TIMES clause must be a positive integer value, either a numeric literal or an integer numeric data item. The value of this data item represents the number of times the indicated statements will be executed.

In the UNTIL form of the PERFORM statement, the indicated statements will be executed as long as the specified condition is not true. This implies that if the condition is initially true, none of the statements within the scope of the PERFORM statement will be executed.

The VARYING form of the PERFORM statement provides a controlled looping construct with one, two or three control variables. Each of these variables must be numeric integer data items. Also the FROM and BY clauses must specify either numeric integer literals or numeric integer data items. The termination conditions may be any valid conditional expression, but normally they will depend on one or more of the control variables.

READ

The READ statement is used to obtain the value of a record in a data file.

Format:

```
READ file-name NEXT RECORD WITH LOCK INTO data-name

   ; AT END imperative statement

READ file-name RECORD WITH LOCK INTO data-name

   ; KEY IS data-name

   ; INVALID KEY imperative statement
```

Examples:

```
READ serial-file NEXT RECORD; AT END MOVE 'n' TO more-data-flag.
READ inx-file; KEY IS primary-key; INVALID KEY GO TO key-error.
```

The READ statement may be executed for any file organization. The file specified in the READ statement must have been previously opened (see the OPEN statement).

In SEQUENTIAL access mode, the READ statement accesses the next logical record in the file. If no records remain when the READ statement is executed, the end of file condition exists. When the end of file condition exists, the AT END statement will be executed if present. For sequential operations, the AT END phrase must be present if no USE procedure has been defined for the file.

INDEXED or RELATIVE files which are open in RANDOM or DYNAMIC mode may access the file either sequentially or randomly using a key value. For RELATIVE files, the READ statement will access the record given by the value of the RELATIVE KEY data item defined for the file. For INDEXED files, the READ statement will access the record given by the primary record key (RECORD KEY) data item, or if the KEY IS phrase is present, the record accessed will be given by the specified key. A READ ... NEXT statement accesses the next sequential record in a file which is open in DYNAMIC mode. Refer to the START statement for a discussion on how the key values apply for INDEXED files.

Records in RELATIVE and INDEXED files may be locked, i.e. made temporarily inaccessable to other programs. If the WITH LOCK phrase is present in the READ statement, then the record accessed will be locked. Locked records may only be accessed by the program which has locked them. If another program has the same file open, that program will be unable to access the locked data records. Records are unlocked by rewriting the record, closing the file or executing an UNLOCK statement for the file.

After the execution of a READ statement, the status of the operation is returned in the file status data item associated with the file. Refer to chapter 3 for complete details on file status codes and how they are interpreted. If any errors are encountered during the READ statement, then the INVALID KEY statement will be executed. If the READ statement has no INVALID KEY statement, a USE procedure must be defined for the file, in which case, that USE procedure will be executed when there is an error.

RELEASE

The RELEASE statement transfers data records to the SORT package.

Format:

```
RELEASE  record-name  FROM  data-name
```

Examples:

```
RELEASE sort-record.
RELEASE sort-record FROM names-record.
```

The RELEASE statement sends one data record to the SORT package. This statement must be executed by the INPUT PROCEDURE of a SORT statement.

When a SORT statement is executed which contains an INPUT PROCEDURE phrase, an implicit file is created which contains the records to be sorted. Records are placed in this file by using the RELEASE statement.

The record name specified in the RELEASE statement must be the name of a data record associated with the file currently being sorted. If the FROM phrase is present, then the FROM data item is first moved to the sort file record area and then the RELEASE statement executed.

RETURN

The RETURN statement is used to read records produced by a SORT or MERGE statement.

Format:

```
RETURN file-name INTO data-name ; AT END imperative statement
```

Examples:

```
RETURN sort-file INTO name-record; AT END MOVE 'n' TO more-data.
```

The RETURN statement accesses the next result record from the output of a SORT or MERGE statement. The RETURN statement must be executed within the scope of an OUTPUT PROCEDURE associated with the SORT or MERGE statement.

When a SORT or MERGE statement is executed with an OUTPUT PROCEDURE phrase, an implicit file is created which contains the result from the SORT or MERGE process. The SORT/MERGE package writes the data records to this file and the RETURN statement is the only way to access the records from this file.

The file name specified must be the name of the sort file currently being sorted or merged. Upon execution of a RETURN statement, the SORT/MERGE package will read the next sequential record from the output file and move it to the data record for the given sort file. If the INTO phrase is present, the data will also be moved to the specified data item. If no more data records are available, the end of file condition occurs and the AT END statement will be executed.

REWRITE

The REWRITE statement replaces a record in a file.

Format:

```
REWRITE  record-name  FROM  data-name
    ;  INVALID  KEY  imperative statement
```

Examples:

```
REWRITE library-record; INVALID KEY GO TO rewrite-error.
REWRITE library-record FROM new-library-record; INVALID KEY STOP RUN.
```

The REWRITE statement logically replaces the data in a specific record in a file with new data. The record name must refer to a data record associated with an open COBOL file. The REWRITE statement may be used with any file organization.

For SEQUENTIAL files, the REWRITE statement replaces the last record read in the file. This implies that a successful READ statement must immediately precede the REWRITE statement for the file. The data record to be written must be exactly the same number of characters as the record read, a restriction which does not apply for the other file organizations.

For RELATIVE and INDEXED files, the record which is replaced depends on the file access mode. If the access mode is SEQUENTIAL then the operation is just as it is for SEQUENTIAL files, i.e. the last record read is the one replaced. For files in RANDOM and DYNAMIC access modes, the record replaced is given by the access keys, either the RELATIVE KEY data item for RELATIVE files, or the RECORD KEY and ALTERNATE KEY data items for INDEXED files.

After the operation, the status of the operation is reflected in the file status data item associated with the file, if defined. If any error is encountered during the REWRITE operation, the INVALID KEY condition exists and the INVALID KEY statement will be executed. If no INVALID KEY statement is present, then a USE section must be provided for the file. The INVALID KEY condition will exist if:

The access mode is SEQUENTIAL and the primary record key data item does not match the key value of the last record read (INDEXED files) or the relative key data item does not match the record number of the last record read (RELATIVE files).

The value in the primary record key data item does not match the key of any record in the file.

The value of an alternate key is the same as that for another record in the file and the WITH DUPLICATES clause was not specified for that key.

SET

The SET statement is used to set or change the value of a table index data item.

Format:

```
SET  data-name  TO        data-name
                UP    BY  literal
                DOWN  BY
```

Examples:

```
SET month-index TO n-month of cobol-date.
SET month-index UP BY 1.
```

The data name must be an index name, defined either as a numeric data item with USAGE INDEX or by an INDEXED BY clause in a data definition. The value must be specified by either a numeric data item or a numeric literal. In either case, the value must be a positive integer.

If the TO form is used, the index data item is set to exactly the specified value. If the UP/DOWN BY form is used, the value of the index data value is incremented/decremented by the specified value.

In this implementation, this statement is equivalent to an appropriate MOVE/ADD/SUBTRACT statement but is included for compatibility.

SORT

The SORT statement is used to sort one or more input files based on a sorting sequence defined by one or more keys, producing a sorted output file.

Format:

```
                        ASCENDING
SORT   file-name  ON    DESCENDING  KEY  data-name  ,  data-name  ...

                        ASCENDING
                  ON    DESCENDING  KEY  data-name  ,  data-name  ...

    INPUT  PROCEDURE  IS  section-name  THROUGH  section-name
                                        THRU

    USING  file-name  ,  file-name  ...

    OUTPUT  PROCEDURE  IS  section-name  THROUGH  section-name
                                         THRU

    GIVING  file-name
```

Examples:

```
SORT sort-file ON ASCENDING KEY sort-key of sort-record
               ON ASCENDING KEY qualified-title of sort-record
     USING names-file
     OUTPUT PROCEDURE IS produce-report-lines.
```

The SORT statement controls the sorting of one or more files producing a sorted output file. The sorting sequence is specified by one or more keys, given as either ASCENDING or DESCENDING. Fields which are specified by an ASCENDING key will be sorted from the smallest key value to the largest. Fields which are specified by a DESCENDING key will be sorted from the largest key value to the smallest. If more than one key specification is given, then the first is the the most significant, the second is the next most significant, etc.

The files to be sorted may be specified literally using the USING phrase. In this case, all the files mentioned must be COBOL file names and must be closed when the SORT statement is executed. The input to the SORT package is the concatenation of all the files given.

Alternatively, the input to the SORT package may be given by a program section, specified by the INPUT PROCEDURE phrase. This phrase must specify a section name to be performed. Before the code is performed, an implicit file is created and opened. Data records may be placed in this file by using the RELEASE statement. When the INPUT PROCEDURE terminates, this file is then given to the SORT package to be sorted. This mechanism is useful for sorting only a selection of records and having a program segment which performs the selection process.

The result of the SORT process may be given as a specific file using the GIVING phrase. This file should be closed at the time of the SORT statement. It will be opened implicitly (in OUTPUT mode, thus replacing any previous contents) and the contents written by the SORT package. The file will be closed at the termination of the sort process.

The output from the sort process may also be obtained using an OUTPUT PROCEDURE. When the input file has been sorted, the program section given in the OUTPUT PROCEDURE phrase will be executed. Just prior to this, an implicit file which contains the result of the sort will be opened for input. Data records may be obtained from this file via the RETURN statement. When no more sorted records are available, the AT END condition will occur. Normally, this is used to terminate the OUTPUT PROCEDURE program segment.

The sort file must have been defined in the FILE SECTION using an SD statement. The keys given in the SORT statement must be alphanumeric data items in one of the data records associated with the sort file. A maximum of 20 keys may be specified, but at least one key must be specified.

START

The START statement is used to specify a starting position for sequential operations in a RELATIVE or INDEXED file.

Format:

```
                              EQUAL TO
                              =
                              GREATER THAN
START  file-name  KEY  IS     >            data-name
                              NOT  LESS
                              NOT  <

    ;  INVALID  KEY  imperative statement
```

Examples:

```
START library-file KEY IS EQUAL TO new-library-key;
  INVALID KEY GO TO start-error.
```

The file name must reference either a RELATIVE or INDEXED file, opened in either SEQUENTIAL or DYNAMIC access mode. If the file is an INDEXED file, the data name specified as the KEY IS value must be either one of the keys defined for the file, or an alphanumeric data item subordinate to one of the keys defined for the file whose leftmost character position is the same as that of the key defined for the file (i.e. the key specified must be a leftmost substring of one of the keys for the file). If the file is a RELATIVE file, the key specified must be exactly the RELATIVE KEY data item defined for the file. If no KEY IS phrase is present, the START statement has the same effect as a START statement with a key value equal to the primary key value.

If the IS EQUAL TO condition is given, then an exact key match must be found.

If any other comparison condition is given, a record which has a key value either greater (GREATER THAN) or greater than or equal to (NOT LESS THAN) to the given key value will be searched for in the file.

If the comparision cannot be satisfied, i.e. the exact key cannot be located for an EQUAL TO comparision, or there is no key value greater (GREATER THAN) or greater than or equal (NOT LESS THAN) in the file, then the START statement is unsuccessful and the INVALID KEY condition exists. If the INVALID KEY phrase is present, the INVALID KEY statement will be executed. If no INVALID KEY phrase is given, a USE procedure must be defined for the file.

After the successful execution of a START statement, subsequent sequential READ statements will access the records in the file, starting with the record which satisfied the START criterion.

When a data record is accessed by a key value using the KEY IS phrase, the specified key is known as the key of reference from the time of that operation until a new key of reference is specified. This key of reference may be specified by either a START statement or a READ statement. For RELATIVE files, the key of reference is always the RELATIVE KEY for the file. For INDEXED files, the key of reference is normally the primary record key. All records in the file must have a unique primary record key value. Sequential operations using the primary record key as the key of reference starting from the beginning of the file will access all records in the file. Sequential operations using an alternate key as the key of reference will only access those records which have the same value for the specific alternate key as the first record accessed via that key. When there are no more records which have this key value, the end of file condition will occur, although there may indeed be many records in the file which have not been accessed.

STOP

The STOP statement causes a permanent or temporary suspension of a COBOL program.

Format:

```
STOP   RUN
       literal
```

Examples:

```
STOP RUN.
STOP "Insert disk - hit C/R to continue:".
```

The first form of the STOP statement causes the program to terminate. There should always be at least one "STOP RUN" statement in every COBOL program. All files which are open at the time of the STOP statement will be closed before the program terminates, although it is good practice for the COBOL program to close files before executing the STOP statement.

The second form of the STOP statement provides a "pause" mechanism to the COBOL program. The literal value will be displayed on the terminal on a new print line with the text "Stop - " preceding the literal value. At this point the COBOL program will wait for the operator to enter characters on the terminal until a carriage return is entered. Once the carriage return has been entered, the program will continue execution with the statement which follows the STOP statement. If the operator wishes to terminate the program at this point, he may enter control-backslash. This form of the STOP statement does not alter any open files unless the program is terminated abnormally.

SUBTRACT / SUBTRACT CORRESPONDING

The SUBTRACT statement computes the difference of two or more numeric values and places the result in a given data item.

Format:

```
SUBTRACT  data-name  ,  data-name  ...  FROM  data-name  ROUNDED
          literal       literal

   ;  ON  SIZE  ERROR  imperative statement

SUBTRACT  data-name  ,  data-name  ...  FROM  data-name
          literal       literal              literal

   GIVING  data-name  ROUNDED

   ;  ON  SIZE  ERROR  imperative statement


SUBTRACT  CORRESPONDING  data-name  FROM  data-name  ROUNDED
          CORR

   ;  ON  SIZE  ERROR  imperative statement
```

Examples:

```
SUBTRACT overtime-hours FROM total-hours GIVING normal-hours.
SUBTRACT 2 FROM count.
SUBTRACT CORR current-data FROM total-data.
```

The SUBTRACT statement subtracts one or more numeric values from a given numeric value and places the difference in a specified data item. If the GIVING phrase is present then the result is stored in the GIVING data item, otherwise the result is stored in the FROM data item.

If more than one data item appears before the FROM clause, then the result is the difference between the FROM data item and the sum of all the data items which appear before the FROM clause. In effect, each of the specified data items is subtracted from the value given in the FROM clause.

If the ROUNDED phrase is present, the value will be rounded before being stored in the destination data item.

If any intermediate result exceeds 18 significant digits or the result cannot be successfully moved into the destination data item the SIZE ERROR condition will exist and the result is undefined. In this case, if the ON SIZE ERROR phrase is present, the statement will be executed.

The data items specified as source values must be either numeric literals or numeric elementary data items. Data items specified as destination fields must be either numeric or numeric edited elementary data items.

The SUBTRACT CORRESPONDING form must reference two group data items. The SUBTRACT CORRESPONDING statement subtracts all corresponding data fields in the first group item from the data fields in the second group. If any of the operations result in a SIZE ERROR condition, the entire operation is undefined and the ON SIZE ERROR statement will be executed. Refer to MOVE CORRESPONDING for a discussion on how corresponding operations are defined.

SUPPRESS

The SUPPRESS statement causes the Report Writer Control System (RWCS) to inhibit the processing of a report group.

Format:

```
SUPPRESS  PRINTING
```

Examples:

```
SUPPRESS  PRINTING.
```

This statement may only appear in a USE BEFORE REPORTING paragraph. When the report group specified in the USE statement is to be presented, then the USE paragraph will be executed. The function of this statement is to suppress any output which would normally result from the presentation of the given report group.

TERMINATE

This statement instructs the Report Writer Control System (RWCS) to complete the processing of a given report.

Format:

```
TERMINATE  report-name
```

Examples:

```
TERMINATE audit-report.
```

The report named must have been previously initiated using the INITIATE statement. The execution of this statement causes the CONTROL FOOTING report groups to be presented in order from highest to lowest (refer to the section on the GENERATE verb for a discussion of the ordering of control variables). The REPORT FOOTING report group will then be presented.

If no GENERATE statements have been executed for the report, then the TERMINATE statement will not cause any report groups to be presented. That is, if no GENERATE statements are executed for a given report, the report will be totally empty.

The TERMINATE statement does not close the file associated with the report and thus a separate CLOSE statement should be included in the COBOL program to close the report file.

UNLOCK

The UNLOCK statement unlocks all records in an INDEXED or RELATIVE file currently locked by the COBOL program.

Format:

```
UNLOCK file-name  RECORD
                  RECORDS
```

Examples:

```
UNLOCK library-file.
```

The UNLOCK statement unlocks all records currently locked by the running COBOL program in an INDEXED or RELATIVE file. The file name must reference an open INDEXED or RELATIVE file. The status of the operation is reflected in the file status data item for the file after the operation.

This statement may be executed whether or not any records are locked in the file.

USE

The USE statement specifies a section of the COBOL program which is to be executed when certain conditions exist during the running of the program.

Format:

```
USE AFTER STANDARD EXCEPTION PROCEDURE ON file-name ...
                   ERROR
```

Format 2:

```
USE BEFORE REPORTING report-group-name
```

Examples:

```
USE AFTER STANDARD ERROR PROCEDURE ON library-file.
USE BEFORE REPORTING audit-count-footer.
```

This statement is only allowed in the DECLARATIVES section of the PROCEDURE DIVISION and only after a SECTION heading. The section of the program being defined is then associated with either a set of files or with a specific report group.

The first format of the USE statement associates a section of the COBOL program with a file or files used by the program. If there are any error conditions encountered when processing one of the files mentioned and the statement which caused the error to occur does not include an error handling clause (AT END, INVALID KEY), then the section corresponding to the USE statement will be performed. This allows the programmer to specify in a single location in the COBOL program the actions to take after an error condition exists for a given file.

No USE statement is required for any file, but if none is specified then error handling clauses must be included in most input/output statements present in the program.

The second format of the USE statement associates the program section with a specific report group. The report group named may be any type of report group, defined in the REPORT SECTION of the program. When the report group specified is about to be presented by the RWCS, this program section will be executed before any output is produced. The most common use of such a section of code is to suppress the output from a given report group under certain conditions. This may be done by executing a SUPPRESS PRINTING statement within the USE program section. In fact, the SUPPRESS PRINTING statement may only occur in this context in the COBOL program.

WRITE

The WRITE statement places a new record in a data file.

Format:

```
   WRITE  record-name  FROM  data-name

                         data-name  LINE
      BEFORE  ADVANCING  integer    LINES
      AFTER              PAGE

      ;  AT  END-OF-PAGE  imperative statement
             EOP

   WRITE  record-name  FROM  data-name

      ;  INVALID  KEY  imperative statement
```

Examples:

```
   WRITE library-record FROM new-library-record;
     INVALID KEY GO TO write-error.
   WRITE detail-line AFTER ADVANCING 2 LINES;
     AT EOP PERFORM top-of-page.
```

The two formats of the WRITE statement correspond to SEQUENTIAL file operations and INDEXED or RELATIVE file operations, respectively.

For SEQUENTIAL files, records are appended, one at a time, to a file using the WRITE statement. Special consideration has been given to files which may eventually be printed on hard copy devices, possibly with some page structure. The WRITE statement may specify a number of blank (empty) lines which are to be inserted before the data record is written (AFTER ADVANCING), or a number of blank lines to be appended after the data record is written (BEFORE ADVANCING). Alternatively, the WRITE statement may specify that the data record appear at the top of the following page (AFTER ADVANCING PAGE) or that the form be positioned to the top of a new page after the data record is written (BEFORE ADVANCING PAGE). If the FD statement which describes the file contains the LINAGE clause (see chapter 7), then the appropriate number of blank lines will be inserted in the output file to achieve this alignment with the top of form. If the LINAGE clause is not present, the top of form will be achieved by the insertion of a line which contains a form feed character ($0C).

If the file has the LINAGE clause specified then after the last line on the page has been written on or spaced to, the end of page condition will exist. If the END-OF-PAGE phrase is present in the statement, then it will be executed after the completion of the write statement. When the END-OF-PAGE statement is executed, no more lines can be written on the current page without advancing to the top of a new page. This statement is most commonly used to print page headers, etc.

For RELATIVE and INDEXED files, data records need not have consecutive key values, i.e. the file may contain holes by simply not writing records where the holes are. Which record is written in the file depends on the file access mode.

If the access mode is SEQUENTIAL then records are appended to the end of the file. If the primary record key (INDEXED files) or the relative key (RELATIVE files) already exists or does not have a value higher than all other keys in the file, the INVALID KEY condition will exist.

If the access mode is RANDOM or DYNAMIC, the record written depends on the relative key value (RELATIVE files) or the primary record key and alternate record key values (INDEXED files). In this case, the current record key value need have no relationship with the previous record as in SEQUENTIAL access mode. If the relative key value (RELATIVE files) or the primary key value (INDEXED files) already exists in the file, the INVALID KEY condition exists.

VIII. THE PROCEDURE DIVISION

The PROCEDURE DIVISION of a COBOL program contains the statements which describe the functions and operations to be performed by the program. This includes statements to perform input/output, numeric calculations, etc. The PROCEDURE DIVISION is constructed of named paragraphs, each paragraph being composed of a sequence of statements. Paragraphs may also be grouped into named sections.

Format 1:

```
PROCEDURE  DIVISION  USING  data-name  ,  data-name  ...  .

DECLARATIVES.

   section-name  SECTION.   Use Statement
   paragraph-name.  sentence  ...   ...   ...

END DECLARATIVES.


section-name  SECTION.
paragraph-name.  sentence   ...   ...   ...
```

Format 2:

```
PROCEDURE  DIVISION  USING  data-name  ,  data-name  ...  .

paragraph-name.  sentence   ...   ...   ...
```

ACCEPT

The ACCEPT verb is used to obtain input from the terminal on which the COBOL program is running. The input format may be described using the SCREEN SECTION or input may be free format.

Format:

```
ACCEPT screen-name ; ON ESCAPE imperative statement
```

Format 2:

```
ACCEPT data-name

    , LINE  identifier
            literal

    , POSITION  identifier
                literal


    , SIZE  identifier
            literal

    , PROMPT

    , ECHO

    , CONVERT

    , SECURE
```

Format 3:

```
                          DAY
ACCEPT data-name  FROM    DATE
                          TIME
```

Examples:

```
ACCEPT menu-0; ON ESCAPE GO TO next-program-function.
ACCEPT cobol-date FROM DATE.
```

There are two unique functions performed by the ACCEPT statement. These functions are represented by the three different formats.

The first format allows an operator to enter data on a terminal device for use by a COBOL program. In most cases, this terminal device is a CRT terminal, and for this purpose, the ACCEPT verb has built-in features which make use of the facilities of CRT display terminals. If the data-name given in the ACCEPT statement is the name of a screen description, then the system will position the cursor on the terminal to the first data field in the screen description which contains either a TO or a USING field. The operator may then enter data for this field, or leave the data unaltered by hitting the TAB key. The cursor will then be positioned to the next input field. The cursor may be repositioned to a previous field in the screen by entering the "back-tab" character. Since not all terminals have a back-tab character defined, COBOL uses the Line Feed character for this purpose. The cursor may be moved backwards as many times as desired, or until the cursor reaches the first field in the screen. This process continues until data has been entered for the last input field in the screen description or the operator enters the control-C key. If the operator enters a control-C, all screen input functions will be aborted and the ON ESCAPE sentence executed. If no ON ESCAPE sentence is present, the program will terminate. See chapter 7 for a complete discussion of screen formatted data using the SCREEN SECTION.

The second format is very similar to the first format in that the user is allowed to enter data from the terminal. In this format, however, each input field is described as it is being input using the optional fields for the statement. The LINE clause specifies the absolute line number on the screen which the cursor is to be positioned before accepting data. The POSITION clause specifies the cursor's horizontal position. The SIZE field specifies how wide the field to be accepted is. If this clause is not present, then sufficient characters to fill the data item will be accepted. The PROMPT clause, if present, instructs the program to display a prompt for the field when it is being accepted. This is done by writing the appropriate number of underscore ('_') characters on the screen at the position where data is to be entered. If the PROMPT clause is not present, the data on the screen is not altered until characters are entered. If the SECURE clause is present, the asterisk character '*' will be displayed on the screen for all characters entered into the field. The ECHO and CONVERT clauses are currently accepted by the compiler but have no runtime function.

The third format of the ACCEPT statement is used to obtain the current day, date or time in a format usable by the COBOL program. These values are considered to be numeric source fields and are moved to the given data-item using the standard move mechanism. The format of the DAY field is: YYDDD, where YY represents the current year (modulo 1900) and DDD represents the Julian day for the current day. The format of the DATE field is: YYMMDD, where YY is the current year, MM is the current month (January being 01 and December being 12) and DD is the day of the month. The format of the TIME field is: HHMMSS0000, where HH is the current hour (24 hour clock), MM is the current minute and SS is the number of seconds in the current minute. The last four (4) digits of this field are always zero.

ADD / ADD CORRESPONDING

The ADD verb is used to add two or more numeric items to obtain their sum.

Format:

```
ADD  data-name   ,  data-name   ...  TO  data-name  ROUNDED
     literal        literal

   ;  ON  SIZE  ERROR  imperative statement
```

```
ADD  data-name  ,  data-name   ...  GIVING  data-name  ROUNDED
     literal       literal

   ;  ON  SIZE  ERROR  imperative statement
```

```
ADD  CORRESPONDING  data-name  TO  data-name  ROUNDED
     CORR

   ;  ON  SIZE  ERROR  imperative statement
```

Examples:

```
ADD amount TO balance; ON SIZE ERROR GO TO balance-overflow.
ADD val-1, val-2 GIVING val-3 ROUNDED.
```

The ADD verb computes the sum of two or more operands, either GIVING a result or adding TO an existing value. All source operands must be numeric data items or numeric literals. If the GIVING form of the verb is used, the receiving identifier may be either numeric or numeric-edited. In all other forms, the receiving identifier must be numeric.

If the result exceeds 18 total digits or the result cannot be moved successfully into the receiving data field, then a SIZE ERROR will occur and the result stored is undefined.

In the ADD CORRESPONDING form, all corresponding numeric fields in the data items are added, with the resultant sum replacing the value in the second data item. The SIZE ERROR will occur if any of the additions exceed 18 total digits. In this case, the values of all the corresponding fields will be undefined. See MOVE CORRESPONDING for a complete description of how corresponding operations are defined.

CALL

The CALL PROGRAM statement allows a COBOL program to invoke the execution of another program.

Format:

```
CALL  PROGRAM  data-name  USING  data-name  ,  data-name   ...
               literal

    ;  ON  EXCEPTION  imperative statement
           OVERFLOW
```

Examples:

```
CALL PROGRAM "prog-1" USING title, page-no.
CALL PROGRAM "+copy file-1 file-2".
```

The CALL PROGRAM statement allows a COBOL program to invoke the execution of another program. There are two ways that a program may be called. First, a COBOL program may call another COBOL program, possibly passing some information to that program via the USING clause. In this case, the called program replaces the calling program, using a chain-like mechanism. The identifier or literal in the statement is the name of the program to be invoked and should be a well-formed UniFLEX file pathname. The called program _ul must be a COBOL program and if the called program contains a USING clause in its PROCEDURE DIVISION header, then the calling program should contain a similar USING clause in the CALL PROGRAM statement. In this form of the CALL PROGRAM statement, the called program never returns to the calling program. If the program to be executed cannot be located or executed, the COBOL program will continue execution with the statement following the CALL PROGRAM. For this form of the CALL PROGRAM statement, control returns to the calling program only in the case of an error condition. The calling COBOL program should be written to take care of this eventuality.

There is a second form of the CALL PROGRAM which does allow the called program to return control to the calling program. Using this form, any variety of program may be called, not just COBOL programs. This form of the CALL PROGRAM statement is distinguished from the first by the presence of the character "+" as the first character of the name of the called program. In this case, the identifer or literal in the CALL PROGRAM statement is actually a command string passed to a shell. When the shell program terminates, the COBOL program continues with the statement following the CALL PROGRAM statement. There is no provision for passing information to the called program when using this form of the CALL PROGRAM statement (except the standard UniFLEX parameter mechanism) as the USING phrase is not allowed.

CLOSE

The CLOSE verb is used to terminate all input/output operations on a file.

Format:

```
CLOSE  file-name  ,  file-name  ...
```

Examples:

```
CLOSE serial-file.
CLOSE data-base, report-file.
```

The purpose of this statement is to terminate all activity on a file. This function is known as "closing" the file. After the file is closed, no other operations except an OPEN statement or DELETE FILE statement may be executed on the file.

The state of the file after the CLOSE file is reported in the file status bytes associated with the file.

It is good programming practice to close all open files before the program terminates. However, the runtime system will close any open files when the program terminates, either normally or abnormally.

COMPUTE

The COMPUTE verb is used to assign the result of a numeric computation to a variable. This statement is a shorthand form of the arithmetic statements ADD, SUBTRACT, MULTIPLY and DIVIDE.

Format:

```
COMPUTE  data-name  ROUNDED  =  arithmetic expression

   ;  ON  SIZE  ERROR  imperative statement
```

Examples:

```
COMPUTE circumference = diameter * 3.14159.
COMPUTE net-pay ROUNDED = (hours * pay-rate) - deductions.
```

This statement assigns the value of an arithmetic expression to a data-item. The receiving data item may be either numeric or numeric edited data item. The expression is made up of terms using either numeric data items or numeric literals and the operators +, -, *, and /.

If the keyword ROUNDED is specified, then the result will be rounded before the value is assigned.

If the result value cannot be successfully moved into the receiving data item, or if any intermediate result exceeds 18 significant digits, then the SIZE ERROR condition occurs and the value of the receiving field is undefined.

COPY

The COPY statement causes lines from a file to be inserted into the text of a COBOL program.

Format:

```
COPY literal-file-name
```

Examples:

```
COPY "seq-file-fd".
COPY "/usr/include/std-defs".
```

The COPY statement allows a COBOL program to include the text of a file into the source of the COBOL program. The COPY statement is replaced by the contents of the specified file. The literal must be a well-formed UniFLEX file pathname. The copied file must not contain any COPY statements, i.e. COPY statements are not allowed to be nested. The COPY statement must be the only statement on the source line. Anything which appears to the right of the period on the same line as the COPY statement will be ignored.

DELETE

The DELETE verb is used to delete specific records from a file or to delete an entire file from the file system.

Format:

```
DELETE file-name RECORD ; INVALID KEY imperative statement
```

Format 2:

```
DELETE FILE file-name
             literal
```

Examples:

```
DELETE random-file RECORD;  INVALID KEY go to delete-error.
DELETE FILE random-file.
DELETE FILE "/library/books-file".
```

There are two forms of this verb. The first form allows the COBOL program to delete a specific record from a RELATIVE or INDEXED file. This verb works differently depending on the ACCESS MODE of the file.

If the file is open in SEQUENTIAL access mode, then the COBOL program must have successfully executed a READ operation on the file immediately prior to the DELETE verb. In this case, the record just read will be the record deleted.

If the file is open in RANDOM or DYNAMIC access mode, then the record to be deleted is given by the record access key, either a relative key (record number) for RELATIVE files, or the primary and alternate record keys for INDEXED files.

After the execution of the DELETE verb, the file status item associated with the file will contain information about the operation. Refer to chapter 3 for complete details on possible file status codes. If the operation fails for any reason, then the INVALID KEY condition exists and the statement in the INVALID KEY clause will be executed. A USE procedure must be defined for the file if the INVALID KEY clause is not present. If both the INVALID KEY clause and a USE procedure are present, then only the INVALID KEY clause will be executed.

The second form of the DELETE verb allows the COBOL program to delete an entire file. This is not a standard ANSI COBOL feature, but has been added to make COBOL more useful. The file to be deleted may be specified either by specifying a COBOL file name in which case the file associated with that COBOL file will be deleted, or by specifying the actual name of the file to be deleted. The user should be cautious when using this verb because if the file to be deleted cannot actually be deleted, the program will abort in the attempt. The most common use for

this mechanism is to clean up scratch files used by a COBOL program when the program has no more need for them.

DISPLAY

The DISPLAY verb is used to display data values on the user's terminal.

Format:

```
DISPLAY  screen-name  ,  screen-name  ...
```

Format 2:

```
DISPLAY  data-name  ,  LINE  data-name  ,  POSITION  data-name
         literal             literal                 literal

   ,  SIZE  data-name
            literal
```

Examples:

```
DISPLAY menu-0.
DISPLAY "*** I/O Error ***", LINE 10, POSITION 1.
```

This verb displays data on the user's terminal. For the most part, it is assumed that this terminal is a CRT device which may be used to display formatted data. To this end, there are two forms of the verb, both designed to give the COBOL programmer wide control over what is displayed on the terminal, where it is displayed and the format it is displayed in.

The first format is used in conjunction with the SCREEN SECTION of the program. Data to be displayed are described in this section using screen record descriptions. See chapter 7 for more detail on the SCREEN SECTION.

When using the first format of the DISPLAY verb, all fields in the given screen descriptions which contain VALUE, FROM or USING clauses will cause data to be displayed on the terminal screen. The terminal is assumed to have two modes of display output. Foreground mode is brighter and represents data which may possibly be changed by the terminal operator. Background mode is lighter and represents data which cannot be modified. (If the terminal being used does not support these different modes, then all data will be displayed in the terminal's normal mode). Values which are specified in a USING clause will be displayed in foreground mode. All other values will be displayed in background mode.

The second format of the DISPLAY verb allows the COBOL program to display data values, one data value at a time, specifying information about where and how to display the value directly in the DISPLAY statement. The vertical position where the data is to be displayed is given by the LINE clause. This clause specifies the absolute vertical position for the data either in the form of a numeric literal value, or given by the value of a numeric data item. If the LINE clause is not specified, then the data is displayed at the current vertical position of the cursor. The horizontal position of the data is given by the POSITION clause. If the POSITION clause is not present and the LINE clause is, then data will be displayed starting in column 1 of the specified line. The SIZE clause allows the COBOL program to specify how many characters to display for a particular data item.

If no LINE or POSITION clause is present, the data will be displayed at the current cursor position and the cursor will be placed at the beginning of the next line following the completion of the DISPLAY statement.

A given COBOL program may use both forms of the DISPLAY verb, but the two formats must be specified separately. That is, a single DISPLAY statement may not contain both screen descriptions and non-screen data items to be displayed.

DIVIDE

The DIVIDE statement divides one numeric data item into another and sets the value of data items to the quotient and remainder.

Format:

```
DIVIDE  data-name  INTO  data-name  ROUNDED
        literal

   ;  ON  SIZE  ERROR  imperative statement


DIVIDE  data-name  INTO  data-name  GIVING  data-name  ROUNDED
        literal          literal

   REMAINDER  data-name  ;  ON  SIZE  ERROR  imperative statement


DIVIDE  data-name  BY  data-name  GIVING  data-name  ROUNDED
        literal        literal

   REMAINDER  data-name  ;  ON  SIZE  ERROR  imperative statement
```

Examples:

```
DIVIDE 3.14159 INTO circumference.
DIVIDE number-of-items INTO total-cost GIVING cost-per-item;
  ON SIZE ERROR go to bad-divide-handler.
DIVIDE circumference BY 3.14159 GIVING diameter.
```

In the INTO format, the DIVIDE statement divides the first data item into the second data item, producing a quotient and remainder. If the GIVING phrase is present, then the quotient is moved into the data item specified by the GIVING phrase, otherwise the quotient is moved into the data item specified by the INTO phrase. In the BY format, the first data item is divided by the second data item and the resulting quotient is moved into the data item specified by the GIVING phrase.

If the REMAINDER phrase is present, then the integer remainder of the division is moved into the data item specified. If this phrase is not specified, then the remainder of the division is discarded.

The resulting value, as well as any intermediate values, must contain no more than 18 significant digits, otherwise the SIZE ERROR condition will occur. Also, if either the quotient or remainder value cannot be moved successfully into the appropriate result field or if the divisor is zero, then the SIZE ERROR condition will occur. If the SIZE ERROR condition occurs, then the result of the operation is undefined and the ON SIZE ERROR statement will be executed.

All source values for this operation must be either numeric literals or numeric data items. The receiving data fields may be numeric or numeric edited data items.

EXIT

The EXIT statement provides a common end point for a series of procedures.

Format:

```
EXIT.
```

Examples:

```
EXIT.
```

The EXIT statement is used to exit a paragraph or section which has been "PERFORMED". The EXIT statement must be the only statement in a paragraph and is primarily used to provide an exit point which can be branched to using the GO TO statement. The EXIT statement has no effect if the paragraph in which it occurs is not the termination of a current PERFORM statement. That is, the statement must occur either as the last statement in a performed section or as the only statement in a paragraph which is mentioned in a THRU clause in a PERFORM statement.

GENERATE

The GENERATE statement directs the Report Writer Control System (RWCS) to produce a report in accordance with a report description given in the REPORT SECTION.

Format:

```
GENERATE  report-detail-group-name
```

Examples:

```
GENERATE summary-detail-line.
```

The GENERATE statement is the only statement used in conjunction with the Report Writer feature which actually generates report data.

The report detail group name specified in the GENERATE statement must refer to a TYPE DETAIL report group and may be qualified by the report name for that group. The report must have been previously initiated, using the INITIATE statement.

The first GENERATE statement executed after the report is initiated will cause any REPORT HEADING report group to be executed. This is followed by the presentation of all CONTROL HEADING report groups, beginning with the lowest control up through and including the highest control variable. Control variables are ordered from lowest to highest based on the order they appear in the CONTROLS ARE clause in the RD statement. The first control variable is known as the lowest control, up to the last control variable which is referred to as the highest.

Before the DETAIL report group is presented, the current value of all control variables will be checked and if any of these values has changed since the last GENERATE statement for this report was executed, a control break will occur. When a control break occurs, the CONTROL FOOTING report groups for each control higher than or equal to the breaking control will be presented, in reverse order beginning with the highest control variable. Then all CONTROL HEADING report groups for the control variables higher than or equal to the breaking control will be presented, in order beginning with the breaking control and ending with the highest control.

After presentation of the DETAIL report group, all sum counters will be updated.

If any report group which is presented causes the logical page to become full, then the PAGE FOOTING report group, if any, will be presented. This is followed by presentation of the PAGE HEADING report group. This comment applies to all report group presentations: those caused either directly or indirectly by a GENERATE statement.

GO TO

The GO TO statement causes control to be transferred from one part of the PROCEDURE DIVISION to another.

Format:

```
GO TO  procedure-name

GO TO  procedure-name , procedure-name ... DEPENDING ON data-name
```

Examples:

```
GO TO rewrite-error.
GO TO function-1, function-2, function-3 DEPENDING ON function-choice.
```

The first form of the GO TO statement is an unconditional transfer of control. The second form allows a choice of destinations, based on the value of the "DEPENDING ON" identifier. This identifier must describe a numeric data item with an integer value. If the data item has the value 1, then control is transferred to the first label in the list. If it has the value 2, control passes to the second label, etc. If the data item has a value less than 1 or greater than the number of labels in the list, execution continues with the statement following the GO TO statement.

IF

The IF statement evaluates a conditional expression and based on the truth value of that expression, executes one of two possible statements.

Format:

```
IF condition ; statement        ; ELSE statement
                NEXT SENTENCE     ELSE NEXT SENTENCE
```

Examples:

```
IF program-choice IS NUMERIC; PERFORM proc-1; ELSE PERFORM proc-2.
IF serial-status NOT = "00" GO TO open-error.
```

Statement-1 and statement-2 may be any statement, including IF statements. The ELSE NEXT SENTENCE phrase may be omitted.

If the conditional expression is true, the statement sequence following the conditonal expression, up to the keyword ELSE, is executed and the statement sequence which follows the ELSE is not executed. If the conditional expression is false, the statement sequence following the keyword ELSE is executed.

INITIATE

The INITIATE statement causes the Report Writer Control System (RWCS) to begin the processing of a report.

Format:

```
INITIATE  report-name
```

Examples:

```
INITIATE audit-report.
```

The report name must have been defined in an RD statement in the REPORT SECTION and must not have had an INITIATE statement executed on it without a TERMINATE statement intervening. The file associated with the report must be open in either OUTPUT or EXTEND mode.

Upon execution of the INITIATE statement, the following functions are performed:

All sum counters are set to zero.

The LINE-COUNTER value for this report is set to zero.

The PAGE-COUNTER value for this report is set to one.

INSPECT

The INSPECT statement provides the ability to tally (count), replace, or tally and replace occurrences of single characters within a data item.

Format:

```
INSPECT  data-name

                                    ALL         data-name
   TALLYING  ,  data-name  FOR  ,   LEADING     literal
                                    CHARACTERS

      BEFORE            data-name
      AFTER    INITIAL  literal    ...  ...

   REPLACING

                     data-name   BEFORE             data-name
      CHARACTERS  BY literal     AFTER    INITIAL   literal

      ALL
      LEADING  ,  data-name  BY  data-name
      FIRST       literal        literal

         BEFORE            data-name
         AFTER    INITIAL  literal    ...  ...
```

Examples:

```
INSPECT middle-name TALLYING space-count ALL SPACES AFTER INITIAL ".".
INSPECT num-item REPLACING LEADING SPACES BY ZEROES.
INSPECT num-item TALLYING space-count FOR LEADING SPACES
                 REPLACING LEADING SPACES BY ZEROES.
```

The INSPECT data item must reference a data item described as USAGE DISPLAY. The TALLYING data item must be a numeric data item. All other data items must be USAGE DISPLAY.

The INSPECT statement searches a given data item for specific characters and either counts (tallies) occurrences or replaces occurrences of characters with other characters. The search may be qualified by using the BEFORE/AFTER clause. If the BEFORE clause is specified, then the search is restricted to all characters in the data item which occur before (and not including) the character given by the BEFORE clause. If the character in the BEFORE clause does not occur in the INSPECT data item, then the all characters in that data item will be searched. Similarly, the AFTER clause restricts the search to all characters following a given character. If the character in the AFTER clause is not found, then no characters in the INSPECT data item will be searched.

The characters to be searched for are specified by the ALL/LEADING/CHARACTERS clause or the ALL/LEADING/FIRST clause. The ALL clause specifies that all occurrences of the given character are to be used in the search. The LEADING clause specifies that only those characters which occur consecutively from the starting position are considered. The CHARACTERS selection clause signifies any possible character. The FIRST selection clause signifies only the first occurrence of the given character and not any subsequent occurrences.

The TALLYING option of the INSPECT verb searches the given data item and counts occurrences of a given character. For every occurrence of the search character (given by the ALL/LEADING/CHARACTERS) clause, the tally data item will be incremented.

The REPLACING option of the INSPECT statement allows characters to be replaced by other characters. The characters which are to be replaced are specified by the ALL/LEADING/FIRST clause. For every character which satisfies the selection, that character is replaced by the character given by the BY clause.

More than one TALLYING or REPLACING option may be present in a given INSPECT statement. When more than one of these optional phrases is present, they will be executed from "top" to "bottom", starting with the phrase closest to the INSPECT verb itself and continuing to the end of the statement. The phrases are not executed in parallel, so some care must be taken to avoid side-effects produced by the REPLACING clause.

MERGE

The MERGE statement combines two or more identically sequenced files on a set of specified keys, and during the process makes records available, in merge order, to an output procedure or an output file.

Format:

```
                        ASCENDING
MERGE  file-name  ON    DESCENDING  KEY  data-name  ,  data-name  ...

                        ASCENDING
                  ON    DESCENDING  KEY  data-name  ,  data-name  ...

    USING  file-name  ,  file-name  ,  file-name  ...

    OUTPUT  PROCEDURE  IS  section-name  THROUGH  section-name
                                         THRU

    GIVING  file-name
```

Examples:

```
MERGE merge-file ON merge-key of merge-record
  USING input-file-1, input-file-2
  GIVING output-file.
```

The MERGE statement will merge all records in the files given in the USING phrase, as specified by the key ordering given by the ON ASCENDING/DESCENDING KEY phrases. This merged file may then be assigned directly to a separate file by using the GIVING phrase, or may be processed one at a time, in merge order, by specifying the OUTPUT PROCEDURE phrase.

Before execution of the MERGE statement, all files named in the statement must be closed. The file named in the GIVING clause will be implicitly opened for OUTPUT, written, and then closed.

The sorting sequence keys must reference alphanumeric data items which are defined as part of the record definition associated with the merge file. There may be no more than 20 key specifications given, but at least one must be given. When the ASCENDING phrase is specified, the merge sequence will be from the lowest value present for that given key to the highest value for the key. When the DESCENDING key phrase is specified, the merge sequence is from the highest key value to the lowest.

If more than one key specification is given, the first key specified is the most significant, the second key is the next most significant, etc.

MOVE / MOVE CORRESPONDING

The MOVE statement transfers data from one data area to another in accordance to the rules of editing.

Format:

```
MOVE  data-name  TO  data-name
      literal
```

Format 2:

```
MOVE CORRESPONDING  data-name  TO  data-name
     CORR
```

Examples:

```
MOVE ALL SPACES TO name.
MOVE serial-no of serial-record TO input-serial-no.
MOVE CORR current-date TO work-date.
```

Any MOVE in which the sending and receiving items are both elementary items is an elementary move. Every elementary item belongs to one of the following categories: numeric, alphabetic, alphanumeric, numeric edited, alphanumeric edited. These categories are described by the PICTURE clause when a data item is defined. The figurative constant SPACE belongs to the category alphabetic. Numeric literals and the figurative constant ZERO belong to the category numeric. Nonnumeric literals and all other figurative constants belong to the category alphanumeric.

The following rules apply to an elementary move between these categories:

The figurative constant SPACE, a numeric edited, alphanumeric edited, or alphabetic data item must not be moved to a numeric or numeric edited data item.

A numeric literal, the figuative constant ZERO, a numeric data item or a numeric edited data item must not be moved to an alphabetic data item.

A non-integer numeric literal or non-integer numeric data item must not be moved to an alphanumeric or alphanumeric edited data item.

The second format of the MOVE statement specifies that corresponding fields between two different structures be moved from the first structure to the second structure. The source and destination fields must represent group data items. This statement then moves each field from the source data structure to the corresponding field in the receiving data structure.

The rules for deciding what fields correspond are:

The data items have the same name, including qualifications, up to, but not including, the top-level data names.

The data item is not named FILLER.

At least one of the data items is elementary.

An example of fields which correspond is:

```
01 normal-date.                01 cobol-date.
   02 n-month  pic 99.             02 n-year      pic 99.
   02 n-day    pic 99.             02 month       pic 99.
   02 n-year   pic 99.             02 n-day       pic 99.
```

In this example, the statement:

```
MOVE CORR normal-date TO cobol-date.
```

would be equivalent to the statements:

```
MOVE n-day of normal-date TO n-day of cobol-date.
MOVE n-year of normal-date TO n-year of cobol-date.
```

In the example above, the "n-month of normal-date" data item does not correspond to any field in the "cobol-date" structure and thus would not be moved.

MULTIPLY

The MULTIPLY statement causes numeric data items to be multiplied and saves the result in the specified data item.

Format:

```
MULTIPLY  data-name  BY  data-name  ROUNDED
          literal

   ;  ON  SIZE  ERROR  imperative statement

MULTIPLY  data-name  BY  data-name  GIVING  data-name  ROUNDED
          literal        literal

   ;  ON  SIZE  ERROR  imperative statement
```

Examples:

```
MULTIPLY diameter BY 3.14159 GIVING circumference.
MULTIPLY 10 BY result.
```

In the first format, the second data item is multiplied by the first data item and the result placed in the second data item. In the second format, the first two data items are multiplied and the result placed in the GIVING data item. In both formats, the source operands must be purely numeric data items or literals and the result field must be either numeric or numeric edited. If the result cannot be stored in the destination data item, the SIZE ERROR condition occurs. When this happens, the value of the result data item is undefined, regardless of whether a SIZE ERROR phrase is present in the statement.

OPEN

The OPEN statement initiates the processing of files.

Format:

```
        INPUT
   OPEN OUTPUT  file-name  ,  file-name  ...
        I-O
        EXTEND
```

Examples:

```
   OPEN INPUT library-file, transaction-file
       OUTPUT report-file.
```

An OPEN statement must be executed before any other operation is permitted on a COBOL file. The OPEN statement specifies the name of the file to be opened and the mode in which the file is to be opened.

If the file is open in the INPUT mode, the file must exist and be readable by the user running the COBOL program. In this mode, only input statements (READ, START) may be executed. If the file does not exist and the COBOL file was specified as being OPTIONAL (in the SELECT statement for the file), then the OPEN will be successful, but the end of file condition will exist for that file.

A file which is opened in OUTPUT mode replaces any existing version of the file being opened. In this mode, only output statements (WRITE) may be executed on the file.

The EXTEND mode is very much like the OUTPUT mode except that data records are placed at the end of the file, thus extending and not replacing the data file. Only SEQUENTIAL files may be opened in EXTEND mode.

The I-O mode allows the program to perform both input and output statements on the given file. This mode specifies that existing data files will simply be changed, not completely replaced. If the corresponding UniFLEX file does not exist, the OPEN statement will create it as in OUTPUT mode.

After the execution of the OPEN statement, the file status data item will reflect the state of the file and the success/failure of the OPEN statement. If the file status code is "00" was the OPEN statement successful.

The following table describes permissible statements after the successful opening of a file, depending on the open mode and the file access mode.

File Access Mode	Statement	Open Mode INPUT	OUTPUT	I-O	EXTEND
SEQUENTIAL	READ	SRI		SRI	
	WRITE		SRI		S
	REWRITE			SRI	
	START	RI		RI	
	DELETE			RI	
RANDOM	READ	RI		RI	
	WRITE		RI	RI	
	REWRITE			RI	
	START				
	DELETE			RI	
DYNAMIC	READ	RI		RI	
	WRITE		RI	RI	
	REWRITE			RI	
	START	RI		RI	
	DELETE			RI	

S - Statement allowed with SEQUENTIAL files
R - Statement allowed with RELATIVE files
I - Statement allowed with INDEXED files

PERFORM

The PERFORM statement transfers program control to one or more procedures and returns control to the statement following the PERFORM statement whenever the specified procedure is complete.

Format:

```
PERFORM  procedure-name  THROUGH  procedure-name
                         THRU

PERFORM  procedure-name  THROUGH  procedure-name  data-name  TIMES
                         THRU                    literal

PERFORM  procedure-name  THROUGH  procedure-name  UNTIL  condition
                         THRU

PERFORM  procedure-name  THROUGH  procedure-name
                         THRU

   VARYING  data-name  FROM  data-name
                             literal

      BY  data-name  UNTIL  condition
          literal

   AFTER    data-name  FROM  data-name
                             literal

      BY  data-name  UNTIL  condition

   AFTER    data-name  FROM  data-name
                             literal

      BY  data-name  UNTIL  condition
```

Examples:

```
PERFORM select-users THRU throw-out-users.
PERFORM count-data 10 TIMES.
PERFORM set-indicators-2 THRU set-indicators-3
  VARYING support-product-index FROM 1 BY 1
    UNTIL support-product-index > max-support-product-index.
```

The function of this statement is to execute the statements which occur within the scope of the mentioned procedure name or procedure names and return control to the statement following the PERFORM statement when those statements have been executed. The statements may be executed unconditionally one or more times, or they may be executed only if certain conditions hold.

The keywords THROUGH and THRU are equivalent. The procedure names must be the names of a paragraph or section appearing in the PROCEDURE DIVISION of the COBOL program.

If the first procedure name is a section name, the second procedure name must also be a section name. In this case, all statements which occur between the beginning of the first section and the end of the second section are within the scope of the PERFORM statement.

If the first procedure name is a paragraph name then the scope of the PERFORM statement extends from the beginning of that paragraph through the end of that paragraph unless the THROUGH clause is present. In this case, the scope extends to the end of the paragraph or section named in the THROUGH clause.

If the scope of one PERFORM statement includes another PERFORM statement, then the scope of the second PERFORM statement must be totally included or totally excluded from the scope of the original PERFORM statement. This implies that the scope of PERFORM statements must not overlap.

Examples of Valid PERFORM Constructs:

```
x  PERFORM  a  THRU  m
a
d  PERFORM  f  THRU  j
f
j
m
```

```
x  PERFORM  a  THRU  m
a
d  PERFORM  f  THRU  j
h
m
f
j
```

Example of Invalid PERFORM Construct:

```
x  PERFORM  a  THRU  m
a
d  PERFORM  f  THRU  j
f
m
j
```

The data item in the TIMES clause must be a positive integer value, either a numeric literal or an integer numeric data item. The value of this data item represents the number of times the indicated statements will be executed.

In the UNTIL form of the PERFORM statement, the indicated statements will be executed as long as the specified condition is not true. This implies that if the condition is initially true, none of the statements within the scope of the PERFORM statement will be executed.

The VARYING form of the PERFORM statement provides a controlled looping construct with one, two or three control variables. Each of these variables must be numeric integer data items. Also the FROM and BY clauses must specify either numeric integer literals or numeric integer data items. The termination conditions may be any valid conditional expression, but normally they will depend on one or more of the control variables.

READ

The READ statement is used to obtain the value of a record in a data file.

Format:

```
READ file-name NEXT RECORD WITH LOCK INTO data-name

   ; AT END imperative statement


READ file-name RECORD WITH LOCK INTO data-name

   ; KEY IS data-name

   ; INVALID KEY imperative statement
```

Examples:

```
READ serial-file NEXT RECORD; AT END MOVE 'n' TO more-data-flag.
READ inx-file; KEY IS primary-key; INVALID KEY GO TO key-error.
```

The READ statement may be executed for any file organization. The file specified in the READ statement must have been previously opened (see the OPEN statement).

In SEQUENTIAL access mode, the READ statement accesses the next logical record in the file. If no records remain when the READ statement is executed, the end of file condition exists. When the end of file condition exists, the AT END statement will be executed if present. For sequential operations, the AT END phrase must be present if no USE procedure has been defined for the file.

INDEXED or RELATIVE files which are open in RANDOM or DYNAMIC mode may access the file either sequentially or randomly using a key value. For RELATIVE files, the READ statement will access the record given by the value of the RELATIVE KEY data item defined for the file. For INDEXED files, the READ statement will access the record given by the primary record key (RECORD KEY) data item, or if the KEY IS phrase is present, the record accessed will be given by the specified key. A READ ... NEXT statement accesses the next sequential record in a file which is open in DYNAMIC mode. Refer to the START statement for a discussion on how the key values apply for INDEXED files.

Records in RELATIVE and INDEXED files may be locked, i.e. made temporarily inaccessable to other programs. If the WITH LOCK phrase is present in the READ statement, then the record accessed will be locked. Locked records may only be accessed by the program which has locked them. If another program has the same file open, that program will be unable to access the locked data records. Records are unlocked by rewriting the record, closing the file or executing an UNLOCK statement for the file.

After the execution of a READ statement, the status of the operation is returned in the file status data item associated with the file. Refer to chapter 3 for complete details on file status codes and how they are interpreted. If any errors are encountered during the READ statement, then the INVALID KEY statement will be executed. If the READ statement has no INVALID KEY statement, a USE procedure must be defined for the file, in which case, that USE procedure will be executed when there is an error.

RELEASE

The RELEASE statement transfers data records to the SORT package.

Format:

```
RELEASE  record-name  FROM  data-name
```

Examples:

```
RELEASE sort-record.
RELEASE sort-record FROM names-record.
```

The RELEASE statement sends one data record to the SORT package. This statement must be executed by the INPUT PROCEDURE of a SORT statement.

When a SORT statement is executed which contains an INPUT PROCEDURE phrase, an implicit file is created which contains the records to be sorted. Records are placed in this file by using the RELEASE statement.

The record name specified in the RELEASE statement must be the name of a data record associated with the file currently being sorted. If the FROM phrase is present, then the FROM data item is first moved to the sort file record area and then the RELEASE statement executed.

RETURN

The RETURN statement is used to read records produced by a SORT or MERGE statement.

Format:

```
RETURN file-name INTO data-name ; AT END imperative statement
```

Examples:

```
RETURN sort-file INTO name-record; AT END MOVE 'n' TO more-data.
```

The RETURN statement accesses the next result record from the output of a SORT or MERGE statement. The RETURN statement must be executed within the scope of an OUTPUT PROCEDURE associated with the SORT or MERGE statement.

When a SORT or MERGE statement is executed with an OUTPUT PROCEDURE phrase, an implicit file is created which contains the result from the SORT or MERGE process. The SORT/MERGE package writes the data records to this file and the RETURN statement is the only way to access the records from this file.

The file name specified must be the name of the sort file currently being sorted or merged. Upon execution of a RETURN statement, the SORT/MERGE package will read the next sequential record from the output file and move it to the data record for the given sort file. If the INTO phrase is present, the data will also be moved to the specified

data item. If no more data records are available, the end of file condition occurs and the AT END statement will be executed.

REWRITE

The REWRITE statement replaces a record in a file.

Format:

```
REWRITE  record-name  FROM  data-name
    ;  INVALID  KEY  imperative statement
```

Examples:

```
REWRITE library-record; INVALID KEY GO TO rewrite-error.
REWRITE library-record FROM new-library-record; INVALID KEY STOP RUN.
```

The REWRITE statement logically replaces the data in a specific record in a file with new data. The record name must refer to a data record associated with an open COBOL file. The REWRITE statement may be used with any file organization.

For SEQUENTIAL files, the REWRITE statement replaces the last record read in the file. This implies that a successful READ statement must immediately precede the REWRITE statement for the file. The data record to be written must be exactly the same number of characters as the record read, a restriction which does not apply for the other file organizations.

For RELATIVE and INDEXED files, the record which is replaced depends on the file access mode. If the access mode is SEQUENTIAL then the operation is just as it is for SEQUENTIAL files, i.e. the last record read is the one replaced. For files in RANDOM and DYNAMIC access modes, the record replaced is given by the access keys, either the RELATIVE KEY data item for RELATIVE files, or the RECORD KEY and ALTERNATE KEY data items for INDEXED files.

After the operation, the status of the operation is reflected in the file status data item associated with the file, if defined. If any error is encountered during the REWRITE operation, the INVALID KEY condition exists and the INVALID KEY statement will be executed. If no INVALID KEY statement is present, then a USE section must be provided for the file. The INVALID KEY condition will exist if:

The access mode is SEQUENTIAL and the primary record key data item does not match the key value of the last record read (INDEXED files) or the relative key data item does not match the record number of the last record read (RELATIVE files).

The value in the primary record key data item does not match the key of any record in the file.

The value of an alternate key is the same as that for another record in the file and the WITH DUPLICATES clause was not specified for that key.

SET

The SET statement is used to set or change the value of a table index data item.

Format:

```
SET  data-name  TO        data-name
                UP    BY  literal
                DOWN  BY
```

Examples:

```
SET month-index TO n-month of cobol-date.
SET month-index UP BY 1.
```

The data name must be an index name, defined either as a numeric data item with USAGE INDEX or by an INDEXED BY clause in a data definition. The value must be specified by either a numeric data item or a numeric literal. In either case, the value must be a positive integer.

If the TO form is used, the index data item is set to exactly the specified value. If the UP/DOWN BY form is used, the value of the index data value is incremented/decremented by the specified value.

In this implementation, this statement is equivalent to an appropriate MOVE/ADD/SUBTRACT statement but is included for compatibility.

SORT

The SORT statement is used to sort one or more input files based on a sorting sequence defined by one or more keys, producing a sorted output file.

Format:

```
                        ASCENDING
SORT   file-name  ON    DESCENDING  KEY  data-name  ,  data-name  ...

                        ASCENDING
                  ON    DESCENDING  KEY  data-name  ,  data-name  ...

    INPUT  PROCEDURE  IS  section-name  THROUGH  section-name

                                        THRU

    USING  file-name  ,  file-name  ...

    OUTPUT  PROCEDURE  IS  section-name  THROUGH  section-name
                                         THRU

    GIVING  file-name
```

Examples:

```
SORT sort-file ON ASCENDING KEY sort-key of sort-record
               ON ASCENDING KEY qualified-title of sort-record
     USING names-file
     OUTPUT PROCEDURE IS produce-report-lines.
```

The SORT statement controls the sorting of one or more files producing a sorted output file. The sorting sequence is specified by one or more keys, given as either ASCENDING or DESCENDING. Fields which are specified by an ASCENDING key will be sorted from the smallest key value to the largest. Fields which are specified by a DESCENDING key will be sorted from the largest key value to the smallest. If more than one key specification is given, then the first is the the most significant, the second is the next most significant, etc.

The files to be sorted may be specified literally using the USING phrase. In this case, all the files mentioned must be COBOL file names and must be closed when the SORT statement is executed. The input to the SORT package is the concatenation of all the files given.

Alternatively, the input to the SORT package may be given by a program section, specified by the INPUT PROCEDURE phrase. This phrase must specify a section name to be performed. Before the code is performed, an implicit file is created and opened. Data records may be placed in this file by using the RELEASE statement. When the INPUT PROCEDURE terminates, this file is then given to the SORT package to be sorted. This mechanism is useful for sorting only a selection of records and having a program segment which performs the selection process.

The result of the SORT process may be given as a specific file using the GIVING phrase. This file should be closed at the time of the SORT statement. It will be opened implicitly (in OUTPUT mode, thus replacing any previous contents) and the contents written by the SORT package. The file will be closed at the termination of the sort process.

The output from the sort process may also be obtained using an OUTPUT PROCEDURE. When the input file has been sorted, the program section given in the OUTPUT PROCEDURE phrase will be executed. Just prior to this, an implicit file which contains the result of the sort will be opened for input. Data records may be obtained from this file via the RETURN statement. When no more sorted records are available, the AT END condition will occur. Normally, this is used to terminate the OUTPUT PROCEDURE program segment.

The sort file must have been defined in the FILE SECTION using an SD statement. The keys given in the SORT statement must be alphanumeric data items in one of the data records associated with the sort file. A maximum of 20 keys may be specified, but at least one key must be specified.

START

The START statement is used to specify a starting position for sequential operations in a RELATIVE or INDEXED file.

Format:

```
                              EQUAL TO
                              =
                              GREATER THAN
START  file-name  KEY  IS     >            data-name
                              NOT  LESS
                              NOT  <

    ;  INVALID  KEY  imperative statement
```

Examples:

```
START library-file KEY IS EQUAL TO new-library-key;
  INVALID KEY GO TO start-error.
```

The file name must reference either a RELATIVE or INDEXED file, opened in either SEQUENTIAL or DYNAMIC access mode. If the file is an INDEXED file, the data name specified as the KEY IS value must be either one of the keys defined for the file, or an alphanumeric data item subordinate to one of the keys defined for the file whose leftmost character position is the same as that of the key defined for the file (i.e. the key specified must be a leftmost substring of one of the keys for the file). If the file is a RELATIVE file, the key specified must be exactly the RELATIVE KEY data item defined for the file. If no KEY IS phrase is present, the START statement has the same effect as a START statement with a key value equal to the primary key value.

If the IS EQUAL TO condition is given, then an exact key match must be found.

If any other comparison condition is given, a record which has a key value either greater (GREATER THAN) or greater than or equal to (NOT LESS THAN) to the given key value will be searched for in the file.

If the comparision cannot be satisfied, i.e. the exact key cannot be located for an EQUAL TO comparision, or there is no key value greater (GREATER THAN) or greater than or equal (NOT LESS THAN) in the file, then the START statement is unsuccessful and the INVALID KEY condition exists. If the INVALID KEY phrase is present, the INVALID KEY statement will be executed. If no INVALID KEY phrase is given, a USE procedure must be defined for the file.

After the successful execution of a START statement, subsequent sequential READ statements will access the records in the file, starting with the record which satisfied the START criterion.

When a data record is accessed by a key value using the KEY IS phrase, the specified key is known as the key of reference from the time of that operation until a new key of reference is specified. This key of reference may be specified by either a START statement or a READ statement. For RELATIVE files, the key of reference is always the RELATIVE KEY for the file. For INDEXED files, the key of reference is normally the primary record key. All records in the file must have a unique primary record key value. Sequential operations using the primary record key as the key of reference starting from the beginning of the file will access all records in the file. Sequential operations using an alternate key as the key of reference will only access those records which have the same value for the specific alternate key as the first record accessed via that key. When there are no more records which have this key value, the end of file condition will occur, although there may indeed be many records in the file which have not been accessed.

STOP

The STOP statement causes a permanent or temporary suspension of a COBOL program.

Format:

```
STOP   RUN
       literal
```

Examples:

```
STOP RUN.
STOP "Insert disk - hit C/R to continue:".
```

The first form of the STOP statement causes the program to terminate. There should always be at least one "STOP RUN" statement in every COBOL program. All files which are open at the time of the STOP statement will be closed before the program terminates, although it is good practice for the COBOL program to close files before executing the STOP statement.

The second form of the STOP statement provides a "pause" mechanism to the COBOL program. The literal value will be displayed on the terminal on a new print line with the text "Stop - " preceding the literal value. At this point the COBOL program will wait for the operator to enter characters on the terminal until a carriage return is entered. Once the carriage return has been entered, the program will continue execution with the statement which follows the STOP statement. If the operator wishes to terminate the program at this point, he may enter control-backslash. This form of the STOP statement does not alter any open files unless the program is terminated abnormally.

SUBTRACT / SUBTRACT CORRESPONDING

The SUBTRACT statement computes the difference of two or more numeric values and places the result in a given data item.

Format:

```
SUBTRACT  data-name  ,  data-name  ...  FROM  data-name  ROUNDED
          literal       literal

   ;  ON  SIZE  ERROR  imperative statement

SUBTRACT  data-name  ,  data-name  ...  FROM  data-name
          literal       literal              literal

   GIVING  data-name  ROUNDED

   ;  ON  SIZE  ERROR  imperative statement


SUBTRACT  CORRESPONDING  data-name  FROM  data-name  ROUNDED
          CORR

   ;  ON  SIZE  ERROR  imperative statement
```

Examples:

```
SUBTRACT overtime-hours FROM total-hours GIVING normal-hours.
SUBTRACT 2 FROM count.
SUBTRACT CORR current-data FROM total-data.
```

The SUBTRACT statement subtracts one or more numeric values from a given numeric value and places the difference in a specified data item. If the GIVING phrase is present then the result is stored in the GIVING data item, otherwise the result is stored in the FROM data item.

If more than one data item appears before the FROM clause, then the result is the difference between the FROM data item and the sum of all the data items which appear before the FROM clause. In effect, each of the specified data items is subtracted from the value given in the FROM clause.

If the ROUNDED phrase is present, the value will be rounded before being stored in the destination data item.

If any intermediate result exceeds 18 significant digits or the result cannot be successfully moved into the destination data item the SIZE ERROR condition will exist and the result is undefined. In this case, if the ON SIZE ERROR phrase is present, the statement will be executed.

The data items specified as source values must be either numeric literals or numeric elementary data items. Data items specified as destination fields must be either numeric or numeric edited elementary data items.

The SUBTRACT CORRESPONDING form must reference two group data items. The SUBTRACT CORRESPONDING statement subtracts all corresponding data fields in the first group item from the data fields in the second group. If any of the operations result in a SIZE ERROR condition, the entire operation is undefined and the ON SIZE ERROR statement will be executed. Refer to MOVE CORRESPONDING for a discussion on how corresponding operations are defined.

SUPPRESS

The SUPPRESS statement causes the Report Writer Control System (RWCS) to inhibit the processing of a report group.

Format:

```
SUPPRESS  PRINTING
```

Examples:

```
SUPPRESS  PRINTING.
```

This statement may only appear in a USE BEFORE REPORTING paragraph. When the report group specified in the USE statement is to be presented, then the USE paragraph will be executed. The function of this statement is to suppress any output which would normally result from the presentation of the given report group.

TERMINATE

This statement instructs the Report Writer Control System (RWCS) to complete the processing of a given report.

Format:

```
TERMINATE  report-name
```

Examples:

```
TERMINATE audit-report.
```

The report named must have been previously initiated using the INITIATE statement. The execution of this statement causes the CONTROL FOOTING report groups to be presented in order from highest to lowest (refer to the section on the GENERATE verb for a discussion of the ordering of control variables). The REPORT FOOTING report group will then be presented.

If no GENERATE statements have been executed for the report, then the TERMINATE statement will not cause any report groups to be presented. That is, if no GENERATE statements are executed for a given report, the report will be totally empty.

The TERMINATE statement does not close the file associated with the report and thus a separate CLOSE statement should be included in the COBOL program to close the report file.

UNLOCK

The UNLOCK statement unlocks all records in an INDEXED or RELATIVE file currently locked by the COBOL program.

Format:

```
UNLOCK file-name  RECORD
                  RECORDS
```

Examples:

```
UNLOCK library-file.
```

The UNLOCK statement unlocks all records currently locked by the running COBOL program in an INDEXED or RELATIVE file. The file name must reference an open INDEXED or RELATIVE file. The status of the operation is reflected in the file status data item for the file after the operation.

This statement may be executed whether or not any records are locked in the file.

USE

The USE statement specifies a section of the COBOL program which is to be executed when certain conditions exist during the running of the program.

Format:

```
USE AFTER STANDARD EXCEPTION PROCEDURE ON file-name ...
                   ERROR
```

Format 2:

```
USE BEFORE REPORTING report-group-name
```

Examples:

```
USE AFTER STANDARD ERROR PROCEDURE ON library-file.
USE BEFORE REPORTING audit-count-footer.
```

This statement is only allowed in the DECLARATIVES section of the PROCEDURE DIVISION and only after a SECTION heading. The section of the program being defined is then associated with either a set of files or with a specific report group.

The first format of the USE statement associates a section of the COBOL program with a file or files used by the program. If there are any error conditions encountered when processing one of the files mentioned and the statement which caused the error to occur does not include an error handling clause (AT END, INVALID KEY), then the section corresponding to the USE statement will be performed. This allows the programmer to specify in a single location in the COBOL program the actions to take after an error condition exists for a given file.

No USE statement is required for any file, but if none is specified then error handling clauses must be included in most input/output statements present in the program.

The second format of the USE statement associates the program section with a specific report group. The report group named may be any type of report group, defined in the REPORT SECTION of the program. When the report group specified is about to be presented by the RWCS, this program section will be executed before any output is produced. The most common use of such a section of code is to suppress the output from a given report group under certain conditions. This may be done by executing a SUPPRESS PRINTING statement within the USE program section. In fact, the SUPPRESS PRINTING statement may only occur in this context in the COBOL program.

WRITE

The WRITE statement places a new record in a data file.

Format:

```
   WRITE  record-name  FROM  data-name

                         data-name  LINE
      BEFORE  ADVANCING  integer    LINES
      AFTER              PAGE

      ;  AT  END-OF-PAGE  imperative statement
             EOP

   WRITE  record-name  FROM  data-name

      ;  INVALID  KEY  imperative statement
```

Examples:

```
   WRITE library-record FROM new-library-record;
     INVALID KEY GO TO write-error.
   WRITE detail-line AFTER ADVANCING 2 LINES;
     AT EOP PERFORM top-of-page.
```

The two formats of the WRITE statement correspond to SEQUENTIAL file operations and INDEXED or RELATIVE file operations, respectively.

For SEQUENTIAL files, records are appended, one at a time, to a file using the WRITE statement. Special consideration has been given to files which may eventually be printed on hard copy devices, possibly with some page structure. The WRITE statement may specify a number of blank (empty) lines which are to be inserted before the data record is written (AFTER ADVANCING), or a number of blank lines to be appended after the data record is written (BEFORE ADVANCING). Alternatively, the WRITE statement may specify that the data record appear at the top of the following page (AFTER ADVANCING PAGE) or that the form be positioned to the top of a new page after the data record is written (BEFORE ADVANCING PAGE). If the FD statement which describes the file contains the LINAGE clause (see chapter 7), then the appropriate number of blank lines will be inserted in the output file to achieve this alignment with the top of form. If the LINAGE clause is not present, the top of form will be achieved by the insertion of a line which contains a form feed character ($0C).

If the file has the LINAGE clause specified then after the last line on the page has been written on or spaced to, the end of page condition will exist. If the END-OF-PAGE phrase is present in the statement, then it will be executed after the completion of the write statement. When the END-OF-PAGE statement is executed, no more lines can be written on the current page without advancing to the top of a new page. This statement is most commonly used to print page headers, etc.

For RELATIVE and INDEXED files, data records need not have consecutive key values, i.e. the file may contain holes by simply not writing records where the holes are. Which record is written in the file depends on the file access mode.

If the access mode is SEQUENTIAL then records are appended to the end of the file. If the primary record key (INDEXED files) or the relative key (RELATIVE files) already exists or does not have a value higher than all other keys in the file, the INVALID KEY condition will exist.

If the access mode is RANDOM or DYNAMIC, the record written depends on the relative key value (RELATIVE files) or the primary record key and alternate record key values (INDEXED files). In this case, the current record key value need have no relationship with the previous record as in SEQUENTIAL access mode. If the relative key value (RELATIVE files) or the primary key value (INDEXED files) already exists in the file, the INVALID KEY condition exists.

A. GLOSSARY

This appendix defines some of the more commonly used terms and phrases found in this manual.

Access Mode. The manner in which records are to be operated upon within a file.

Actual Decimal Point. The physical representation using either of the decimal point characters period (.) or comma (,), of the decimal point position in a data item.

Alphabetic Character. A character that belongs to the following set of letters: A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z, and the space.

Alphanumeric Character. Any character in the ASCII character set.

Alternate Key Record. A key, other than the prime record key, whose contents identify a record within an indexed file.

Arithmetic Expression. An arithmetic expression can be an identifier or a numeric elementary item, a numeric literal, such identifiers and literals separated by arithmetic operators, two arithmetic expressions separated by an arithmetic operator, or an arithmetic expression enclosed in parentheses.

Arithmetic Operator. A single character, or a fixed two-character combination, that belongs to the following set:

Character	Meaning
+	addition
-	subtraction
*	multiplication
/	division

Ascending Key. A key upon the values of which data is ordered starting with the lowest value of key up to the highest value of key in accordance with the rules for comparing data items.

Assumed Decimal Point. A decimal point position which does not involve the existence of an actual character in a data item. The assumed decimal point has logical meaning but no physical representation.

At End Condition. A condition caused:

(1) During the execution of a READ statement for a sequentially accessed file.

(2) During the execution of a RETURN statement, when no next logical record exists for the associated sort or merge file.

Called Program. A program which is the object of a CALL statement combined at object time with the calling program to produce a run unit.

Calling Program. A program which executes a CALL to another program.

Character Position. A character position is the amount of physical storage required to store a single standard data format character described as usage is DISPLAY.

Character-String. A sequence of contiguous characters which form a COBOL word, a literal, a PICTURE character-string, or a comment-entry.

Class Condition. The proposition, for which a truth value can be determined, that the content of an item is wholly alphabetic or is wholly numeric.

Clause. A clause is an ordered set of consecutive COBOL character-strings whose purpose is to specify an attribute of an entry.

COBOL Character Set. The complete COBOL character set consists of the 76 characters listed below:

Character	Meaning
0,1,...,9	digit
A,B,...,Z	letter
a,b,...,z	
	space (blank)
+	plus sign
-	minus sign (hyphen)
*	asterisk
/	stroke (virgule, slash)
=	equal sign
$	currency sign
,	comma (decimal point)
;	semicolon
.	period (decimal point)
"	quotation mark
(	left parenthesis
)	right parenthesis
>	greater than symbol
<	less than symbol

Collating Sequence. The sequence in which the characters that are acceptable in a computer are ordered for purposes of sorting, merging, and comparing.

Column. A character position within a line. The columns are numbered from 1, by 1, starting at the leftmost character position of the line and extending to the rightmost position of the line.

Combined Condition. A condition that is the result of connecting two or more conditions with the 'AND' or the 'OR' logical operator.

Comment-Entry. An entry in the Identification Division that may be any combination of characters from the printable ASCII character set.

Comment Line. A source program line represented by an asterisk in column 1 of the line and any characters from the computer's character set in the remainder of that line. The comment line serves only for documentation in a program. A special form of comment line represented by a stroke (/) in column 1 of the line and any characters from the computer's character set in the remainder of that line causes page ejection prior to printing the comment.

Compile Time. The time at which a COBOL source program is translated, by a COBOL compiler, to a COBOL object program.

Compiler Directing Statement. A statement, beginning with a compiler directing verb, that causes the compiler to take a specific action during compilation.

Complex Condition. A condition in which one or more logical operators act upon one or more conditions. (see Negated Simple Condition, Combined Condition, Negated Combined Condition.)

Computer-Name. A system-name that identifies the computer upon which the program is to be compiled or run.

Condition. A status of a program at execution time for which a truth value can be determined. Where the term 'condition' appears in this manual in or in reference to 'condition' of a general format, it is a conditional expression consisting of either a simple condition optionally parenthesized, or a combined condition consisting of the syntactically correct combination of simple conditions, logical operators, and parentheses, for which a truth value can be determined.

Condition-Name. A user-defined word assigned to a specific value, set of values, or range of values, within the complete set of values that a conditional variable may possess; or the user-defined word assigned to a status of a runtime switch.

Condition-Name-Condition. The proposition, for which a truth value can be determined, that the value of a conditional variable is a member of the set of values attributed to a condition-name associated with the condition variable.

Conditional Expression. A simple condition or a complex condition specified in an IF, or PERFORM statement. (See Simple Condition and Complex Condition.)

Conditional Statement. A conditional statement specifies that the truth value of a condition is to be determined and that the subsequent action of the object program is dependent on this truth value.

Conditional Variable. A data item one or more values of which has a condition-name assigned to it.

Configuration Section. A section of the Environment Division that describes overall specifications of source and object computers.

Contiguous Items. Items that are described by consecutive entries in the Data Division, and that bear a definite hierarchic relationship to each other.

Control Break. A change in the value of a data item that is referenced in the CONTROL clause. More generally, a change in the value of a data item that is used to control the hierarchical structure of a report.

Control Break Level. The relative position within a control hierarchy at which the most major break occurred.

Control Data Item. A data item, a change in whose contents may produce a control break.

Control Data-Name. A data-name that appears in a CONTROL clause and refers to a control data item.

Control Footing. A report group that is presented at the end of the control group of which it is a member.

Control Group. A set of body groups that is presented for a given value of a control data item or of FINAL. Each control group may begin with a CONTROL HEADING, end with a CONTROL FOOTING, and contain DETAIL report groups.

Control Heading. A report group that is presented at the beginning of the control group of which it is a member.

Control Hierarchy. A designated sequence of report subdivisions defined by the positional order of FINAL and the data-names within a CONTROL clause.

Counter. A data item used for storing numbers or number representations in a manner that permits these numbers to be increased or decreased by the value of another number, or to be changed or reset to zero or to an arbitrary positive or negative value.

Currency Sign. The character '$' of the COBOL character set.

Currency Symbol. The character defined by the CURRENCY sign clause in the SPECIAL-NAMES paragraph. If no CURRENCY SIGN clause is present in a COBOL source program, the currency symbol is identical to the currency sign.

Current Record. The record which is available in the record area associated with the file.

Current Record Pointer. A conceptual entity that is used in the selection of the next record.

Data Clause. A clause that appears in a data description entry in the Data Division and provides information describing a particular attribute of a data item.

Data Description Entry. An entry in the Data Division that is composed of a level-number followed by a data-name, if required, and then followed by a set of data clauses, as required.

Data Item. A character or set of contiguous characters (excluding in either case literals) defined as a unit of data by the COBOL program.

Data-Name. A user defined word that names a data item described in a data description entry in the Data Division. When used in the general formats, 'data-name' represents a word which can neither be subscripted, indexed, nor qualified unless specifically permitted by the rules for that format.

Declaratives. A set of one or more special purpose sections, written at the beginning of the Procedure Division, the first of which is preceded by the key word DECLARATIVES and the last of which is followed by the key words END DECLARATIVES. A declarative is composed of a section header, followed by a USE compiler directing sentence, followed by a set of zero, one or more associated paragraphs.

Declarative-Sentence. A compiler-directing sentence consisting of a single USE statement terminated by the separator period.

Delimiter. A character or sequence of contiguous characters that identify the end of a string of characters and separates that string of characters from the following string of characters. A delimiter is not part of the string of characters that it delimits.

Descending Key. A key upon the values of which data is ordered starting with the highest value of key down to the lowest value of key, in accordance with the rules for comparing data items.

Digit-Position. A digit position is the amount of physical storage required to store a single digit. This amount may vary depending on the usage of the data item describing the digit position.

Division. A set of zero, one or more sections of paragraphs, called the division body, that are formed and combined in accordance with a specific set of rules. There are four (4) divisions in a COBOL program: Identification, Environment, Data, and Procedure.

Division Header. A combination of words followed by a period and a space that indicates that beginning of a division. The division headers are:

```
IDENTIFICATION DIVISION
ENVIRONMENT DIVISION
DATA DIVISION
PROCEDURE DIVISION [USING data-name  [ ,  data-name ]... ]
```

Dynamic Access. An access mode in which specific logical records can be obtained from or placed into a file in a non-sequential manner (See Random Access) and obtained from a file in a sequential manner (See Sequential Access), during the scope of the same OPEN statement.

Editing Character. A single character or a fixed two-character combination belonging to the following set:

Character	Meaning
B	space
0	zero
+	plus
-	minus
CR	credit
DB	debit
Z	zero suppress
*	check protect
$	currency sign
,	comma (decimal point)
.	period (decimal point)
/	stroke (virgule, slash)

Elementary Item. A data item that is described as not being further logically subdivided.

End of Procedure Division. The physical position in a COBOL source program after which no further procedures appear.

Entry. Any descriptive set of consecutive clauses terminated by a period and written in the Identification Division, Environment Division, or Data Division of a COBOL source program.

Environment Clause. A clause that appears as part of an Environment Division entry.

Execution Time. (See Object Time.)

Extend Mode. The state of a file after execution of an OPEN statement, with the EXTEND phrase specified, for that file and before the execution of a CLOSE statement for that file.

Figurative Constant. A compiler generated value referenced through the use of certain reserved words.

File. A collection of records.

File Clause. A clause that appears as part of any of the following Data Division entries:

File-description (FD)
Sort-merge file description (SD)

FILE-CONTROL. The name of an Environment Division paragraph in which the data files for a given source program are declared.

File Description Entry. An entry in the File Section of the Data Division that is composed of the level indicator FD, followed by a file-name, and then followed by a set of file clauses as required.

File-Name. A user-defined word that names a file described in a file description entry or a sort-merge file description entry within the File Section of the Data Division.

File Organization. The permanent logical file structure established at the time that a file is created.

File Section. The section of the Data Division that contains file description entries and sort-merge file description entries together with their associated record descriptions.

Format. A specific arrangement of a set of data.

Group Item. A named contiguous set of elementary or group items.

High Order End. The leftmost character of a string of characters.

I-O-CONTROL. The name of an Environment Division paragraph in which object program requirements for specific input-output techniques, and sharing of same areas by several data files are specified.

I-O Mode. The state of a file after execution of an OPEN statement, with the I-O phrase specified, for that file and before the execution of a CLOSE statement for that file.

Identifier. A data-name, followed as required, by the syntactically correct combination of qualifiers, subscripts, and indices necessary to make unique reference to a data item.

Imperative Statement. A statement that begins with an imperative verb and specifies an unconditional action to be taken. An imperative statement may consist of a sequence of imperative statements.

Index. A computer storage position or register, the contents of which represent the identification of a particular element in a table.

Index Data Item. A data item in which the value associated with an index-name is stored.

Index-Name. A user-defined word that names an index associated with a specific table.

Indexed Data-Name. An identifier that is composed of a data-name, followed by one or more index-names enclosed in parentheses.

Indexed File. A file with indexed organization.

Indexed Organization. The permanent logical file structure in which each record is identified by the value of one or more keys within that record.

Input File. A file that is opened in the input mode.

Input Mode. The state of a file after execution of an OPEN statement, with the INPUT phrase specified, for that file and before the execution of a CLOSE statement for that file.

Input-Output File. A file that is opened in the I-O mode.

Input-Output Section. The section of the Environment Division that names the files and the external media required by an object program which provides information required for transmission and handling of data during execution of the object program.

Input Procedure. A set of statements that is executed each time a record is released to the sort file.

Integer. A numeric literal or numeric data item that does not include any characters positions to the right of the assumed decimal point. Where the term 'integer' appears in the general formats, integer must not be a numeric data item, and must not be signed, nor zero unless specifically allowed by the rules of that format.

Invalid Key Condition. A condition, at object time, caused when a specific value of the key associated with an indexed or relative file is determined to be invalid.

Key. A data item which identifies the location of a record, or a set of data items which serve to identify the ordering of data.

Key of Reference. The key, either prime or alternate, currently being used to access records within an indexed file.

Key Word. A reserved word whose presence is required when the format in which the word appears is used in a source program.

Level Indicator. Two alphabetic characters that identify a specific type of file or a position in hierarchy.

Level-Number. A user-defined word which indicates the position of a data item in the hierarchical structure of a logical record or which indicates special properties of a data description entry. A

level-number is expressed as a one- or two-digit number. Level-numbers in the range 1 through 49 indicate the position of a data item in the hierarchical structure of a logical record. Level-numbers in the range 1 through 9 may be written either as a single digit or as a zero followed by a significant digit. Level-numbers 77, and 88 identify special properties of a data description entry.

Line Number. An integer that denotes the vertical position of a report line on a page, or a data field on a CRT screen.

Linkage Section. The section in the Data Division of the called program that describes data items available from the calling program. These data items may be referred to by both the calling and called.

Literal. A character-string whose value is implied by the ordered set of characters comprising the string.

Logical Operator. One of the reserved words AND, OR, or NOT. In the formation of a condition, both or either of AND and OR can be used as logical connectives. NOT is used for logical negation.

Logical Record. The most inclusive data item. The level-number for a record is 01. (See Report Writer Logical Record.)

Low Order End. The rightmost character of a string of characters.

Merge File. A collection of records to be merged by a MERGE statement. The merge file is created and can be used only by the merge function.

Negated Combined Condition. The 'NOT' logical operator immediately followed by a parenthesized combined condition.

Negated Simple Condition. The 'NOT' logical operator immediately followed by a simple condition.

Next Executable Sentence. The next sentence to which control will be transferred after execution of the current statement is complete.

Next Executable Statement. The next statement to which control will be transferred after execution of the current statement is complete.

Next Record. The record which logically follows the current record of a file.

Noncontiguous Items. Elementary data items, in the Working-Storage and Linkage sections, which bear no hierarchical relationship to other data items.

Nonnumeric Item. A data item whose description permits its contents to be composed of any combination of characters taken from the computer's character set. Certain categories of nonnumeric items may be formed from more restricted character sets.

Nonnumeric Literal. A character-string bounded by quotation marks. The string of characters may include any character in the computer's character set. To represent a single quotation mark character within a nonnumeric literal, two contiguous quotation marks must be used.

Numeric Character. A character that belongs to the following set of digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

Numeric Item. A data item whose description restricts its contents to a value represented by characters chosen from the digits '0' through '9'; if signed, the item may also contain a '+', '-', or other representation of an operational sign.

Numeric Literal. A literal composed of one or more numeric characters that also may contain either a decimal point, or an algebraic sign, or both. The decimal point must not be the rightmost character. The algebraic sign, if present, must be the leftmost character.

OBJECT-COMPUTER. The name of an Environment Division paragraph in which the computer environment, within which the object program is executed, is described.

Object of Entry. A set of operands and reserved words, within a Data Division entry, that immediately follows the subject of the entry.

Object Program. A set or group of executable machine language instructions and other material designed to interact with data to provide problem solutions. In this context, an object program is the machine language result of the COBOL compiler on a source program. Where there is no danger of ambiguity, the word 'program' alone amybe used in place of the phrase 'object program.'

Object Time. The time at which an object program is executed.

Open Mode. The state of a file after execution of an OPEN statement for that file and before the execution of a CLOSE statement for that file. The particular open mode is specified in the OPEN statement as either INPUT, OUTPUT, I-O, or EXTEND.

Operand. Whereas the general definition of operand is 'that component which is operated upon,' for the purposes of this publication, any lowercase word (or words) that appears in a statement or entry format may be considered to be an operand and, as such, is an implied reference to the data indicated by the operand.

Operational Sign. An algebraic sign, associated with a numeric data item or a numeric literal, to indicate whether its value is positiveor negative.

Optional Word. A reserved word that is included in a specific format only to improve the readability of the language and whose presence is optional to the user when the format in which the word appears is used in a source program.

Output File. A file that is opened in either the output mode or extend mode.

Output Mode. The state of a file after execution of an OPEN statement, with the OUTPUT or EXTEND phrase specified, for that file and before the execution of a CLOSE statement.

Output Procedure. A set of statements to which control is given during execution of a SORT statement after the sort function is completed, or during execution of a MERGE statement after the merge function has selected the next record in merged order.

Page. A vertical division of a report representing a physical separation of report data, the separation being based on internal reporting requirements and/or external characteristics of the reporting medium.

Page Body. That part of the logical page in which lines can be written and/or spaced.

Page Footing. A report group that is presented at the end of a report page as determined by the Report Writer Control System.

Page Heading. A report group that is presented at the beginning of a report page and determined by the Report Writer Control System.

Paragraph. In the Procedure Division, a paragraph-name followed by a period and a space and by zero, one or more sentences. In the Identification and Environment Divisions, a paragraph header followed by a zero, one, or more entries.

Paragraph Header. A reserved word, followed by a period and a space, that indicates the beginning of a paragraph in the Identification and Environment Divisions. The permissible paragraph headers are:

In the Identification Division:

```
PROGRAM-ID.
AUTHOR.
INSTALLATION.
DATE-WRITTEN.
DATE-COMPILED.
SECURITY.
```

In the Environment Division:

```
SOURCE-COMPUTER.
OBJECT-COMPUTER.
SPECIAL-NAMES.
FILE-CONTROL.
I-O-CONTROL.
```

Paragraph-Name. A user-defined word that identifies and begins a paragraph in the Procedure Division.

Phrase. A phrase is an ordered set of one or more consecutive COBOL character-strings that form a portion of a COBOL procedural statement or of a COBOL clause.

Prime Record Key. A key whose contents uniquely identify a record within an indexed file.

Printable Group. A report group that contains at least one print line.

Printable Item. A data item, the extent and contents of which are specified by an elementary report entry. This elementary report entry contains a COLUMN NUMBER CLAUSE, a PICTURE clause, and a SOURCE, SUM, or VALUE clause.

Procedure. A paragraph or group of logically successive paragraphs, or a section or group of logically successive sections, within the Procedure Division.

Procedure-Name. A user-defined word that identifies a COBOL source program.

Punctuation Character. A character that belongs to the following set:

Character	Meaning
,	comma
;	semicolon
.	period
"	quotation mark
(	left parenthesis
)	right parenthesis
	space
=	equal sign

Qualified Data-Name. An identifier thatis composed of a data-name followed by one or more sets of either of the connectives OF and IN followed by a data-name qualifier.

Qualifier. A data-name which is used in a reference together with another data-name at a lower level in the same hierarchy.

Random Access. An access mode in which the program-specified value of a key data item identifies the logical record that is obtained from, deleted from, or placed into a relative or indexed file.

Record Area. A storage area allocated for the purpose of processing the record described in a record description entry in the File Section.

Record Description. (See Record Description Entry.)

Record Description Entry. The total set of data description entries associated with a particular record.

Record Key. A key, either the prime record key or an alternate record key, whose contents identify a record within an indexed file.

Record-Name. A user-defined word that names a record described in a record description entry in the Data Division.

Relation. (See Relational Operator.)

Relation Character. A character that belongs to the following set:

Character	Meaning
>	greater than
<	less than
=	equal to

Relation Condition. The proposition, for which a truth value can be determined, that the value of an arithmetic expression or data item has a specific relationship to the value of another arithmetic expression or data item. (See Relational Operator.)

Relational Operator. A reserved word, a relation character, a group of consecutive reserved words, or a group of consecutive reserved words and relation characters used in the construction of a relation condition. The permissible operators and their meanings are:

Relational Operator	Meaning
IS [NOT] GREATER THAN	Greater than or not greater than
IS [NOT] >	
IS [NOT] LESS THAN	Less than or not less than
IS [NOT] <	
IS [NOT} EQUAL TO	Equal to or not equal to
IS [NOT] =	

Relative File. A file with relative organization.

Relative Key. A key whose contents identify a logical record in a relative file.

Relative Organization. The permanent logical file structure in which each record is uniquely identified by an integer value greater than zero, which specifies the record's logical ordinal position in the file.

Report Clause. A clause, in the Report Section of the Data Division, that appears in a report description entry or a report group description entry.

Report Description Entry. An entry in the Report Section of the Data Division that is composed of the level indicator RD, followed by a report name, followed by a set of report clauses.

Report File. An output file whose file description entry contains a REPORT clause. The contents of a report file consist of records that are written under control of the Report Writer Control System.

Report Footing. A report group that is presented only at the end of a report.

Report Group. In the Report Section of the Data Division, an 01 level-number entry and its subordinate entries.

Report Group Description Entry. A entry in the Report Section of the Data Division that is composed of the level-number 01, the optional data-name, a TYPE clause, and an optional set of report clauses.

Report Heading. A report group that is presented only at the beginning of a report.

Report Line. A division of a page representing one row of horizontal character positions. Each character position of a report is aligned vertically beneath the corresponding character position of the report line above it. Report lines are numbered from 1, by 1, starting at the top of the page.

Report-Name. A user-defined word that names a report described in a report description entry within the Report Section of the Data Division.

Report Section. The section of the Data Division that contains one or more report description entries and their associated report group description entries.

Report Writer Control System (RWCS). An object time control system that accomplishes the construction of reports.

Report Writer Logical Record. A record that consists of the Report Writer print line and associated control information necessary for its vertical positioning.

Reserved Word. A COBOL word specified in the list of words which may be used in COBOL source programs, but which must not appear in the programs as user-defined words. A complete list of COBOL reserved words appears in Appendix G.

RWCS. (See Report Writer Control System).

Section. A set of zero, one, or more paragraphs or entries, called a section body, the first of which is preceded by a section header. Each section consists of the section header and the related section body.

Section Header. A combination of words followed by a period and a space that indicates the beginning of a section in the Environment, Data and Procedure Division.

In the Environment and Data Divisions, a section header is composed of reserved words followed by a period and a space. The permissible section headers are:

```
In the Environment Division:
  CONFIGURATION SECTION.
  INPUT-OUTPUT SECTION.

In the Data Division:
  FILE SECTION.
  WORKING-STORAGE SECTION.
  LINKAGE SECTION.
  SCREEN SECTION.
  REPORT SECTION.
```

In the Procedure Division, a section header is composed of a section-name, followed by the reserved word SECTION, followed by a space and a period.

Section-Name. A user-defined word which names a section in the Procedure Division.

Sentence. A sequence of one or more statements, the last of which is terminated by a period followed by a space.

Separator. A punctuation character used to delimit character-strings.

Sequential Access. An access mode in which logical records are obtained from or placed into a file in a consecutive predecessor-sucessor logical record sequence determined by the order of records in the file.

Sequential file. A file with sequential organization.

Sequential Organization. The permanent logical file structure in which a record is identified by a predecessor-successor relationship established when the record is placed in the file.

Sign Condition. The proposition, for which a truth value can be determined, that the algebraic value of a data item or arithmetic expression is either less than, equal to, or greater than zero.

Simple Condition. Any single condition chosen from the set:

```
relation condition
class condition
condition-name condition
switch-status condition
sign condition
(simple condition)
```

Sort File. A collection of records to be sorted by a SORT statement. The sort file is created and can be used by the sort function only.

Sort-Merge Description Entry. An entry in the File Section of the Data Division that is composed of the level indicator SD, followed by a file-name, and then followed by a set of file clauses as required.

SOURCE-COMPUTER. The name of an Environment Division paragraph in which the computer environment, within which the source program is compiled, is described.

Source Item. An identifier designated by a SOURCE clause that provides the value of a printable item.

Source Program. A set of COBOL statements, beginning with an Identification Division and ending with the end of a Procedure Division.

Special Character. A character that belongs to the following set:

Character	Meaning
+	plus sign
-	minus sign
*	asterisk
/	stroke (virgule, slash)
=	equal sign
$	currency sign
,	comma (decimal point)
;	semicolon
.	period (decimal point)
"	quotation mark
(	left parenthesis
)	right parenthesis
>	greater than symbol
<	less than symbol

Statement. A syntactically valid combination of words and symbols written in the Procedure Division beginning with a verb.

Subject of Entry. An operand or reserved word that appears immediately following the level indicator or the level-number in a Data Division entry.

Subscript. An integer whose value identifies a particular element in a table.

Subscripted Data-Name. An identifier that is composed of a data-name followed by one or more subscripts enclosed in parentheses.

Sum Counter. A signed numeric data item established by a SUM clause in the Report Section of the Data Division. The sum counter is used by the Report Writer Control System to contain the result of

designated summing operations that take place during the production of a report.

Switch-Status Condition. The proposition, for which a truth value can be determined that a user-defined switch-name has been set either 'on' or 'off' as the result of a runtime option.

Table. A set of logically consecutive items of data that are defined in the Data Division by means of the OCCURS clause.

Table Element. A data item that belongs to the set of repeated items that comprise a table.

Truth Value. The representation of the result of evaluating a condition in terms of one of two values, either true, or, false.

Unary Operator. A plus (+) or minus (-) sign, which precedes a variable or a left parenthesis in an arithmetic expression and which has the effect of multiplying the expression by +1 or -1 respectively.

User-Defined Word. A COBOL word that must be supplied by the user to satisfy the format of a clause or statement.

Variable. A data item whose value may be changed by the execution of the object program. A variable used in an arithmetic expression must be a numeric elementary item.

Verb. A word that expresses an action to be taken by a COBOL compiler or object program.

Word. A character-string which forms a user-defined word or reserved word. Words may be any length, but only the first 30 characters are significant.

Working-Storage Section. The section of the Data Division that describes working storage data items, composed of non-contiguous items or of working storage records or of both.

77-Level-Description-Entry. A data description entry that describes a non-contiguous data item with the level-number 77.

B. COMPILER ERROR MESSAGES

The following is a list of the possible error messages which can occur during the compilation of a COBOL program. In general, these messages will have the offending source line printed directly above them along with a pointer indicating the approximate position in the line where the error was detected. Because of the nature of the compilation process, if an error actually occurs at the end of a line, the message may not be printed until the next line has been partially processed. In this case, the pointer may be slightly misleading in that it will be pointing to this new line and not the line which actually contains the error.

In the descriptions below, "XXX" and "YYY" stand for actual parts of error message texts which are different for different errors. The descriptions detail what these are for each type of error.

XXX expected
: This is not an actual error message, but rather represents a class of error messages, where the "XXX" is replaced by an actual syntactic entity. This message indicates that the given entity is required at this point in the program, but is not present. For example, the message "IDENTIFIER expected" indicates that an identifier (COBOL name) is required but could not be located.

XXX verb not implemented
: Again a template of real error messages, this error occurs whenever the COBOL program contains statements which use verbs which are not implemented in this COBOL system.

XXX syntax
: Again, this is not an actual error message, but a template. The XXX is a keyword phrase which was partially recognized but could not be completely recognized, most likely because of a missing or misspelled keyword. "XXX" may be a complicated entity, such as "ADD/SUBTRACT CORRESPONDING statement".

XXX DIVISION missing
: This is a major error indicating that the compiler could not locate the XXX DIVISION in the program, where "XXX" is either IDENTIFICATION, DATA, ENVIRONMENT or PROCEDURE. Most likely, one or both of these keywords is misspelled.

YYY- unresolved XXX
: This error indicates that a given name ("YYY") was used in the ENVIRONMENT DIVISION which was not later defined in the DATA DIVISION. The "XXX" is an indication of what type of entity this item is which may give a clue as to where to find the appropriate reference. For example, "XXX" could be "file name" which indicates that the name "YYY" was used as the name of a file but was never declared in the DATA DIVISION.

"YYY" improperly declared file name
    The data item "YYY" was used as a file name in a SELECT clause and was not properly declared as an alphanumeric data item in the WORKING-STORAGE SECTION or the LINKAGE SECTION.

"YYY" improperly declared file status item for file "XXX"
    The data item "YYY" was used in the SELECT clause for file "XXX" but was not properly declared as a two character alphanumeric data item in the WORKING-STORAGE SECTION or the LINKAGE SECTION.

"YYY" improperly declared relative key for file "XXX"
    The data item "YYY" was used in a RELATIVE KEY or ALTERNATE KEY clause in the SELECT clause for the file "XXX". This item was not properly declared as an alphanumeric data item within one of the data records for XXX if XXX is an INDEXED file, or a numeric data item in the WORKING-STORAGE SECTION if XXX is a RELATIVE file.

"XXXX" - Undefined procedure or section name
    This error indicates that the name "XXXX" was used as a procedure or section name is the PROCEDURE DIVISION, but was never declared as such.

77 level entry must be elementary
    Level 77 entries cannot be subdivided and thus must be elemental. This implies that they must contain a PICTURE clause and this error will occur if one is found which does not.

88 Level entry not allowed here
    Level 88 entries must immediately follow the declaration of the condition variable. This error will occur if a level 88 entry is found at a point where no condition variable has yet been defined, such as at the beginning of the WORKING-STORAGE SECTION.

ALL must be followed by a string
    The construct "ALL literal" must be either the keyword ALL followed by a figurative constant such as SPACE or ALL followed by a non-numeric literal (string).

AT END phrase required here
    The RETURN statement, used in conjunction with the SORT/MERGE verbs, must specify the AT END phrase.

Ambiguous identifier
    The specified identifier is not fully qualified or has no unique name. This error may actually be an error in definition but will not occur until the identifier is used.

BY/FROM/INTO operand must be a variable
    The operand of a BY, FROM or INTO clause in an arithmetic statement must be a variable if there is not a GIVING clause in the same statement.

CURRENCY SIGN must be a single character
    The character literal in the CURRENCY SIGN IS phrase must be exactly one character long.

Can't access COPY file - ignored
    The file mentioned in the COPY statement could not be located or if it was located, read permission was not present. In this case, the COPY statement is simply ignored and nothing is inserted into the program text at this point.

Can't use GIVING clause with only 1 operand
    Use of the GIVING clause in an arithmetic statement requires at least two operands.

Clause not allowed in SD entry
    A clause was encountered in an SD entry which is not allowed for such an entry.

DATA RECORDS clause error
    The keyword phrase DATA RECORD or DATA RECORDS was being processed but could not be completely recognized. This is most likely caused by misspelling the keyword RECORD or RECORDS.

DECLARATIVES section syntax
    The DECLARATIVES section was being processed and the END DECLARATIVES statement could not be located.

DETAIL group must have a name
    In the REPORT SECTION, report group items which are declared as DETAIL groups must have a label as this is the way they are referenced in GENERATE statements.

Data entry name missing
    In the DATA DIVISION, data items which are declared in structures must have a name or the keyword FILLER directly following the level number.

Data structure definition error
    This error indicates that the data structure currently being defined does not make sense structurally. This can happen if the level numbers are not nested properly.

Duplicate SCREEN clause
    An entry in the SCREEN SECTION contains more than one occurrence of some clause.

Duplicate ON/OFF STATUS phrase
    Switches must be uniquely defined and this error indicates that a given switch has more than one ON/OFF status indication.

Expecting "= <expression>"
    The COMPUTE verb was being examined and the right hand side of the statement could not be successfully located. Most likely, the

    equals sign ("=") is missing.

Expression too complex
    An expression was encountered which was too complex to be handled by the COBOL runtime. This expression should be broken down into two or more simple expressions.

EXIT statement must end with "."
    The EXIT verb must be the only statement in a given paragraph.

FD name is not a FILE NAME
    The name mentioned in the FD statement was not previously declared to be a file in the FILE-CONTROL section of the ENVIRONMENT DIVISION.

FILLER not allowed at level 01/77/88
    The keyword FILLER cannot be used as the name of a data entry which has a level of 77, 88 or 01.

File must be a SORT file (SD)
    The file named in a SORT or MERGE statement must have been declared in the FILE SECTION of the DATA DIVISION as a sort file using the SD statement.

File organization conflicts with operation
    The file named in the operation is not of the proper organization to allow that particular operation. For example, a READ WITH LOCK operation is only allowed on INDEXED or RELATIVE files.

INPUT PROCEDURE or USING phrase required
    The SORT verb requires that the input for the sort operation be specified, either by naming the file (USING) or by providing a section of code which actually produces the input records (INPUT PROCEDURE).

INPUT PROCEDURE phrase not allowed with MERGE verb
    The MERGE verb only allows one or more files to be named as input to the operation.

Identifier must be unique in INDEXED BY
    In the declaration of a table, an INDEXED BY clause was found where the index variable was not unique within the program.

Illegal ACCEPT source
    The ACCEPT verb may specify as input only DAY, DATE, TIME as the source of input if it is not to come from the terminal.

Illegal BLANK clause in SCREEN
    One of the keyword phrases BLANK LINE or BLANK SCREEN was expected, but could not be found, most likely due to a misspelling.

Illegal COPY statement
    The COPY keyword must be followed by a non-numeric literal which is the name of the file to be copied at that point. This string must be followed by a period, with nothing else on the line.

Illegal CURRENCY SIGN character
The CURRENCY SIGN character may be replaced by a single character which must not be any of the following characters:
' 0123456789ABCDLPRSVXZ*+-,.()"/='.

Illegal data hierarchy
A data hierarchy was being defined which was inconsistent. For example, a group item was declared which has no elementary subordinate items.

Illegal identifier syntax
An identifier was detected which ended with a hyphen ("-") or which contained a period (".").

Illegal PICTURE string
The current PICTURE string was not properly formed. See chapter 7 for a complete discussion of PICTURE strings.

Illegal SET value
The value specified in a SET statement must be either a numeric variable or a numeric constant. In either case, the value must represent an integer.

Illegal SIGN clause
The SIGN IS clause must specify either LEADING or TRAILING.

Illegal USAGE clause
The USAGE IS clause was being processed, but the usage specified was not one of DISPLAY, COMPUTATIONAL, COMP, COMPUTATIONAL-1, COMP-1, COMPUTATIONAL-3 or COMP-3.

Illegal character in comment indicator field
Column 1 of the source file contained a character other than ' ', '*', '-' or 'D'. The most probable cause of this error is a label starting in column 1, instead of column 2.

Illegal continuation line
Column 1 contained a continuation indicator ('-'), but columns 2-7 were not totally blank.

Illegal file access mode
File access modes must be one of the following: SEQUENTIAL, RANDOM, or DYNAMIC. No other access modes are supported.

Illegal file name
The external name of a file specified in a SELECT clause must be given by a literal string, an alphanumeric variable or one of the keywords DISC, STD-IN, or STD-OUT.

Illegal file name for DELETE FILE
The DELETE FILE verb requires either a file name, literal string or an alphanumeric data item as an operand.

Illegal file organization
    File organizations must be one of the following: SEQUENTIAL, RELATIVE, or INDEXED. No other file organizations are supported.

Illegal level number
    Level numbers must be 01 through 49, 77 or 88. Level 66 entries are not supported.

Illegal literal value
    A literal value was expected, but could not be located. Legal literals are the figurative literals ZERO, SPACE, QUOTE, HIGH-VALUE, and LOW-VALUE, character strings and numbers.

Illegal number
    A numeric value was present which could not be interpreted by the compiler. Normally, this means that the number had a value outside the range -32768 to +32767.

Illegal option for ACCEPT/DISPLAY verb
    A DISPLAY or ACCEPT statement contained an unrecognizable option. The legal options are: SIZE, LINE, POSITION, ERASE, ECHO, CONVERT and PROMPT.

Imperative statement required here
    Some COBOL statements have a part of the statement which is conditional, based on some condition, e.g. the INVALID KEY phrase in a READ statement. This phrase must contain an imperative statement. See page NN for the list of imperative statements.

Inconsistent PAGE LIMITS
    The PAGE LIMIT values given in a RD entry are inconsistent. The values must be in the relationship:
    1 <= HEADING <= FIRST DETAIL <= LAST DETAIL <= FOOTING <= PAGE LIMIT

Invalid MOVE variable
    A variable was used in a MOVE statement which was not properly declared in the WORKING-STORAGE, LINKAGE or FILE section of the DATA DIVISION. Condition names, switch names, file names, etc, cannot be used in the MOVE statement.

Invalid REPORT GROUP name
    The report group named in a USE BEFORE REPORTING statement was not properly defined as a group in the REPORT SECTION.

Invalid Report type for "XXX"
    "XXX" is the name of a report which was specified in an FD statement. This name must be resolved in the REPORT SECTION using an RD statement.

Invalid SORT/MERGE key
    Keys used in the SORT/MERGE verb must be alphanumeric and must be contained within the data record specified for the SD file.

Invalid SOURCE clause
    A SOURCE clause in the REPORT SECTION must refer to either a variable or one of the keywords PAGE-COUNTER or LINE-COUNTER.

Invalid START KEY phrase
    Some portion of the START KEY phrase could not be recognized. This will occur if the keyword KEY cannot be located or if the starting condition is not EQUAL, GREATER or NOT LESS.

Invalid TYPE entry
    The TYPE IS phrase in the REPORT SECTION must be one of the following; RH, REPORT HEADING, RF, REPORT FOOTING, PH, PAGE HEADING, PF, PAGE FOOTING, DE, DETAIL.

LABEL RECORDS clause error
    The LABEL RECORDS clause was partially recognized but could not be totally recognized. This is due to the keyword RECORDS being misspelled or missing.

LABEL RECORDS clause missing
    The LABEL RECORDS clause is required in all FD entries.

LINAGE and REPORT clauses not allowed together
    Sequential files may not be declared with both LINAGE and REPORT clauses. This is because the REPORT clause conflicts with the LINAGE option.

LINAGE clause error
    The LINAGE clause is improperly specified.

LINAGE clause not allowed with this file organization
    The LINAGE clause is only allowed with SEQUENTIAL files.

Level 01 entry must have TYPE clause
    A level 01 entry in the REPORT SECTION must contain a TYPE IS clause. If this is a TYPE IS DETAIL clause, this item must also be named.

Level 77 or 88 items not allowed in ADD/SUBTRACT/MOVE CORRESPONDING
    The items named in a ADD/SUBTRACT/MOVE CORRESPONDING statement must not have been declared at either level 77 or 88.

MERGE requires two or more input files
    The MERGE verb requires that the USING clause contain two or more file names.

Missing FD or SD entry for "XXX"
    The file "XXX" was mentioned in the FILE-CONTROL but never appeared in either an FD or SD entry in the DATA DIVISION.

Missing paragraph or section name
    The compiler could not locate the name of a paragraph or section at the begining of the PROCEDURE DIVISION.

Missing RD entry for "XXX"
    The file "XXX" was mentioned in a REPORT IS clause of an FD entry but never appeared in a RD clause in the REPORT SECTION.

Missing SOURCE/VALUE/SUM clause
    Each output field specification in the REPORT SECTION must define a source for that field. This error occurs if none of the valid source specifications have been found.

Mixed DISPLAY statement modes
    The DISPLAY statement allows more than one item to be "DISPLAYED". These items must be either all screen items or non-screen items.

Must be a paragraph or section name
    The identifiers specified in a PERFORM statement, or INPUT PROCEDURE or OUTPUT PROCEDURE phrases of the SORT or MERGE verbs, must refer to either a paragraph or a section name.

Must be group items
    In a ADD/SUBTRACT/MOVE CORRESPONDING statement, the items mentioned must both be group items.

NEXT GROUP clause error
    The format of a NEXT GROUP clause in the REPORT SECTION is in error.

No USE SECTION defined for file
    A READ, REWRITE or START verb was processed which did not contain an INVALID KEY or AT END phrase and the corresponding file does not have a USE section defined for it. All operations of this type must have one of these clauses or have a USE section defined for the file.

No corresponding fields
    A MOVE CORRESPONDING operation was specified for which there were no fields that correspond between the two operands.

No such CONTROL in this report
    A CONTROL HEADING or CONTROL FOOTING report group in the REPORT SECTION uses a variable which was not included in the CONTROLS ARE list for the report being processed.

Not allowed at this level
    In the REPORT SECTION, the keyword phrases GROUP INDICATE, JUSTIFIED RIGHT, and BLANK WHEN ZERO are not allowed in level 01 entries.

Numeric value truncated
    The numeric value specified in a VALUE clause in the DATA DIVISION cannot be represented within the PICTURE of the data item.

OCCURS not allowed at level 01 or 77
    The OCCURS phrase is not allowed for items declared with a level of 01 or 77.

ON ASCENDING/DESCENDING KEY phrase missing
The SORT and MERGE verbs require at least one of these phrases to determine how to SORT/MERGE the files.

OPTIONAL clause only allowed with SEQUENTIAL files
The OPTIONAL clause, which allows files to be opened for INPUT even if they do not exist, is only permissible for SEQUENTIAL files.

OUTPUT PROCEDURE or GIVING phrase required
The SORT and MERGE verbs require either a GIVING file-name or OUTPUT PROCEDURE clause. Neither of these clauses could be located in the SORT/MERGE statement currently being processed.

Operand type error
This error indicates that one of the operands for an arithmetic operation is invalid for that operation. Variables used in arithmetic operations must have been declared either numeric or numeric edited in either the WORKING-STORAGE, LINKAGE or FILE sections of the DATA DIVISION. Also, numeric edited fields may only be specified as destinations, i.e. they may not be used as source operands for operations. Non-numeric literals are not allowed as operands for arithmetic operations.

PIC and VALUE not allowed together
In the SCREEN SECTION, the PIC and VALUE phrases are mutually exclusive.

PICTURE string is inconsistent
This error indicates that the PICTURE string, while being syntactically correct, does not make sense as a representation of a data object. In particular, this error will occur if the 'S' picture element appears as any character other than the first character in a picture string. Also, this error may occur if the picture string contains picture elements which do not make sense when specified together, such as 'DB' with an alphabetic field. See chapter 7 for more details.

PICTURE string too long
PICTURE strings must contain no more than 30 characters, including the expansion of repeated values such as 9(6). The only exception to this rule is for alphanumeric items such as X(50) for which there is no limit on the "size" of the PICTURE string.

Paragraph or Section name multiply defined
The specified name has already been defined as a paragraph name or a section name. This compiler does not currently support paragraph names qualified by section names, so all paragraphs and sections in the program must have unique names.

Previous item must be elementary
The data item declared just prior to the current data item is restricted to be an elementary data item. This error occurs if the

data item currently being defined has a level number greater than the data item just defined (and the level is not 77 or 88) and the previous data item was restricted to be an elementary data item.

Data items which contain any of the phrases SYNCHRONIZED, JUSTIFIED, BLANK WHEN ZERO or PICTURE must be elementary. Also, FILLER items must be elementary.

REDEFINES cannot be nested
The REDEFINES clause may not appear within a structure which itself contains a REDEFINES clause.

REDEFINES must occur at same level
The item which is being REDEFINED must have the same structural level as the current item being defined.

REDEFINES not allowed at level 01 in FILE SECTION
The REDEFINES clause is not allowed for any entries declared in the FILE SECTION with a level of 01.

REPORT DETAIL group item required
The GENERATE verb requires as its operand a report group which was declared with TYPE IS DETAIL in the REPORT SECTION.

REPORT NAME (RD) item required
The INITIATE and TERMINATE verbs require the name of a report as their operands. This item must have been declared using the RD clause in the REPORT SECTION.

REPORT clause only allowed with Sequential file
The REPORT IS clause can only be used with a SEQUENTIAL file.

REPORT structure (RD) inconsistent
This error indicates that a detail group item was found in a GENERATE statement, but the compiler could not determine which report that detail item was for. This is a error in the compiler and not necessarily the program. Please report occurrences of this error to Technical Systems Consultants.

Report Group already defined for this CONTROL
A CONTROL HEADING or CONTROL FOOTING has already been processed which used this particular control variable.

Report group definition syntax
The syntactic structure of a report group in the REPORT SECTION is invalid. This is most likely due to improper nesting of levels.

SD file must be Sequential file
Files named in an SD entry must have been specified as SEQUENTIAL files in the FILE-CONTROL paragraph.

SIGN clause or S picture element not allowed
The SIGN IS clause or the 'S' picture element may only be specified for numeric or numeric edited fields.

SORT/MERGE keys must be alphanumeric display item
The keys used in the SORT/MERGE verbs must be declared as alphanumeric display data items within a data record associated with the sort file.

SUM clause only allowed in CONTROL FOOTING group
In the REPORT SECTION, the SUM clause can only appear in a TYPE IS CONTROL FOOTING report group.

SUPPRESS verb only allowed in USE BEFORE REPORTING sections
The SUPPRESS verb is only allowed in the DECLARATIVES section of the program and then only in a USE BEFORE REPORTING section.

String syntax error
A literal string was being processed but the end of the line was encountered before the closing string quote, either "'" or '"', was encountered. This is most likely due to an unbalanced set of quotation marks. If it is necessary to include one of the quotation marks in a string, use the other quotation mark to enclose the string.

Subscript must be an integer variable
The value used to subscript a table must be either an integer constant or a variable which has an integer value. In the case of a variable, the PICTURE for that variable must not contain any places to the right of the decimal point.

Subscripted variable not allowed here
The current statement does not allow subscripted variables to be used in that statement. The desired value must be MOVED to a non-subscripted variable for the purposes of this statement.

Subscripting not allowed for non-elemental items
Data items which contain an OCCURS clause but are not elementary data items may not be accessed directly. Only elementary data items may be accessed using subscripting.

TO / UP/DOWN BY clause required
The SET statement requires either a TO clause or an UP/DOWN BY clause to be specified. This error indicates that none of these clauses could be recognized.

TO item cannot be EDITED
In the SCREEN SECTION, variables used in a TO or USING clause must refer to data items whose picture is not numeric edited or alphanumeric edited.

TO item must be USAGE DISPLAY
The variables used in a TO or USING clause in the SCREEN SECTION must have USAGE DISPLAY. USAGE COMPUTATIONAL, or any of its variations, is not allowed.

TYPE phrase allowed only at level 01
    In the REPORT SECTION, the TYPE IS phrase can only be specified on a report group item which is declared at level 01.

Too many ALTERNATE KEY definitions
    An SELECT statement for an INDEXED file may only specify a primary key and up to 4 alternate keys.

Too many CONTROLS
    The CONTROLS ARE clause in the REPORT SECTION is limited to 10 levels of control variables.

Too many SUM counters in REPORT
    The compiler limits COBOL programs to 50 total SUM clauses in the REPORT SECTION.

Too many UPON values for this SUM counter
    A single SUM counter may reference no more than 10 UPON report groups.

Too many labels in GOTO DEPENDING
    The GOTO ... DEPENDING ON statement is restricted to 25 paragraph or section names. If more names than this are necessary, break the statement down into multiple cases.

Too many sources for this SUM counter
    A single SUM counter in the REPORT SECTION may refer to no more than 10 SOURCE variables.

USAGE COMP with non-NUMERIC item
    USAGE COMPUTATIONAL may only be specified with elementary numeric data items.

USAGE must be DISPLAY
    This error indicates that the current statement uses a variable which was declared to be COMPUTATIONAL but which must be USAGE DISPLAY.

USE statement required here
    In the DECLARATIVES section of the PROCEDURE DIVISION, a USE statement must occur after each SECTION name. This error indicates that the compiler could not locate the USE statement for the SECTION being defined.

USING/GIVING file must be SEQUENTIAL
    Files named in a USING or GIVING phrase in the SORT or MERGE statement must have been defined as sequential files in the FILE-CONTROL paragraph.

Undeclared identifier
    The specified identifier is not declared in the DATA DIVISION or the ENVIRONMENT DIVISION.

Unkown SCREEN clause name
    The valid clauses in the SCREEN SECTION are: SECURE, AUTO, REQUIRED, FULL, LINE, COL, BLANK SCREEN, BLANK LINE and BELL.

Unrecognizable verb
    A statement was processed which could not be recognized as a COBOL statement.

VALUE clause not allowed in this section
    The VALUE clause may only appear in the WORKING-STORAGE section of the DATA DIVISION.

VALUE conflicts with data type
    The value specified for a data item does not correspond with the PICTURE which was given for that data item.

VALUE not allowed with OCCURS clause
    The VALUE IS clause may not be included in a data definition which contains an OCCURS clause.

VALUE not allowed within REDEFINES
    The VALUE IS clause must not appear with the scope of a REDEFINES clause. The VALUE clause is only allowed for primary elementary data items.

VALUE size exceeds item size
    The value given in the VALUE IS clause will not fit in the associated variable as defined.

Variable declared with too many dimensions
    Tables may only have 3 or fewer dimensions. This error occurs only when a data item which is an element of a table that has more than 3 dimensions is referenced and is not detected when the data definition occurs in the DATA DIVISION.

Variable is not an array
    A variable was used with a subscript but was not declared as part of a table using the OCCURS clause.

Wrong number of subscripts
    A variable is being referenced for which the number of subscripts does not match the number of dimensions for the data item.

UP/DOWN BY clause syntax
    The keyword BY could not be located when processing an UP BY or DOWN BY clause in a SET statement.

C. RUNTIME ERROR MESSAGES

The following is a list of the messages which a COBOL program may generate as the result of some error condition at runtime. The messages are grouped by the function being performed, i.e. all messages associated with the Report Writer are grouped together.

General errors

System error code = NNNNN
: This error will be printed after some other error found in this list, normally an I/O error. The code number NNNNN will be the UniFLEX error code, a small decimal number, and may be found in the UniFLEX programming manual. Normally, this error indicates a major problem with the COBOL program or the hardware on which it runs.

COBOL Program Abnormal Termination - at line NNNNN
: This message will be printed any time the COBOL program is terminated other than the program executing the 'STOP RUN' statement. The line number printed is the number of the line being executed when the program was terminated. This line number may be correlated with a source listing to determine the actual location of the failure. In most cases, this message will be prefaced by some other message which indicates the reason the program was terminated.

STOP errors

PAUSE error
: This error indicates an Input/Output error during the execution of a 'STOP literal' statement.

PAUSE - can't move data
: This error occurs if the 'STOP literal' statement was being executed and the system was unable to MOVE the literal data in order to print it. This indicates an error in the runtime system and should be reported.

ACCEPT errors

ACCEPT - Illegal source
: This error indicates that the ACCEPT verb processor could not recognize the source for the statement. This occurs during an ACCEPT FROM ... statement and the error indicates something major wrong with the program or the compiler. This indicates an error in the runtime system and should be reported.

ACCEPT - I/O error
    This error will occur if there are any errors encountered during Input/Output operations performed by the ACCEPT verb processing.

CALL PROGRAM errors

Error creating communications file during CALL
Error closing communications file during CALL
Error opening communications file during CALL
Error writing communications file during CALL
Error setting up communications file for CALL
Error deleting communications file in CALL
    These errors are self-explanatory and are accompanied by a system error code.

Argument overflow in CALL
    This error indicates that the CALL PROGRAM statement could not be processed successfully since the argument list, in this case one which is being passed to shell, was too large.

Unable to execute "xxxxx"
    The program to be executed by the CALL PROGRAM statement could not be located or was not executable.

Unable to fork sub-task in CALL
    This error indicates that the user has too many tasks running in the system. See the UniFLEX System Manager's guide for more details.

DISPLAY errors

DISPLAY - No Clear Screen function defined
    The "/etc/termcap" file does not have the clear screen function defined for this terminal. This function is part of the minimal set for COBOL and must be specified. See appendix D for details on how this file should be set up.

DISPLAY - Error moving data
    This error occurs if the DISPLAY processor gets an error while trying to MOVE data to be displayed. The most common cause of this error is trying to display uninitialized data.

DISPLAY - I/O Error!
    This error occurs if there are any Input/Output errors encountered by the DISPLAY processing. It will be accompanied by another message indicating the actual error code.

Indexing errors

Integer conversion error
: This error occurs if a variable used as an index in an indexed expression cannot be converted to binary. The most common cause of this error is the variable containing characters which are not digits.

Index value out of range - value = NNNNN
: This error occurs if an index value is less than or equal to zero, or has a value greater than the number of elements in a table.

Initialization errors

Error during CRT initialization
: This error indicates that the system could not properly read the file '/etc/termcap', or it was not properly formed. See appendix D for more details on how to get the environment set up to run COBOL programs.

Program ABORT
: This error is printed if the control-backslash character is typed at the terminal. This is the only mechanism for aborting a COBOL program from the user's terminal.

INSPECT errors

Illegal INSPECT option
: This error indicates that the INSPECT processor could not recognize the INSPECT statement being processed. This error indicates a flaw in either the compiler or the runtime and should be reported.

Input/Output Statement errors

Illegal KEY specified
: This error indicates that a key was used in an INDEXED I/O statement which was not properly declared as one of the keys for the given file.

Illegal operation on file
: An I/O statement was attempted on a file which was not opened properly to allow that operation. For example, a WRITE statement used with a file open for INPUT only.

Can't DELETE file 'xxxxx'
    The DELETE FILE verb could not successfully delete the specified file.

REL routine called and REL not loaded!
VSAM routine called and VSAM not loaded!
    These errors indicate a major problem with the COBOL compiler/runtime and should be reported immediately.

PERFORM errors

PERFORM table overflow
    The PERFORM verb uses an internal table to keep track of what procedures are being performed. This error indicates that more procedures are active than can be handled, with the limit currently being 20 active procedures and sections.

REPORT WRITER errors

RWCS - Report file not open
The file associated with a report must have been previously opened for OUTPUT before an INITIATE statement may be executed.

RWCS - Report previously INITIATED
This error indicates that the report has already been INITIATED. A TERMINATE statement must be executed before another INITIATE statement can be executed.

RWCS - Report not INITIATED
This error occurs if any statement other than INITIATE is attempted on a report which has not been INITIATED.

RWCS - Illegal Report Group Type Code
This is an error in the COBOL compiler/runtime and should be reported.

RWCS - Illegal Line value
This error indicates that the value in a LINE clause is not valid, i.e. less than one or greater than the number of lines on the page.

RWCS - Error moving data
This error indicates that the RWCS could not properly MOVE some piece of data. Most likely this is due to uninitialized data being referred to in the report.

RWCS - Internal buffer overflow
This is an error in the COBOL compiler/runtime and should be reported.

RWCS - Column too large
This error indicates that the value in a COLUMN clause exceeds the line width for the report.

SORT / MERGE errors

SORT/MERGE INPUT KEY too large
SORT/MERGE FILE NAME exceeds 256 characters
Error creating SORT/MERGE parameter file
Error writing SORT/MERGE parameter file
Error closing SORT/MERGE parameter file
Error redirecting SORT/MERGE output file
Error deleting SORT/MERGE temp input file
Error creating SORT/MERGE temp output file
Error deleting SORT/MERGE temp output file
Error opening SORT/MERGE output file
Error "XX" while opening SORT/MERGE temp input file
Error "XX" while writing SORT/MERGE temp input file
Error "XX" while opening SORT/MERGE temp output file
    These errors are self-explanatory and most of them are accompanied by a system error code related to the error.

SORT/MERGE operation already in progress
    Only one SORT or MERGE operation may be in progress (in a single program) as one time.

No SORT/MERGE in progress
    A RELEASE or RETURN statement was executed but no SORT or MERGE operation is currently in progress.

No SORT/MERGE active for this file
    The file used in a RELEASE or RETURN statement is not currently being SORTED or MERGED.

Unable to fork SORT/MERGE sub-task
    The user has too many tasks running in the system. See the UniFLEX System Manager's guide for more details.

Unable to execute SORT/MERGE package
    The COBOL program was unable to find or execute the SORT/MERGE package. This package is provided with COBOL and such an error indicates either the file (/bin/psort) has been deleted or tampered with.

D. CRT TERMCAP DESCRIPTION

With today's plethora of terminals in use in computer systems, a unified mechanism for describing the available functions for each terminal is necessary. This mechanism makes possible the use of general programs which can function independently of the terminal device on which they are running. For example, COBOL has many CRT screen oriented functions, both for input and output. In order to allow a single COBOL program to run on any given terminal at a particular installation, that COBOL program must be able to determine how to interact with the specific terminal. In this fashion, a COBOL program may be written with little or no foreknowledge of the types of terminals on which it may run. Also, the operator environment is more stable since all terminals will behave similarly.

In the COBOL system, along with cooperation with the UniFLEX operating system, a mechanism has been defined to allow these functions to take place. To this end, a file is defined which describes the various functions for all of the terminals at the particular installation. This file is called the "termcap" file, for terminal capabilities file, and is located in the "/etc" directory. It must be defined for each specific terminal on the system. Then, when a COBOL program runs, it can interrogate this file to determine how to perform the screen functions.

The termcap file is created by using a program, "/etc/crt_termcap". This program reads two files. The first file describes the functional capabilities of all the terminals on the system. The second file indicates what type of terminal is on each active port in the system. The information in these two files together is used to make up the termcap file. The format of each of these files will be described below.

The first file is the actual description of the functional capabilities of each possible terminal type present on the system. This file is known as the "ttycap" file. There is one logical "line" or entry per terminal in the file. The basic form of the "line" is:

```
       terminal-name: capabilities :
where
       terminal-name - up to 10 characters describing the terminal
       capabilities - keyword values, separated by spaces.  These
         have the individual form:
           kw=dvs       {keyword=decimal value string}
       decimal value strings are strings of decimal values,
       separated by plus signs, e.g. 1+2+3
```

The keywords currently supported are:

```
ho - Home cursor.
cu - Move cursor up without modifying display.
cd - Move cursor down.
cl - Move cursor left.
cr - Move cursor right.
```

cs - Clear entire screen.
nr - Number of rows on screen (counting from 1).
nc - Number of columns on screen (counting from 1).
wt - Number of seconds to wait after a clear screen function.
before sending more information to the terminal.
is - A string of characters, sent to the terminal when processing
begins, for purposes of initializing the terminal.
bl - Blank the current line, from the present cursor position
through the end of the line, without moving the cursor.
bm - Place the terminal in "background" mode. This is normally
a mode where all characters written to the terminal are written
with a lower intensity (brightness) than normal.
fm - Place the terminal in foreground (normal) mode.
ku - Sent by terminal in response to up-arrow key.
kd - Sent by terminal in response to down-arrow key.
kl - Sent by terminal in response to left-arrow key.
kr - Sent by terminal in response to right-arrow key.
kh - Sent by terminal in response to home key.
k0-k9 - Sent by terminal in response to "other" special keys.

An example of a "line" describing a terminal would be:

```
  ct82:ho=16  cu=01  cd=02  cr=09  cl=04  cs=30+07+12  nr=20  nc=82
     is=28+18+30+19+30+20+30+07 bl=06  bm=28+05  fm=28+21+30+07  wt=1 :
```

This "line" describes the SWTPC "ct82" terminal.

The second file used by crt_termcap is a list indicating what terminal types are actually hooked up on which ports on the system. This file is called the "ttyassoc" file, for terminal association file, and is an extended form of the "/etc/ttylist" file. There is one line per terminal in the file, each line having the form:

```
     + nn :xxxx:yyyy:
```

where

nn - The terminal number, such as 07.
xxxx - The type of the terminal attached to this port. This name
MUST be one of the terminals described in the "ttycap" file
mentioned above.
yyyy - A descriptive name, primarily for documentation.
This name should normally be the name of the person most
commonly using the terminal. However, it has no functional
meaning and can be left empty.

The procedure for creating a termcap file is quite simple. First, the ttycap file must be edited, adding or modifying the entries to describe the various terminals on the system. A starter ttycap file which describes some common terminals is provided with the system to make this easier. Second, the ttyassoc file must be created. The easiest way is to take the existing /etc/ttylist file and copy it to the ttyassoc file. Then edit this file, adding the appropriate fields as described above. The last step is to actually run the crt_termcap program. The command to do this is:

```
++ /etc/crt_termcap TTYCAP TTYASSOC TERMCAP
```

The various files used by the program are listed above in uppercase. This means that when the program is actually run, the actual name of the various files to be used must be substituted for the uppercase names listed. For example, if all the necessary files were contained in the /etc directory, the command could be:

```
++ /etc/crt_termcap /etc/ttycap /etc/ttyassoc /etc/termcap
```

Alternatively:

```
++ chd /etc
++ crt_termcap ttycap ttyassoc termcap
```

After executing this command, the termcap file would be created and you will be ready to execute a COBOL program.

E. INTERACTIVE COBOL DEBUG TOOL

This appendix describes the functions and usage of the Interactive COBOL Debug tool (ICD). The ICD is included in any program which is compiled with the "D" option. When the program begins execution, the ICD will take over control of the terminal from which the program was run. The user may then enter commands to the ICD to run the program, stop at particular points in the program and display variable contents. The ICD is a passive debug tool in that the flow of the program and the values of variables in the program may only be interrogated, not modified. However, the operations provided give a large amount of debugging power.

The commands for the ICD are quite simple and are entered at the user's terminal. Each of the possible commands has a complete form as well as an abbreviated form. In the discussion below, the abbreviated form is written in capital letters. When commands are actually entered to the ICD, either case of letters is acceptable. If the abbreviated form of a keyword is not specified, then the entire keyword must be spelled out. Some commands also have optional keywords which may be left out if the user desires. When an address is indicated in the discussion below (as <addr>), that address may be any of the following: an absolute integer line number; the name of a paragraph or section in the program; or the special character ! which stands for the end of the program. The description of each command follows:

Breakpoint at <addr>

> This command will set an execution breakpoint at the specified address. When the program executes the statement at that address, the program will be temporarily halted and the ICD will enter command mode. Breakpoints are removed using the "remove" command.

Clear breakpoints

> This command removes all current program breakpoints.

DEbug on <device-name>

> This command allows the debug process to take place on a terminal other than the one which initiated the execution of the program. It is particularly useful if the program being debugged uses screen oriented input/output statements. Without using this command, the ICD will destroy the "current" cursor position on the screen, making the output of such programs less coherent. When this command is executed, the device specified must be that of a terminal device and must be accessable to the user who originally executed the program. This means that the terminal must be "owned" by that user and must not have any other program currently running on it which is expecting terminal input. This situation may be achieved by either having the specified terminal not enabled for login purposes (see the System Manager's Guide) or to login on the terminal with the same user name as the user of the program and run

a program which does nothing. A program which "does nothing" very efficiently follows:

```
          lib sysdef
     start sys stop
          ldd 0
          sys term
          end start
```

By running this program before executing the "debug" command, the ICD will be able to access the specified terminal. When the debug process is complete, simply enter control-C on the alternate debug device.

DIsplay <variable-name>

This command allows the user to display the contents of a given variable. The variable named cannot be indexed and if it is a group item, all items subordinate to the item named will be displayed. Some care should be taken when displaying group items however, since no conversions will take place and the output may be difficult to interpret. If the name of the item to be displayed has to be qualified, the qualification keywords "of" and "in" should be replaced by the character "/". For example, to display the variable "x of y in z", enter the command:

```
display x/y/z
```

Exit

This command instructs the ICD to terminate the program immediately. All currently open files will be closed and the program terminated.

Go [till <addr>]

This command instructs the ICD to begin/continue execution of the program. If the optional "till" phrase is specified, a breakpoint will be set at that address before the execution is resumed.

List breakpoints

This command lists all the current breakpoints, indicating their types, either line number, paragraph name or section name.

Remove <addr>

This command allows the user to remove a specific breakpoint from the breakpoint lists. This is the only mechanism (except for clear) for removing breakpoints. Even if a breakpoint is actually executed, the breakpoint remains in place.

Step [n]

This command allows the user to "step" through his program. When entered, the ICD will allow the program to run for "n" steps before returning to the command mode. If the integer value "n" is not given, one step will be executed. The ICD interprets any of the following actions as a "step":

Program Abnormal Termination
Program Normal Termination
Execution of a new line in the program
Execution of a PAUSE statement
Enter or Exit a PROCEDURE or SECTION

Trace [n]

This command instructs the ICD to allow the program to execute for "n" steps, with a trace will being printed at the debug terminal for each step executed. After "n" steps have been executed, the ICD returns to command mode.

F. COBOL UTILITIES

In addition to the COBOL system itself, there are utility programs which are used in conjunction with the COBOL file system. The following are descriptions of these programs.

"unlock-all" - Unlock records in an Indexed or Relative file.

The COBOL system attempts to guarantee that records which were locked by a program are unlocked before that program terminates. However, if the

COBOL program should terminate in such a manner that normal closing procedures are not executed, it may leave some records locked in the file. Until these records are unlocked, they will be inaccessible to other COBOL programs. The "unlock-all" utility is used to unlock such records. This utility may be called in any one of three different ways.

```
unlock-all file-name
unlock-all file-name +a
unlock-all file-name terminal-list
```

In each of these forms, "file-name" is the name of the Indexed or Relative file which contains records that have been left locked.

In the first form, where only the file name is specified, all records which had been locked by the terminal which called "unlock-all" are unlocked. In the second form, where a "+a" is specified, all records are unlocked. The third and final form has a list of terminal numbers as an argument. Only records locked by those terminals are unlocked. The terminal numbers are specified as decimal numbers.

"vsamcopy" - Copy an Indexed file

The utility program "vsamcopy" will copy an Indexed file record by record. The most frequent application of this program is in the reconstruction of an Indexed file that was damaged by a hardware failure or system crash. This utility is called in the following way:

```
vsamcopy old-file new-file
```

Both arguments are required and "new-file" may not be the same name as "old-file". The "old-file" is read in the "sequential access" mode and therefore does not rely on much of the complicated structure of an Index file. It is not necessary to specify any information concerning the record and key structure of the Indexed file since such information is obtained by "vsamcopy" by reading information encoded in the file itself.

G. RESERVED WORDS

The following list contains all keywords and special names used by the COBOL compiler. These words are truly reserved and may only be used within certain contexts which are described in the syntax diagrams. They are listed here in lower case although they may be written in either case in a COBOL program.

accept	access	add	advancing
after	all	alphabetic	also
alter	alternate	and	are
area	areas	ascending	assign
at	author	auto	before
bell	blank	blink	block
bottom	by	call	cancel
cd	cf	ch	character
characters	clock-units	close	cobol
code	code-set	col	collating
column	comma	communication	comp
computational	comp-1	computational-1	comp-3
computational-3	compute	configuration	contains
control	controls	convert	copy
corr	corresponding	count	currency
data	date	date-compiled	date-written
day	de	debug-contents	debug-item
debug-line	debug-name	debug-sub-1	debug-sub-2
debug-sub-3	debugging	decimal-point	declaratives
delete	delimited	delimiter	depending
descending	destination	detail	disable
disc	display	divide	division
down	duplicates	dynamic	echo
egi	else	emi	enable
end	end-of-page	enter	environment
eop	equal	erase	error
escape	esi	every	exception
exit	extend	fd	file
file-control	filler	final	first
footing	for	from	full
generate	giving	go	greater
group	heading	high-value	high-values
i-o	i-o-control	identification	if
in	index	indexed	indicate
initial	initiate	input	input-output
inspect	installation	into	invalid
is	just	justified	key
label	last	leading	left
length	less	limit	limits
linage	line	line-counter	lines
linkage	lock	low-value	low-values
memory	merge	message	mode
modules	move	multiple	multiply

native
negative
next
no
not
number
numeric
object-computer
occurs
of
off
omitted
on
open
optional
or
organization
output
overflow
page
page-counter
perform
pf
ph
pic
picture
plus
pointer
position
positive
printing
procedure
procedures
proceed
program
program-id
prompt
queue
quote
quotes
random
rd
read
receive
record
records
redefines
reel
references
relative
release
remainder
removal
renames
replacing
report
reporting
reports
required
rerun
reserve
reset
return
reversed
rewind
rewrite
rf
rh
right
rounded
run
same
screen
sd
search
section
secure
security
segment
segment-limit
select
send
sentence
separate
sequence
sequential
set
sign
size
sort
sort-merge
source
source-computer
space
spaces
special-names
standard
standard-1
start
status
std-in
std-out
stop
string
sub-queue-1
sub-queue-2
sub-queue-3
subtract
sum
suppress
switch
symbolic
sync
synchronized
table
tallying
tape
terminal
terminate
text
than
through
thru
time
times
to
top
trailing
type
unit
unlock
unstring
until
up
upon
usage
use
using
value
values
varying
when
with
words
working-storage
write
zero
zeroes
zeros

H. SYNTAX DIAGRAMS

The following pages contain diagrams which describe the details of the syntax of COBOL programs. They have been copied from the ANSI 1974 COBOL standard document and modified to reflect this particular implementation.

General Format for IDENTIFICATION DIVISION

```
IDENTIFICATION DIVISION.
PROGRAM-ID.  program-name.
AUTHOR.        comment-entry ...
INSTALLATION.  comment-entry ...
DATE-WRITTEN.  comment-entry ...
DATE-COMPILED. comment-entry ...
SECURITY.      comment-entry ...
```

General Format for ENVIRONMENT DIVISION

```
ENVIRONMENT DIVISION.

CONFIGURATION SECTION.

SOURCE-COMPUTER.  computer-name WITH DEBUGGING MODE .

OBJECT-COMPUTER.  computer-name

                                WORDS
    ,  MEMORY  SIZE  integer    CHARACTERS   .
                                MODULES

SPECIAL-NAMES.

   SWITCH  literal  ,  ON   STATUS  IS  condition-name
                    ,  OFF  STATUS  IS  condition-name

  , CURRENCY  SIGN  IS  literal

  , DECIMAL  POINT IS  COMMA .


INPUT-OUTPUT SECTION.

FILE-CONTROL.
   File Control Entry  ...


I-O-CONTROL.

          RECORD
  SAME    SORT           AREA  FOR  file-name   , file-name
          SORT-MERGE
```

General Format for File Control Entry

Format 1:

```
   SELECT  OPTIONAL  file-name

                 DISC
                 STD-IN
      ASSIGN  TO STD-OUT
                 literal
                 file-name-variable

      ;  RESERVE  integer  AREA
                           AREAS

      ;  ORGANIZATION  IS  SEQUENTIAL

      ;  ACCESS  MODE  IS  SEQUENTIAL

      ;  FILE  STATUS  IS  data-name    .
```

Format 2:

```
   SELECT  file-name

                 DISC
                 STD-IN
      ASSIGN  TO STD-OUT
                 literal
                 file-name-variable

      ;  RESERVE  integer  AREA
                           AREAS

      ;  ORGANIZATION  IS  RELATIVE

      ;  ACCESS  MODE  IS  SEQUENTIAL  ,  RELATIVE  KEY  IS  data-name

                           RANDOM
                           DYNAMIC     ,  RELATIVE  KEY  IS  data-name

      ;  FILE  STATUS  IS  data-name    .
```

General Format for File Control Entry

Format 3:

```
SELECT  file-name

                DISC
                STD-IN
     ASSIGN  TO STD-OUT
                literal
                file-name-variable

     ;  RESERVE  integer  AREA
                          AREAS

     ;  ORGANIZATION  IS  INDEXED

                          RANDOM
     ;  ACCESS  MODE  IS  SEQUENTIAL
                          DYNAMIC

     ;  RECORD  KEY  IS  data-name

     ;  ALTERNATE  RECORD  KEY  IS  data-name  WITH  DUPLICATES

     ;  FILE  STATUS  IS  data-name    .
```

Format 4:

```
                                    DISC
                                    STD-IN
SELECT  file-name    ASSIGN  TO     STD-OUT
                .                   literal
                                    file-name-variable
```

General Format for DATA DIVISION

```
DATA DIVISION.

FILE  SECTION.

FD  file-name

   ;  BLOCK  CONTAINS  integer   TO   integer   RECORDS
                                                CHARACTERS

   ;  RECORD  CONTAINS  integer  TO  integer  CHARACTERS

   ;  LABEL  RECORD   IS     STANDARD
             RECORDS  ARE    OMITTED

   ;  DATA   RECORD   IS     data-name

   ;  LINAGE  IS  integer  LINES
      ,  WITH  FOOTING  AT  integer
      ,  LINES  AT  TOP  integer
      ,  LINES  AT  BOTTOM  integer

   ;  REPORT  IS  report-name

   Record  Description  Entry


SD  file-name

   ;  RECORD  CONTAINS  integer  TO  integer  CHARACTERS

   ;  DATA   RECORD   IS     data-name

   Record  Description  Entry


WORKING-STORAGE  SECTION.

   77 - Level Description Entry
   Record  Description  Entry

LINKAGE  SECTION.

   77 - Level Description Entry
   Record  Description  Entry

SCREEN  SECTION.

  Screen  Group  Description  Entry
```

General Format for DATA DIVISION

```
REPORT  SECTION.

RD  report-name

   ;  CONTROL   IS    data-name  ,  data-name
      CONTROLS  ARE   FINAL  ,  data-name

   ;  PAGE  LIMIT   IS   integer   LINE
            LIMITS  ARE           LINES

         ,  HEADING  integer

         ,  FIRST  DETAIL  integer

         ,  LAST  DETAIL   integer

         ,  FOOTING   integer  .

   Report  Group  Description  Entry
```

General Format for Data Description Entry

Format 1:

```
level-number  FILLER
              data-name

      ;  REDEFINES  data-name

      ;  PICTURE  IS  picture-specification
         PIC

                   COMPUTATIONAL
                   COMP
                   COMPUTATIONAL-1
      ;  USAGE  IS  COMP-1
                   COMPUTATIONAL-3
                   COMP-3
                   DISPLAY
                   INDEX

      ;  SIGN  IS   LEADING    SEPARATE   CHARACTER
                   TRAILING

      ;  OCCURS  integer  TIMES   INDEXED  BY  index-name


      ;  SYNCHRONIZED   LEFT
         SYNC           RIGHT

      ;  JUSTIFIED      RIGHT
         JUST

      ;  BLANK  WHEN  ZERO

      ;  VALUE  IS  literal  .
```

Format 2:

```
   88  condition-name;  VALUE   IS   literal   THROUGH   literal  .
                        VALUES  ARE            THRU
```

General Format for Report Group Description Entry

Format 1:

```
   01   data-name

      ;  LINE  NUMBER  IS  integer  ON  NEXT  PAGE
                           PLUS  integer

      ;  NEXT  GROUP  IS  integer
                          PLUS  integer
                          NEXT  PAGE

                    REPORT HEADING
                    RH
                    PAGE HEADING
                    PH
                    CONTROL  HEADING   data-name
                    CH                 FINAL
      ;  TYPE  IS   DETAIL
                    DE
                    CONTROL FOOTING    data-name
                    CF                 FINAL
                    PAGE FOOTING
                    PF
                    REPORT FOOTING
                    RF

      ;  USAGE  IS  DISPLAY  .
```

Format 2:

```
   level-number  data-name

      ;  LINE  NUMBER  IS  integer  ON  NEXT  PAGE
                           PLUS  integer

      ;  USAGE  IS  DISPLAY   .
```

General Format for Report Group Description Entry

Format 3:

```
level-number  data-name

    ;  BLANK  WHEN  ZERO

    ;  GROUP  INDICATE

    ;  JUSTIFIED  RIGHT
       JUST

    ;  LINE  NUMBER  IS  integer  ON  NEXT  PAGE
                         PLUS  integer

    ;  COLUMN  NUMBER  IS  integer

    ;  PICTURE  IS  picture-string
       PIC

    ;  SOURCE  IS  data-name

    ;  VALUE  IS  literal

    ;  SUM  data-name  ,  data-name   ...
         UPON  data-name  ,  data-name   ...    ...
         RESET  ON  data-name
                    FINAL

    ;  USAGE  IS  DISPLAY   .
```

General Format for Screen Group Description Entry

```
level-number  screen-name

   ;  BLANK  SCREEN

   ;  BLANK  LINE

   ;  BELL

   ;  LINE  NUMBER  IS  PLUS  integer

   ;  COLUMN  NUMBER  IS  PLUS  integer
      COL

                                  ;  FROM   data-name
                                                literal
   ;  PICTURE  IS  picture-string  ;  TO     data-name
      PIC                         ;  USING  data-name

   ;  VALUE  IS  literal

   ;  BLANK  WHEN  ZERO

   ;  JUSTIFIED  RIGHT
      JUST

   ;  SECURE

   ;  AUTO

   ;  REQUIRED

   ;  FULL    .
```

General Format for PROCEDURE DIVISION

Format 1:

```
PROCEDURE  DIVISION  USING  data-name  ,  data-name  ...  .

DECLARATIVES.

   section-name  SECTION.   Use Statement
   paragraph-name.  sentence  ...   ...   ...

END DECLARATIVES.

section-name  SECTION.
paragraph-name.  sentence   ...   ...   ...
```

Format 2:

```
PROCEDURE  DIVISION  USING  data-name  ,  data-name  ...  .

paragraph-name.  sentence   ...   ...   ...
```

General Format for Verbs

```
ACCEPT screen-name  ;  ON  ESCAPE  imperative statement


ACCEPT  data-name

   ,  LINE  identifier
            literal

   ,  POSITION  identifier
                literal

   ,  SIZE  identifier
            literal

   ,  PROMPT   ,  ECHO   ,  CONVERT   ,  SECURE



                          DAY
ACCEPT  data-name  FROM   DATE
                          TIME


ADD  data-name   ,  data-name   ...  TO  data-name  ROUNDED
     literal        literal

   ;  ON  SIZE  ERROR  imperative statement


ADD  data-name  ,  data-name   ...  GIVING  data-name  ROUNDED
     literal       literal

   ;  ON  SIZE  ERROR  imperative statement


ADD  CORRESPONDING  data-name  TO  data-name  ROUNDED
     CORR

   ;  ON  SIZE  ERROR  imperative statement


CALL  PROGRAM  data-name  USING  data-name  ,  data-name   ...
               literal

   ;  ON  EXCEPTION  imperative statement
          OVERFLOW

CLOSE  file-name  ,  file-name  ...
```

General Format for Verbs

```
COMPUTE  data-name  ROUNDED  =  arithmetic expression

   ;  ON  SIZE  ERROR  imperative statement


COPY  literal-file-name


DELETE  file-name  RECORD  ;  INVALID  KEY  imperative statement

DELETE  FILE  file-name
              literal

DISPLAY  screen-name  ,  screen-name  ...

DISPLAY  data-name  ,  LINE  data-name  ,  POSITION  data-name
         literal           literal                  literal

   ,  SIZE  data-name
            literal


DIVIDE  data-name  INTO  data-name  ROUNDED
        literal

   ;  ON  SIZE  ERROR  imperative statement


DIVIDE  data-name  INTO  data-name  GIVING  data-name  ROUNDED
        literal          literal

   REMAINDER  data-name  ;  ON  SIZE  ERROR  imperative statement


DIVIDE  data-name  BY  data-name  GIVING  data-name  ROUNDED
        literal        literal

   REMAINDER  data-name  ;  ON  SIZE  ERROR  imperative statement


EXIT.

GENERATE  report-detail-group-name

GO TO  procedure-name

GO TO  procedure-name  ,  procedure-name  ...  DEPENDING ON data-name


IF  condition  ;  statement        ;  ELSE  statement
                 NEXT  SENTENCE       ELSE  NEXT  SENTENCE
```

General Format for Verbs

```
INITIATE  report-name


INSPECT  data-name

                                    ALL         data-name
   TALLYING  ,  data-name  FOR  ,   LEADING     literal
                                    CHARACTERS

      BEFORE            data-name
      AFTER    INITIAL  literal    ...  ...

   REPLACING

                    data-name   BEFORE            data-name
      CHARACTERS  BY literal     AFTER    INITIAL  literal

      ALL
      LEADING  ,  data-name  BY  data-name
      FIRST       literal        literal

         BEFORE            data-name
         AFTER    INITIAL  literal    ...  ...


                       ASCENDING
MERGE  file-name  ON   DESCENDING  KEY  data-name  ,  data-name  ...

                       ASCENDING
                  ON   DESCENDING  KEY  data-name  ,  data-name  ...

   USING  file-name  ,  file-name  ,  file-name  ...

   OUTPUT  PROCEDURE  IS  section-name  THROUGH  section-name
                                        THRU

   GIVING  file-name


MOVE  data-name  TO  data-name
      literal

MOVE CORRESPONDING  data-name  TO  data-name
     CORR
```

General Format for Verbs

```
MULTIPLY  data-name  BY  data-name  ROUNDED
          literal

   ;  ON  SIZE  ERROR  imperative statement

MULTIPLY  data-name  BY  data-name  GIVING  data-name  ROUNDED
          literal        literal

   ;  ON  SIZE  ERROR  imperative statement


      INPUT
OPEN  OUTPUT  file-name  ,  file-name  ...
      I-O
      EXTEND


PERFORM  procedure-name  THROUGH  procedure-name
                         THRU

PERFORM  procedure-name  THROUGH  procedure-name  data-name  TIMES
                         THRU                     literal

PERFORM  procedure-name  THROUGH  procedure-name  UNTIL  condition
                         THRU

PERFORM  procedure-name  THROUGH  procedure-name
                         THRU

   VARYING  data-name  FROM  data-name
                             literal

      BY  data-name  UNTIL  condition
          literal

   AFTER    data-name  FROM  data-name
                             literal

      BY  data-name  UNTIL  condition
          literal

   AFTER    data-name  FROM  data-name
                             literal

      BY  data-name  UNTIL  condition
          literal
```

General Format for Verbs

```
READ  file-name  NEXT  RECORD  WITH  LOCK  INTO  data-name

   ;  AT  END  imperative statement


READ  file-name  RECORD  WITH  LOCK  INTO  data-name

   ;  KEY  IS  data-name

   ;  INVALID  KEY  imperative statement


RELEASE  record-name  FROM  data-name

RETURN  file-name  INTO  data-name  ;  AT  END  imperative statement

REWRITE  record-name  FROM  data-name

   ;  INVALID  KEY  imperative statement

SET  data-name  TO        data-name
                UP    BY  literal
                DOWN  BY

                     ASCENDING
SORT   file-name  ON  DESCENDING  KEY  data-name  ,  data-name  ...

                     ASCENDING
                  ON  DESCENDING  KEY  data-name  ,  data-name  ...

   INPUT  PROCEDURE  IS  section-name  THROUGH  section-name
                                       THRU

   USING  file-name  ,  file-name  ...

   OUTPUT  PROCEDURE  IS  section-name  THROUGH  section-name
                                        THRU

   GIVING  file-name


                           EQUAL TO
                           =
                           GREATER THAN
START  file-name  KEY  IS  >             data-name
                           NOT  LESS
                           NOT  <

   ;  INVALID  KEY  imperative statement


STOP  RUN
```

literal

General Format for Verbs

```
SUBTRACT  data-name  ,  data-name  ...  FROM  data-name  ROUNDED
          literal       literal

   ;  ON  SIZE  ERROR  imperative statement

SUBTRACT  data-name  ,  data-name  ...  FROM  data-name
          literal       literal               literal

   GIVING  data-name  ROUNDED

   ;  ON  SIZE  ERROR  imperative statement


SUBTRACT  CORRESPONDING  data-name  FROM  data-name  ROUNDED
          CORR

   ;  ON  SIZE  ERROR  imperative statement


SUPPRESS  PRINTING

TERMINATE  report-name

UNLOCK file-name  RECORD
                  RECORDS


USE  AFTER  STANDARD  EXCEPTION  PROCEDURE  ON  file-name  ,  file-name  ...
                      ERROR

USE  BEFORE  REPORTING  report-group-name


WRITE  record-name  FROM  data-name


                      data-name  LINE
   BEFORE  ADVANCING  integer    LINES
   AFTER              PAGE

   ;  AT  END-OF-PAGE  imperative statement
          EOP

WRITE  record-name  FROM  data-name

   ;  INVALID  KEY  imperative statement
```

General Format for Conditions

Relation Condition:

```
data-name              IS  NOT  GREATER THAN  data-name
literal                IS  NOT  LESS THAN     literal
arithmetic expression  IS  NOT  EQUAL  TO     arithmetic expression
                       IS  NOT  >
                       IS  NOT  <
                       IS  NOT  =
```

Class Condition:

```
data-name  IS  NOT  NUMERIC
                    ALPHABETIC
```

Sign Condition:

```
                                POSITIVE
arithmetic expression  IS  NOT  NEGATIVE
                                ZERO
```

Negated Simple Condition:

```
NOT  simple condition
```

Combined Condition:

```
condition  AND  condition  ...
           OR
```